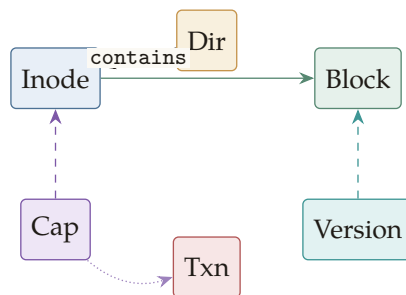


sotFS

un filesystem como grafo

Modelo, reescritura DPO, transacciones, verificación formal,
adaptador FUSE y monitoreo estructural.



Lucas Sotomayor

Versión del libro: 14 de mayo de 2026 • sotFS v0.2.5

sotFS: un filesystem como grafo.

Copyright © 2026 Lucas Sotomayor.

Este libro acompaña al código fuente de el crate `sotFS`, distribuido bajo licencia MIT. La prosa, los diagramas y los ejercicios de este volumen se distribuyen también bajo licencia MIT salvo que se indique otra cosa explícitamente. Las figuras tomadas de fuentes externas (IPFS, Yggdrasil, BTRFS) están citadas individualmente.

Repositorio:

<https://github.com/sotomayorlucas/sotfs>

Compilado con:

L^AT_EX, lualatex, algorithm2e, TikZ, biblatex.

Tipografía: *TeX Gyre Pagella* y *Latin Modern Mono*.

Versión del manuscrito: 14 de mayo de 2026.

Prefacio

Este libro es la presentación pedagógica y autocontenida del sistema **sotFS**: un filesystem cuya estructura interna no es un árbol de inodos, como en ext4 o XFS, sino un grafo dirigido tipado, donde cada operación POSIX se modela como una reescritura algebraica (DPO, *double pushout*) y donde las propiedades de seguridad, consistencia y crash-safety se enuncian como invariantes formales y se verifican parcialmente con TLA⁺ y Coq.

Para quién es este libro

Tres lectores en mente, en orden de comodidad creciente con el material:

- Un *ingeniero de sistemas* con base en filesystems POSIX, storage e idea de qué hace `fsync`, pero sin background previo en teoría de categorías ni verificación formal.
- Un *estudiante de posgrado* en sistemas o métodos formales, interesado en cómo se baja un modelo matemático a una implementación Rust real, con tests, fuzzers, benchmarks y CI.
- Un *practitioner de seguridad* interesado en provenance, monitoreo estructural (curvatura de Ricci-Ollivier sobre el grafo del FS) y proyecciones engañosas como mecanismo defensivo.

Cómo está organizado

Catorce capítulos organizados en tres bloques:

Núcleo (caps. 1–5). Motivación, preliminares matemáticos, el TypeGraph, hashing de contenido y reescritura DPO de todas las operaciones POSIX.

Implementación y persistencia (caps. 6–9). Cómo se baja el modelo a Rust seguro y concurrente; persistencia con REDB; capabilities y delegación; transacciones GTXN y *crash refinement*.

Verificación, monitoreo y exposición (caps. 10–14). Specs TLA⁺, pruebas en Coq con su deuda explícita, métricas estructurales (treewidth y curvatura), el *functor de decepción* D_{FS} y el adaptador FUSE que expone todo al kernel.

Cinco apéndices (catálogo de invariantes, tablas DPO, resultados TLC, setup reproducible, soluciones a ejercicios * y **) cierran el volumen.

Convenciones

- **Cajas tipadas** sirven para destacar material verificable o peligroso. Las *invariantes* (en azul) son propiedades que el sistema mantiene siempre. Las *trampas* (en rojo) son errores históricos del proyecto, con su *lección* explícita; muchas están enlazadas a un commit. Los *ejercicios* (en ámbar) aparecen al final de cada capítulo.
- **Algoritmos** se presentan en pseudo-código estilo CLRS con `algorithm2e`, numerados por capítulo. Su correspondencia con código Rust real se cita por archivo y línea, p. ej. `sotfs-graph/src/graph.rs:571`.
- **Citas al repositorio** usan macros: `formal/spec.tla`, `formal/coq/archivo.v`, el crate nombre y commit sha. Las citas académicas siguen el estilo numérico estándar.

- **Ejercicios graduados.** [*] memorística; [**] aplicación; [* * *] diseño o extensión. El apéndice [E](#) cubre los primeros dos niveles; los [* * *] llevan sólo una pista.

Lo que este libro no hace

- No reemplaza la lectura del código. Documenta y motiva la arquitectura de la versión `v0.2.5`; la fuente de verdad sigue siendo el repositorio.
- No oculta las debilidades formales. Hay cinco lemas `Admitted` en `Coq` (4 en `DpoRmdir.v`, 1 en `DpoUnlink.v`); el cap. [11](#) los lista uno a uno.
- No cubre el `crate sotfs-experimental`: es código aspiracional, no integrado al runtime estable.

Agradecimientos

A las herramientas que hicieron este libro posible: $\text{\LaTeX}$ y `algorithm2e`, `TikZ`, `biblatex`, `TLA Tools`, `coqc` y `coqdoc`, `cargo-fuzz` y `cargo-llvm-cov`. A la comunidad de teoría de categorías aplicada por insistir en que un *pushout* es la cosa más útil que uno aprende en la vida.

Buenos Aires, 14 de mayo de 2026

Notación

Conjuntos y estructuras

Símbolo	Significado
$\mathbb{N}, \mathbb{Z}, \mathbb{R}$	naturales, enteros, reales
$\mathbb{B}$	$\{\perp, \top\}$, valores booleanos
Set	categoría de conjuntos y funciones
Graph	categoría de grafos dirigidos y morfismos
TypeGraph	categoría de grafos <i>tipados</i> (la que vive sotFS)
$G \xrightarrow{f} H$	morfismo de grafos f entre G y H
$A \sqcup_C B$	pushout de A y B a lo largo de C
$ S $	cardinalidad del conjunto S
$\mathcal{P}(S)$	conjunto potencia de S

TypeGraph (capítulo 3)

Símbolo	Significado
Inode	nodo de tipo inodo (metadata POSIX)
Directory	nodo de tipo directorio
Capability	nodo de tipo capability
Transaction	nodo de tipo transaction
Version	nodo de tipo version (snapshot)
Block	nodo de tipo block (datos)
contains, grants, delegates	aristas tipadas
derivedFrom, supersedes, pointsTo, hasXattr	aristas tipadas
G	un <i>TypeGraph</i> concreto (instancia)
$h(x)$	hash de contenido BLAKE3 de x

DPO

Símbolo	Significado
$L \leftarrow K \rightarrow R$	producción DPO con <i>left</i> , <i>glue</i> , <i>right</i>
$m : L \rightarrow G$	<i>matching</i> de la regla en el host G
$G \xrightarrow{\rho, m} H$	paso DPO de G a H vía (ρ, m)
nac	<i>negative application condition</i>

Operaciones POSIX y errno

Las operaciones POSIX se denotan en monospace: `create_file`, `mkdir`, `unlink`, `rename`, `link`, `rmdir`, `write`, `read`. Los errores POSIX se citan como `EEXIST`, `ENOENT`, `ENOTEMPTY`, `EACCES`, `ELOOP`.

Verificación

Símbolo	Significado
TLA ⁺	lenguaje de specs de Lamport [6]
TLC	el model checker de TLA ⁺
Coq	asistente de pruebas [2]
Admitted	lema enunciado pero no probado en Coq
formal/x.tla	cita al spec x.tla en formal/
formal/coq/x.v	cita al archivo x.v en formal/coq/
el crate x	cita al crate x en el workspace
commit sha	cita al commit sha del repo

Cajas

Tipo	Uso
<i>Definición</i>	introducción de un objeto formal
<i>Teorema</i>	resultado central, probado o citado
<i>Lema</i>	resultado auxiliar
<i>Invariante</i>	propiedad mantenida por construcción
<i>Trampa</i>	error histórico del proyecto + lección
<i>Observación</i>	nota lateral, no crítica para la lectura
<i>Ejercicio</i>	propuesta al lector, graduada */**/* * *

Índice general

Prefacio	iii
Notación	v
Índice de figuras	xi
Índice de tablas	xii
Índice de algoritmos	xiii
1 Introducción: por qué un FS basado en grafos	1
1.1 Filesystems como árbol vs. como grafo	1
1.2 Lo que sotFS es y lo que no	2
1.3 Mapa de capítulos	2
1.4 Convenciones del libro	3
1.5 Cómo leer este libro	3
2 Preliminares matemáticos	5
2.1 Grafos dirigidos tipados	5
2.2 Funciones de hash criptográfico	6
2.3 Pushouts	6
2.4 Reescritura DPO (<i>double-pushout</i>)	7
2.5 Por qué DPO y no SPO o ARS	8
2.6 Notación que se usa todo el libro	8
3 El TypeGraph de sotFS	11
3.1 Por qué un grafo tipado y no un árbol	11
3.2 Definición formal del TypeGraph	11
3.3 Los seis tipos de nodo	12
3.4 Los siete tipos de arista	13
3.5 Diagrama de un TypeGraph mínimo	14
3.6 Los siete invariantes verificados tras cada paso	15
3.7 La cara TLA ⁺ del TypeGraph	18
3.8 Resumen del capítulo	19
4 Hashing de contenido y direccionamiento	21
4.1 El hash de un nodo	21
4.2 Identidad por contenido y deduplicación	21
4.3 Verificación incremental	22
4.4 Snapshots gratis	22
4.5 Sharing entre dominios	23
4.6 Comparación con árbol POSIX	23
4.7 Inmutabilidad de nodo	24

5	Reescritura DPO de operaciones POSIX	26
5.1	Repaso: pushout en Set y en Graph	26
5.2	Una regla DPO	26
5.3	Producciones DPO de las operaciones POSIX	27
5.3.1	create_file	27
5.3.2	mkdir	28
5.3.3	link	29
5.3.4	unlink	29
5.3.5	rmdir	30
5.3.6	rename	30
5.3.7	write	31
5.4	Gluing condition vs. invariantes	31
5.5	Algoritmo genérico de aplicación de una regla DPO	32
5.6	Trampas históricas	32
5.7	Resumen del capítulo	33
6	Implementación: arenas, RCU y motor de reglas	36
6.1	Arenas: asignación densa con reuso	36
6.2	Cómo se usa la arena en TypeGraph	37
6.3	RCU: lectores lock-free, escritor único	37
6.4	El motor de reglas: cómo se aplica un paso DPO	38
6.5	Provenance log: el sidecar opcional	39
7	Persistencia: redb e índices secundarios	41
7.1	El backend redb	41
7.2	Save y load	41
7.3	El truco: rehidratación de índices secundarios	42
7.4	El comando sotfscctl	42
7.5	Crash safety: lo que da redb	42
8	Capabilities y delegación	45
8.1	El nodo Capability	45
8.2	El invariante central: monotonicidad	45
8.3	Acciones formales en el spec TLA	46
8.4	El CapContext: cap-mediated context	46
8.5	Algoritmo de chequeo de acceso	47
8.6	Revocación y el rol del epoch	47
9	Transacciones y crash refinement	49
9.1	Modelo formal vs. implementación	49
9.2	La GTXN del runtime	50
9.3	El protocolo abstracto del modelo TLA: GTXN con WAL	51
9.4	Crash refinement	51
9.5	Las propiedades verificadas	52
9.6	El protocolo de fsync	52
9.7	Trampas históricas	53
9.8	La invariante de <i>readers see committed-or-pre-tx</i>	53
9.9	Origen del diseño	54

9.10	Resumen del capítulo	54
10	Verificación con TLA⁺	57
10.1	La biblioteca de specs	57
10.2	Anatomía de un spec: sotfs_graph	57
10.3	Cómo correr TLC	59
10.4	Cómo leer un contraejemplo de TLC	60
10.5	Crash refinement: el spec más sofisticado	60
10.6	Status TLC v0.2.5	60
10.7	Lo que TLA ⁺ no prueba	60
11	Verificación con Coq	63
11.1	El modelo Records: SotfsGraph.v	63
11.2	Los teoremas de preservación	64
11.3	Deuda formal: los tres Admitted	65
11.4	Cómo correr las pruebas Coq	66
11.5	Lo que Coq prueba (y lo que no)	66
12	Topología del grafo: treewidth y curvatura	69
12.1	Por qué importan estas métricas en un FS	69
12.2	Treewidth: definición y aproximación	69
12.3	Curvatura de Ollivier-Ricci	70
12.4	Detección de anomalías	71
12.5	El spec TLA de curvatura	72
13	Decepción: el functor de proyección	74
13.1	Decepción como functor	74
13.2	Los cuatro modos de proyección	74
13.3	Lemma de well-formedness	74
13.4	Detectabilidad por timing	75
13.5	Honeypots: perfiles pre-built	75
14	Adaptador FUSE y observabilidad	77
14.1	El callback FUSE: anatomía	77
14.2	Mapeo VFS → DPO	77
14.3	El CapContext desde una syscall	77
14.4	Provenance log: el sidecar JSONL	77
14.5	Graph Hunter: exporter para threat-hunting	78
14.6	Mount: opciones runtime	79
14.7	Resumen del capítulo	79
A	Catálogo completo de invariantes	81
B	Tablas DPO por operación POSIX	83
C	Resultados de model checking con TLC	86
D	Setup reproducible	88

E Soluciones a ejercicios seleccionados	91
Bibliografía	99

Índice de figuras

2.1	Diagrama universal del pushout. D es el “más pequeño” objeto que cierra el cuadrado conmutativo: cualquier otro D' que lo cierre factoriza por D via u	6
3.1	Un TypeGraph well-formed mínimo con dos directorios, dos archivos compartiendo el bloque B2, y una cap autorizando lectura de I3. Las aristas contains son sólidas (negro); las pointsTo verdes; las grants violetas dashed.	15
4.1	Sharing intrínseco: file1.txt y file2.txt comparten el bloque B2 por construcción porque ambos lo producen con el mismo hash. No es un hard link; es la semántica natural del Merkle DAG. El refcount de B2 es 2.	22
5.1	Diagrama de pushout. D es el menor objeto que cierra el cuadrado conmutativo.	26
5.2	Diagrama de un paso DPO: dos cuadrados pushout (PO) compartiendo el lado central $K \rightarrow D$. El de la izquierda <i>borra</i> lo que está en $L \setminus K$; el de la derecha <i>inserta</i> lo que está en $R \setminus K$	27

Índice de cuadros

1.1	Mapa de capítulos y dependencias estrictas para una lectura lineal. Los apéndices son lookup, no lectura lineal.	2
3.1	Atributos de cada tipo de nodo. Definidos en <code>sotfs-graph/src/types.rs</code> : 91 (Inode), :142 (Directory), :174 (Capability), :227 (Transaction), :238 (Version), :246 (Block).	12
3.2	Signatura de las siete aristas. La columna <i>Origen</i> y <i>Destino</i> fijan los tipos admisibles; un par fuera de la signatura viola el <code>TypeInvariant</code> . El <code>XAttr</code> aparece como séptimo tipo de nodo en algunas vistas; en este libro lo tratamos como una variante de <code>Inode</code> para mantener el conteo en seis. La definición canónica vive en <code>sotfs-graph/src/types.rs</code> : 346.	14
3.3	Costo de cada chequeo. [†] El worst-case $O(N^3)$ del chequeo de ciclos viene de un patrón patológico de cadenas de <code>mkdir</code> profundas; está documentado y mitigable con un índice adicional. Cf. trampa 3.1.	17
4.1	Tradeoffs de árbol POSIX vs. Merkle DAG.	23
9.1	Modelo formal vs. implementación. El modelo describe un WAL explícito; el runtime delega la durabilidad al backend <code>redb</code> . La correspondencia es por <i>inspección humana</i> , no por <i>extracción mecánica</i> ; cap. 10 discute las consecuencias de esa elección.	49
10.1	Los seis specs TLA^+ de <code>sotFS</code> y la propiedad que cada uno verifica.	57
11.1	Los seis archivos de prueba DPO. Para cada uno, el teorema final es de la forma “ $\forall g, \text{precond} \Rightarrow \text{WellFormed}(g) \Rightarrow \text{WellFormed}(\text{op } g)$ ”.	64
14.1	Cada VFS callback de FUSE se traduce en una llamada a una o más funciones DPO de el crate <code>sotfs-ops</code>	78

Índice de algoritmos

1	CHECKINVARIANTS	17
2	NODEHASH	21
3	APPLYDPO	32
4	CHECKGLUING	32
5	ARENA.ALLOC	37
6	RCUGRAPH.WRITE	38
7	ESQUELETO DE CADA OP DPO EN SOTFS-OPS	39
8	BACKEND.SAVE	41
9	BACKEND.LOAD	41
10	GRAPH.REBUILDDirNameIdx	42
11	CHECKACCESS	47
12	WITHTRANSACTION	50
13	GTXNCOMMIT (modelo formal)	51
14	GREEDYMINDEGREE TREewidth	70
15	OLLIVIERRICCILLY	71
16	SKELETON DE UN CALLBACK FUSE EN SOTFS	77

Introducción: por qué un FS basado en grafos

Este libro es la presentación pedagógica del sistema sotFS: un filesystem cuya estructura interna no es un árbol de inodes, como en ext4 o XFS, sino un *grafo dirigido tipado* sobre el que cada operación POSIX se ejecuta como una reescritura algebraica. Antes de entrar en formalismos, conviene ver por qué la elección estructural importa.

1.1 Filesystems como árbol vs. como grafo

El árbol POSIX clásico. ext4 y compañía modelan los datos como un árbol jerárquico: directorios contienen inodes, inodes apuntan a bloques. Es una decisión simple, eficiente y bien entendida. Tres consecuencias se derivan de esa simplicidad:

- Los *hard links* no son una propiedad del modelo; son un bolt-on. Dos paths que apuntan al mismo inode existen, pero el modelo subyacente no los refleja como “misma cosa con dos nombres”.
- La *deduplicación* post-escritura es cara: hay que rescanear bloques iguales y reemplazar referencias.
- Los *snapshots* se hacen vía copy-on-write o *shadow paging*; son técnicas de implementación, no propiedades del modelo.

El grafo tipado. sotFS usa una estructura distinta: un *grafo dirigido tipado* (TypeGraph) con seis tipos de nodo y siete tipos de arista. Lo que en el árbol son convenciones aparece como relaciones formales:

- **Sharing entre dominios.** Un mismo bloque referenciado desde dos archivos sin bolt-on; emerge naturalmente del modelo.
- **Snapshots gratis.** Un snapshot es un puntero a un nodo raíz en cierto momento; nada se copia.
- **Capabilities first-class.** Una capability es un nodo del grafo, no una entrada en una tabla paralela.
- **Operaciones como reescritura.** Cada `create_file`, `mkdir`, `unlink`, etc. es una regla DPO sobre el grafo.

El precio. Mayor complejidad teórica. Un grafo tipado tiene invariantes que un árbol no necesita; las operaciones tienen *gluing conditions* formales; los chequeos son menos triviales. El libro es básicamente la presentación ordenada de ese costo y de las herramientas (TLA⁺, Coq, fuzzers) para mantenerlo bajo control.

1.2 Lo que sotFS es y lo que no

Lo que es.

- Un filesystem POSIX-compatible que monta en Linux vía FUSE.
- Un substrato algebraico (**TypeGraph**) sobre el que se prueba la preservación de invariantes en TLA⁺ y Coq.
- Un workspace Rust de ocho crates (cap. 6 los recorre).
- Una herramienta de seguridad ofensiva-defensiva: provenance log con cap-mediated context, métricas estructurales (treewidth, curvatura), y functor de decepción D_{FS} (caps. 12 y 13).

Lo que no es (todavía).

- Un filesystem listo para producción a gran escala. La versión cubierta es v0.2.5; el roadmap planea Tier-2 (multi-host 2PC), `sotfsctl repair`, y suite `xfstests/LTP`.
- Un sistema con prueba formal mecanizada *end-to-end*. Hay cinco lemmas Admitted en Coq que el cap. 11 documenta uno por uno. La correspondencia DPO matemático $\rightarrow$ Rust es por inspección, no por extracción certificada.
- Un reemplazo del crate `sotfs-experimental`: ese crate es código aspiracional, no integrado al runtime estable, y queda *fuera* de este libro.

1.3 Mapa de capítulos

La Tabla 1.1 resume el contenido y las dependencias.

Cap.	Contenido	Depende de
1	Introducción y motivación.	—
2	Preliminares: grafos tipados, pushouts, hashing.	1
3	El TypeGraph, los seis nodos y siete aristas, los siete invariantes.	2
4	Hashing de contenido (BLAKE3) y direccionamiento.	3
5	Reescritura DPO de las operaciones POSIX.	3, 4
6	Implementación Rust: arenas, RCU, motor de reglas.	5
7	Persistencia con REDB e índices secundarios.	6
8	Capabilities y delegación.	3, 7
9	GTXN y crash refinement.	6, 7
10	Verificación con TLA ⁺ .	5, 9
11	Verificación con Coq, deuda Admitted explícita.	5, 10
12	Treewidth y curvatura Ollivier-Ricci.	3
13	El functor de decepción D_{FS} .	3, 8
14	Adaptador FUSE y observabilidad.	6, 8, 9

Cuadro 1.1: Mapa de capítulos y dependencias estrictas para una lectura lineal. Los apéndices son lookup, no lectura lineal.

1.4 Convenciones del libro

Cajas tipadas.

- *Definición* (violeta): introduce un objeto formal.
- *Teorema, Lema* (verde, teal): resultado central / auxiliar.
- *Invariante* (azul): propiedad que el sistema mantiene siempre, a veces probada en TLA⁺ o Coq.
- *Trampa* (rojo): error histórico del proyecto, con su *lección* explícita; muchas enlazan a un commit (commit <sha>).
- *Observación* (gris): nota lateral, no crítica.
- *Ejercicio* (ámbar): propuesta al lector, graduada [*] / [**] / [* * *].

Algoritmos. En pseudocódigo estilo CLRS con `algorithm2e`, numerados por capítulo. Cada algoritmo en el libro tiene su contrapartida en código Rust real, citada por archivo y línea: `sotfs-graph/src/graph.rs:571`.

Citas al repositorio. Las macros `formal/spec.tla` (specs TLA⁺), `formal/coq/archivo.v` (pruebas Coq), el crate nombre (crates Rust del workspace) y `commit sha` (commits) puntúan referencias al código fuente. Las citas académicas siguen el estilo numérico estándar y aparecen en la bibliografía al final del volumen.

1.5 Cómo leer este libro

Lectura lineal. Para alguien que quiere entender el sistema de cero, leer en orden 1 a 14 es lo correcto. Cada capítulo descansa sobre el anterior y los caps. 3 y 5 son los más densos: si en algún momento te perdés en un capítulo posterior, volvé a esos.

Lectura selectiva.

Implementadores en Rust: caps. 3, 5, 6, 7 y los apéndices A (catálogo de invariantes) y D (setup reproducible).

Verificacionistas: caps. 5, 10, 11 y los apéndices B (tablas DPO) y C (resultados TLC).

Practitioners de seguridad: caps. 8, 12, 13, 14.

Educadores: cualquier capítulo, con los ejercicios [*] y [**]; el apéndice E cubre las soluciones.

Resumen del capítulo

- sotFS abandona el árbol POSIX y modela su metadata como un grafo tipado con seis nodos y siete aristas.
- La elección habilita sharing, snapshots y capabilities como propiedades del modelo, no como bolt-ons.
- El precio se paga en complejidad teórica: invariantes, reescritura DPO, verificación.
- El libro recorre las catorce capas de ese costo, en orden pedagógico, con ejercicios graduados y apéndices de lookup.

Ejercicios

Ejercicio 1.1 [*]

Liste los seis tipos de nodo del TypeGraph mencionados en este capítulo. (El cap. 3 los desarrolla.)

Ejercicio 1.2 [*]

Tres consecuencias del modelo de árbol POSIX se mencionan como “costos”. Listelas en una línea cada una.

Ejercicio 1.3 [**]

En el árbol POSIX, ¿cómo se implementan las hard links? ¿Cómo cambia el mecanismo en sotFS según lo que adelanta este capítulo?

Ejercicio 1.4 [**]

Dé un ejemplo concreto donde el sharing intrínseco de un Merkle DAG ahorraría espacio respecto de ext4. ¿Cuántos bloques se ahorrarían si dos archivos de 1 MiB tienen 80% de contenido en común?

Ejercicio 1.5 [* * *]

Liste tres propiedades formales que sotFS apunta a probar y que ext4 *no* prueba. Para cada una, anticipá qué herramienta (TLA⁺, Coq, o test/fuzz) sería la natural para verificarla. (El cap. 10 y 11 desarrollan esto.)

Preliminares matemáticos

Este capítulo es la caja de herramientas que el resto del libro usa sin volver a justificar. Cubre tres temas: (i) grafos dirigidos tipados, la estructura sobre la que vive `sotFS`; (ii) funciones de hash criptográfico, pieza central del direccionamiento por contenido; y (iii) pushouts y reescritura DPO, el formalismo en el que se enuncian las operaciones POSIX.

Si el lector ya viene con experiencia en estos temas, puede saltarse este capítulo y volver a él como referencia. El estilo es deliberadamente operativo: definiciones precisas, ejemplos chicos, sin esoterismo categórico más allá de lo necesario.

2.1 Grafos dirigidos tipados

Definición 2.1 (*Grafo dirigido*)

Un *grafo dirigido* es una tupla $G = (V, E, \text{src}, \text{tgt})$ donde V y E son conjuntos finitos y $\text{src}, \text{tgt} : E \rightarrow V$ son funciones que asignan a cada arista su origen y destino.

Definición 2.2 (*Morfismo de grafos*)

Un *morfismo de grafos* $f : G \rightarrow H$ es un par de funciones $f_V : V_G \rightarrow V_H$ y $f_E : E_G \rightarrow E_H$ tales que $f_V \circ \text{src}_G = \text{src}_H \circ f_E$ y análogamente para tgt . Los morfismos preservan la estructura de adyacencia.

Definición 2.3 (*Grafo tipado*)

Un *grafo tipado* sobre una signatura (T_V, T_E, σ) es un grafo dirigido más dos funciones $\tau_V : V \rightarrow T_V$ y $\tau_E : E \rightarrow T_E$ que respetan σ : para toda $e \in E$, $\sigma(\tau_E(e)) = (\tau_V(\text{src}(e)), \tau_V(\text{tgt}(e)))$. La signatura $\sigma : T_E \rightarrow T_V \times T_V$ fija qué pares de tipos puede unir cada tipo de arista.

La categoría de los grafos tipados sobre una signatura fija se denota **TypeGraph**. Sus objetos son grafos tipados; sus morfismos son los morfismos de grafos que además preservan los tipos ($\tau_V \circ f_V = \tau'_V$ y análogo para τ_E).

Observación (*TypeGraph es completa y co-completa*)

Para los efectos del libro alcanza con un hecho: **TypeGraph** admite todos los pushouts (constructibles componente a componente: la construcción en **Set**, una vez en V y otra vez en E , respeta los tipos por construcción). Esto es lo que permite definir reescritura DPO sobre grafos tipados sin sutilezas adicionales.

2.2 Funciones de hash criptográfico

Definición 2.4 (*Función de hash criptográfico*)

Una función $h : \{0, 1\}^* \rightarrow \{0, 1\}^n$ es una *función de hash criptográfico* si satisface las siguientes propiedades:

Resistencia a pre-imagen. Dado $y \in \{0, 1\}^n$, encontrar x tal que $h(x) = y$ requiere esfuerzo $\Theta(2^n)$.

Resistencia a segunda pre-imagen. Dado x_1 , encontrar $x_2 \neq x_1$ con $h(x_1) = h(x_2)$ requiere esfuerzo $\Theta(2^n)$.

Resistencia a colisión. Encontrar cualquier par $x_1 \neq x_2$ con $h(x_1) = h(x_2)$ requiere esfuerzo $\Theta(2^{n/2})$ (cota del paradoja del cumpleaños).

La elección de sotFS: BLAKE3. sotFS usa BLAKE3 [1] con $n = 256$ bits. La probabilidad de colisión accidental entre dos contenidos distintos es $\sim 2^{-128}$, valor menor que el de un error cósmico no detectado en una RAM ECC. BLAKE3 además es muy rápido (6 GiB/s en una CPU moderna single-thread) y paraleliza naturalmente sobre la entrada vía un Merkle tree interno: importante para el direccionamiento por contenido del cap. 4.

2.3 Pushouts

En Set

Definición 2.5 (*Pushout en Set*)

Dadas dos funciones $f : A \rightarrow B$ y $g : A \rightarrow C$ con dominio común A , su *pushout* es un objeto D junto con dos funciones $h : B \rightarrow D$ y $k : C \rightarrow D$ tales que $h \circ f = k \circ g$ y, para cualquier otro D' con flechas h', k' que satisfagan la misma ecuación, existe una única flecha $u : D \rightarrow D'$ con $u \circ h = h'$ y $u \circ k = k'$.

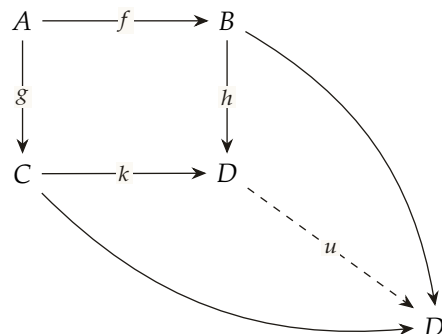


Figura 2.1: Diagrama universal del pushout. D es el “más pequeño” objeto que cierra el cuadrado conmutativo: cualquier otro D' que lo cierre factoriza por D via u .

Teorema 2.1 (Construcción del pushout en Set)

D existe siempre y es único salvo isomorfismo. Concretamente, $D = (B \sqcup C) / \sim$ donde $\sim$ es la relación de equivalencia generada por $f(a) \sim g(a)$ para todo $a \in A$.

Esquema. La existencia se verifica construyendo D como en el enunciado y chequeando la propiedad universal directamente. La unicidad sale del hecho de que cualquier otro D' con la misma propiedad universal admite una flecha $u : D \rightarrow D'$ y una $u' : D' \rightarrow D$, ambas factorizando las inclusiones, y la unicidad de u, u' obliga $u' \circ u = \text{id}_D$. $\square$

En Graph y TypeGraph

La construcción del pushout en **Graph** es la misma idea, pero duplicada: una vez en el conjunto de vértices y otra en el de aristas. Se puede hacer porque las funciones $\text{src}, \text{tgt} : E \rightarrow V$ son funciones de conjuntos, y los morfismos de grafos las preservan. En **TypeGraph** además los tipos se preservan por construcción si la signatura σ es respetada por los morfismos de partida.

2.4 Reescritura DPO (*double-pushout*)

La reescritura DPO es la herramienta para describir transformaciones locales de un grafo de manera *algebraica*: una regla especifica qué borrar, qué dejar y qué insertar; un *paso DPO* aplica la regla a un grafo *host* preservando el resto.

Definición 2.6 (Producción / regla DPO)

Una *producción DPO* es un span en la categoría **TypeGraph**:

$$\rho = L \leftarrow K \rightarrow R$$

donde $K \rightarrow L$ y $K \rightarrow R$ son inclusiones (morfismos inyectivos). L es el patrón a remover; R el patrón a insertar; K el *glue* que sobrevive.

Definición 2.7 (Paso DPO)

Dado un grafo *host* G y un *matching* $m : L \rightarrow G$, un paso DPO $G \xrightarrow{\rho, m} H$ existe si y sólo si:

1. Existe un grafo D y un morfismo $D \rightarrow G$ tal que el cuadrado izquierdo (con $L \rightarrow G$, $K \rightarrow L$ y $K \rightarrow D$) sea un pushout. Este D es el *pushout complement* y existe si y sólo si se cumple la *gluing condition* (def. 2.8).
2. El cuadrado derecho (con $R, K \rightarrow R, K \rightarrow D$) tiene un pushout, que es H .

Definición 2.8 (Gluing condition)

La *gluing condition* para $\rho = L \leftarrow K \rightarrow R$ y *matching* $m : L \rightarrow G$ exige dos cosas:

Identification. Si dos elementos distintos de L (nodos o aristas) se mapean al mismo elemento de G vía m , deben ambos pertenecer a la imagen de $K \rightarrow L$.

Dangling. Para todo nodo $v \in m(L) \setminus m(K)$ (un nodo que la regla quiere borrar), no debe haber aristas en G incidentes a v que no provengan de $m(L)$.

La condición Dangling es la responsable de los errores POSIX que veremos en el cap. 5: “no podés borrar un directorio que todavía tiene entradas”.

Teorema 2.2 (*Existencia y unicidad del paso DPO*)

Dada una producción ρ y un matching m que satisface la gluing condition, existen D y H únicos salvo isomorfismo, y son los construidos vía pushout complement y pushout final.

La prueba completa vive en el clásico [5]; es una aplicación directa de las propiedades universales del pushout.

2.5 Por qué DPO y no SPO o ARS

Existen otros formalismos para reescritura de grafos: SPO (*single-pushout*, [4]), term rewriting con *attribute systems*, sistemas de reescritura abstracta (ARS). ¿Por qué DPO en sotFS?

- DPO requiere la gluing condition explícita; un paso “ilegal” *no existe*, no es “hace algo raro”. En SPO, el paso siempre existe pero puede dejar el grafo en un estado degenerado (aristas colgando que se reinterpretan). DPO es *conservador* en el sentido de Hippocratic: si no se puede hacer bien, no se hace.
- DPO es la formulación más cercana a las pre-condiciones POSIX: EEXIST, ENOTEMPTY y ENOTDIR son instancias literales de gluing conditions o NACs.
- Las pruebas formales en Coq se simplifican porque cada paso requiere un certificado (la gluing condition explícita), lo que hace los teoremas de preservación de invariantes más directos.

2.6 Notación que se usa todo el libro

- G, H : grafos tipados (instancias de **TypeGraph**). G típicamente es un host; H un resultado tras un paso.
- L, K, R : los tres componentes de una producción DPO.
- $m : L \rightarrow G$ un matching.
- $h(x)$: el hash BLAKE3 de x , 256 bits.
- τ_V, τ_E : las funciones de tipado.
- Inode, Directory, Capability, Transaction, Version, Block: los seis tipos de nodo de sotFS (cap. 3).
- contains, grants, delegates, derivedFrom, supersedes, pointsTo, hasXattr: los siete tipos de arista.

Resumen del capítulo

- Un grafo tipado es un grafo dirigido con tipos asignados a nodos y aristas, sujetos a una signatura.

- Una función de hash criptográfico tiene tres propiedades de resistencia; **BLAKE3** con 256 bits es la elección de sotFS.
- Un pushout en **Set** es el cociente de la unión disjunta por la relación inducida por las flechas; en **TypeGraph** se construye componente a componente.
- Una producción DPO es un $\text{span } L \leftarrow K \rightarrow R$; un paso requiere matching y gluing condition; produce un grafo H único salvo isomorfismo.
- DPO se elige sobre SPO porque la gluing condition explícita cuadra con las pre-condiciones POSIX y simplifica las pruebas.

Ejercicios

Ejercicio 2.1 [*]

Defina “morfismo de grafos” en una línea.

Ejercicio 2.2 [*]

¿Cuál es la cota del paradoja del cumpleaños para una función de hash de 256 bits? Expresé el resultado como una potencia de 2.

Ejercicio 2.3 [**]

Calcule el pushout en **Set** de $f : \{a, b\} \rightarrow \{1, 2, 3\}$ con $f(a) = 1, f(b) = 2$, y $g : \{a, b\} \rightarrow \{x, y, z\}$ con $g(a) = x, g(b) = y$. Liste explícitamente los elementos de D y las funciones h, k .

Ejercicio 2.4 [**]

Dé un ejemplo de producción DPO sobre **Graph** (sin tipar) que añada una arista entre dos nodos existentes. Especifique L, K, R con diagramas o conjuntos.

Ejercicio 2.5 [**]

Construya un caso donde la gluing condition Dangling falle: un host G con tres nodos y dos aristas, un matching m que selecciona uno de los nodos para borrarlo, y la arista incidente sobreviviente que hace fallar la condición.

Ejercicio 2.6 [**]

Dé un ejemplo en sotFS donde la condición Identification de la gluing condition es relevante. Pista: ¿qué pasa si una regla DPO mapea dos Directory distintos al mismo Directory del host?

Ejercicio 2.7 [**]

Compare DPO y SPO en términos de qué pasa con un `unlink` de un inode hard-linked múltiples veces.

Ejercicio 2.8 [* * *]

Demuestre que el pushout complement, cuando existe, es único salvo isomorfismo. Pista: aplicar la propiedad universal del pushout izquierdo dos veces.

Ejercicio 2.9 [* * *]

Lea el cap. 3 de [5]. Encuentre dos teoremas (con número y página) que justifiquen por qué **TypeGraph** es una categoría adecuada para reescritura DPO.

El TypeGraph de sotFS

Antes de hablar de operaciones, hace falta fijar la ontología sobre la que esas operaciones reescriben. Este capítulo define el *TypeGraph* de sotFS: la estructura de datos sobre la que el filesystem entero se construye. Después de leerlo, el lector tiene las definiciones formales de los seis tipos de nodo, las siete aristas tipadas y los siete invariantes que el runtime verifica tras cada reescritura.

Este capítulo y el capítulo 5 son los más densos del libro: si algún día hay que reabrir un único capítulo para entender qué objetos viven en el grafo, es éste.

3.1 Por qué un grafo tipado y no un árbol

Un filesystem POSIX clásico (ext4, XFS, NTFS) modela su metadata como un *árbol de inodes*: los directorios son nodos internos, los archivos son hojas, y la única manera de *tener dos nombres* para un inode es el bolt-on llamado *hard link*, que rompe la disciplina arbórea.

sotFS abandona el árbol y modela todo como un *grafo dirigido tipado*. Más concretamente:

- Los *vértices* están particionados en seis tipos: inodes, directorios, capabilities, transacciones, versiones y bloques. Cada tipo tiene su propio conjunto de atributos.
- Las *aristas* están particionadas en siete tipos (contains, grants, delegates, derivedFrom, supersedes, pointsTo, hasXattr); cada arista lleva una etiqueta o atributos propios (un nombre de archivo, un bitmask de derechos, un offset).
- Cada operación POSIX se ejecuta como una *reescritura* sobre el grafo, sujeta a un conjunto fijo de *invariantes estructurales* que el sistema verifica después de cada paso.

La elección no es estética. Permite codificar como *aristas tipadas* mecanismos que el árbol clásico tiene que codificar como *convenciones fuera de banda*: hard links, snapshots, capabilities, dependencias de transacciones. Cuando una propiedad de seguridad o de consistencia se puede expresar como “ningún ciclo en cierta sub-relación” o “el *rights* del hijo es subconjunto del del padre”, tener la relación como una arista en un grafo tipado significa que el mecanismo de verificación es uniforme: el chequeador de invariantes recorre el grafo, no inspecciona estructuras heterogéneas.

3.2 Definición formal del TypeGraph

Definición 3.1 (*TypeGraph*)

Un *TypeGraph* es una séxtupla

$$G = (V, E, \text{src}, \text{tgt}, \tau_V, \tau_E, \text{attr})$$

donde:

- V es un conjunto finito de vértices.

- E es un conjunto finito de aristas dirigidas.
- $\text{src} : E \rightarrow V$ y $\text{tgt} : E \rightarrow V$ asignan extremos a cada arista.
- $\tau_V : V \rightarrow T_V$ asigna a cada vértice uno de los seis tipos

$$T_V = \{\text{Inode}, \text{Directory}, \text{Capability}, \text{Transaction}, \text{Version}, \text{Block}\}.$$

- $\tau_E : E \rightarrow T_E$ asigna a cada arista uno de los siete tipos

$$T_E = \{\text{contains}, \text{grants}, \text{delegates}, \text{derivedFrom}, \text{supersedes}, \text{pointsTo}, \text{hasXattr}\}.$$

- attr asigna a cada vértice y cada arista los atributos propios de su tipo (definidos en las secciones 3.3 y 3.4).

Los pares $(\tau_E, (\tau_V, \tau_V))$ admisibles están restringidos por una *signatura* fija σ (Tabla 3.2): no toda arista puede unir cualquier par de tipos.

Observación (*Implementación en el runtime*)

La séxtupla anterior se materializa en Rust como la struct `TypeGraph` definida en `sotfs-graph/src/graph.rs:52`. Los conjuntos V se almacenan en *arenas* tipadas (una por tipo de vértice) con *free-list* de slots reusables; el conjunto E vive en una sola arena de aristas. Los índices auxiliares (`dir_contains`, `cap_parent`, `dir_name_idx`, ...) son derivados, no parte del estado canónico, y se reconstruyen tras una deserialización (cf. cap. 7).

3.3 Los seis tipos de nodo

Definición 3.2 (*Inode, Directory, Capability, Transaction, Version, Block*)

Cada tipo de nodo tiene los atributos de la Tabla 3.1.

Tipo	Atributos
Inode	<code>id</code> , <code>vtype</code> , <code>permissions</code> , <code>uid</code> , <code>gid</code> , <code>size</code> , <code>link_count</code> , <code>ctime</code> , <code>mtime</code> , <code>atime</code>
Directory	<code>id</code> , <code>inode_id</code> (el inodo que "." apunta)
Capability	<code>id</code> , <code>rights</code> (bitmask u8), <code>epoch</code> (generación)
Transaction	<code>id</code> , <code>tier</code> , <code>state</code> , <code>read_set</code> , <code>write_set</code> , <code>begin_ts</code>
Version	<code>id</code> , <code>timestamp</code> , <code>root_inode_id</code>
Block	<code>id</code> , <code>sector_start</code> , <code>sector_count</code> , <code>refcount</code>

Cuadro 3.1: Atributos de cada tipo de nodo. Definidos en `sotfs-graph/src/types.rs:91` (Inode), :142 (Directory), :174 (Capability), :227 (Transaction), :238 (Version), :246 (Block).

Notas sobre cada tipo

Inode. Es la metadata POSIX clásica. Notar que el contenido de archivos no vive en el inode: vive en *nodos* Block a los que el inode apunta vía aristas `pointsTo`. `link_count` es la cuenta de hard links, sostenida como invariante por el sistema (cf. sec. 3.6).

Directory. Lleva sólo dos campos: su `id` y el `inode_id` del inodo que el “.” del directorio apunta. La metadata del directorio (permisos, owner) vive en su Inode pareado, no en el Directory mismo. Esta separación es deliberada: el Directory es el *namespace* (un conjunto de aristas `contains` salientes), y el Inode es la *metadata*.

Capability. Una *capability* es un nodo de primera clase, no una bolt-on de POSIX. Cada cap tiene un `rights` de 8 bits (READ, WRITE, EXECUTE, GRANT, REVOKE) y un `epoch` para invalidación. El cap. 8 desarrolla el modelo completo.

Transaction. Una transacción es un nodo. Tiene estado (Active, Preparing, Committed, Aborted), *tier* (Tier 0/1/2 mapean a las jerarquías de SOT) y dos conjuntos `read_set/write_set` usados para la detección de conflictos.

Version. Una versión es un *snapshot pointer*: un timestamp y un `root_inode_id`. La cadena `derivedFrom` entre versiones forma el linaje de snapshots; es DAG, no árbol.

Block. Un bloque de datos es un *extent* con `sector_start`, `sector_count` y `refcount`. El `refcount` es la cuenta de inodes que lo apuntan vía `pointsTo`; es crítico para el GC del backend de almacenamiento (cap. 7).

3.4 Los siete tipos de arista

Definición 3.3 (*Aristas tipadas y signatura*)

Cada arista pertenece a uno de siete tipos. Su signatura $\tau_E \rightarrow \tau_V \times \tau_V$ y atributos propios están en la Tabla 3.2.

Por qué siete y no menos

Cada arista codifica una relación que en un FS POSIX clásico está *disuelta* en convenciones, en estructuras separadas o en *flags*. Cuando se empuja la relación a una arista tipada, dos cosas mejoran:

- La verificación es uniforme. “No hay ciclos vía delegates” es el mismo tipo de propiedad que “No hay ciclos vía contains excluyendo . .”. Ambas se chequean con el mismo molde.
- La extensión es modular. Un futuro tipo de arista (digamos `auditedBy`: Inode a un nodo de auditoría) se suma a la signatura sin reescribir todo.

Tipo	Origen	Destino	Atributos / sentido
contains	Directory	Inode	name: nombre POSIX bajo el que el inodo aparece en el dir.
grants	Capability	Inode	rights: subconjunto de los rights de la cap.
delegates	Capability	Capability	forma el árbol CDT (<i>capability derivation tree</i>).
derivedFrom	Version	Version	linaje de snapshots; DAG.
supersedes	Inode	Inode	reemplazo atómico durante rename.
pointsTo	Inode	Block	offset: posición del bloque dentro del archivo.
hasXattr	Inode	Inode(XAttr)	cada xattr es un nodo de primera clase.

Cuadro 3.2: Signatura de las siete aristas. La columna *Origen* y *Destino* fijan los tipos admisibles; un par fuera de la signatura viola el *TypeInvariant*. El *XAttr* aparece como séptimo tipo de nodo en algunas vistas; en este libro lo tratamos como una variante de *Inode* para mantener el conteo en seis. La definición canónica vive en `sotfs-graph/src/types.rs:346`.

La trampa simétrica es que cada arista nueva amplía la superficie de invariantes que hay que probar y mantener. El cap. 5 muestra que cada operación POSIX se diseña explícitamente para preservar todas las invariantes; el cap. 11 muestra hasta dónde esa preservación está mecanizada hoy y dónde quedan deudas (los cinco *Admitted*).

Observación (*Evolución del modelo*)

La versión preliminar de sotFS (la que vivía como subsistema de sotX, caps. 17–18 de su libro) listaba seis aristas: *contains*, *extent*, *named-as*, *parent*, *captured* y *owns*. La reorganización de la versión v0.2.0 colapsó algunas de esas relaciones y agregó otras: la arista *extent* pasó a ser *pointsTo* con *offset*; *named-as* se fundió con *contains* (el nombre vive como atributo de la arista, no como nodo intermedio); *parent* se suprimió y se deriva por consulta inversa con el índice *dir_for_inode_idx*; *captured* se generalizó a *grants*; *owns* se reemplazó por el mecanismo de *domain_id* en el *CapContext*. Las nuevas *delegates*, *derivedFrom*, *supersedes* y *hasXattr* no existían como aristas: estaban codificadas como bolt-ons (snapshots como copias, xattrs como diccionarios laterales). El libro adopta el modelo del runtime v0.2.5; este recuadro existe sólo para evitar confusión a quien venga con la versión vieja en mano.

3.5 Diagrama de un TypeGraph mínimo

La figura 3.1 muestra un TypeGraph well-formed con un directorio raíz, un subdirectorio, dos archivos que comparten un bloque de datos, y una capability que autoriza acceso a uno de los inodes.

Notar tres cosas que el árbol POSIX no expresaría así:

1. B2 es un único nodo apuntado por dos inodes distintos; en ext4 esa situación requeriría duplicar el bloque (no hay sharing nativo) o hacer reflinks (extensión específica de XFS/btrfs).

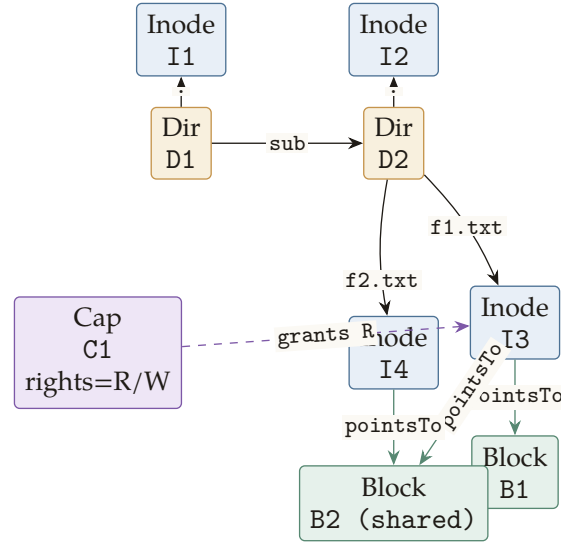


Figura 3.1: Un TypeGraph well-formed mínimo con dos directorios, dos archivos compartiendo el bloque B2, y una cap autorizando lectura de I3. Las aristas contains son sólidas (negro); las pointsTo verdes; las grants violetas dashed.

2. La cap C1 es un nodo, no una entrada en una tabla paralela. La autorización a I3 es la arista grants.
3. Cada Directory tiene un Inode pareado (el inode_id del Directory, alcanzable por la arista "."). Esa pareja es lo que el invariante 3.3 obliga a mantener.

3.6 Los siete invariantes verificados tras cada paso

El runtime expone una función `check_invariants()` (`sotfs-graph/src/graph.rs:874`) que se llama tras cada paso DPO en el camino de tests, en los fuzzers y, opcionalmente, en producción (modo paranoid). Verifica siete invariantes más una consistencia de índice secundario.

Invariante 3.1 (*Link-count consistency, I2 + G3*)

Para todo $i \in \text{Inode}$,

$$i.\text{link_count} = |\{e \in E : \tau_E(e) = \text{contains} \wedge \text{tgt}(e) = i \wedge e.\text{name} \neq ".."\}|.$$

Es decir, la cuenta de hard links almacenada en el inode coincide con la cantidad de aristas contains entrantes, excluyendo las aristas "..". Implementado en `sotfs-graph/src/graph.rs:888`.

Invariante 3.2 (*Unique names per directory, C1 + I4*)

Para todo $d \in \text{Directory}$ y todo par de aristas e_1, e_2 contains que salen de d ,

$$e_1 \neq e_2 \implies e_1.\text{name} \neq e_2.\text{name}.$$

Implementado en `sotfs-graph/src/graph.rs:918`.

Invariante 3.3 (*Dir self-reference, I₃*)

Para todo directorio $d \in \text{Directory}$ existe una arista `contains` desde d con `name = "."` y destino $d.\text{inode_id}$. Es decir, la pareja `Directory/Inode` siempre se cierra sobre sí misma vía la entrada `"."`. Implementado en `sotfs-graph/src/graph.rs:936`.

Invariante 3.4 (*No dangling edges, G₂*)

Para toda arista $e \in E$, $\text{src}(e), \text{tgt}(e) \in V$: ambos extremos son nodos vivos. La consecuencia operativa es que la remoción de un nodo obliga a remover todas sus aristas adyacentes en el mismo paso DPO. Implementado en `sotfs-graph/src/graph.rs:964`.

Invariante 3.5 (*Block refcount, B₁*)

Para todo bloque $b \in \text{Block}$,

$$b.\text{refcount} = |\{e \in E : \tau_E(e) = \text{pointsTo} \wedge \text{tgt}(e) = b\}|.$$

Vital para la corrección del GC en el backend de almacenamiento: si el `refcount` divergiera del conjunto real de aristas `pointsTo`, el GC liberaría bloques aún apuntados. Implementado en `sotfs-graph/src/graph.rs:996`.

Invariante 3.6 (*No directory cycles, A₁*)

La sub-relación inducida por `contains` restringida a directorios (ignorando `"."` y `".."`) es acíclica: el conjunto de directorios alcanzables desde cualquier directorio dado no se vuelve sobre sí mismo. Implementado en `sotfs-graph/src/graph.rs:1015` con DFS desde cada raíz; cf. la trampa 3.2.

Invariante 3.7 (*Capability monotonicity, G₄*)

Para todo par de capabilities $c, c' \in \text{Capability}$ unidas por una arista `delegates` de c a c' ,

$$c'.\text{rights} \subseteq c.\text{rights}.$$

Una delegación nunca crea derechos; sólo restringe los del padre. Implementado en `sotfs-graph/src/graph.rs:1030`.

A las anteriores se suma la consistencia del índice secundario `dir_name_idx` (`sotfs-graph/src/graph.rs`) que no es invariante *semántico* sino correspondencia entre dos representaciones del mismo dato y se chequea aparte.

Algoritmo de verificación

El `check_invariants()` ejecuta los siete chequeos en cascada y aborta al primer fallo, devolviendo un `GraphError::InvariantViolation`.

Algoritmo 1: CHECKINVARIANTS

Entrada: `TypeGraph G`

Salida: `ok` si G es well-formed; error si no.

```

1.1 CHECKLINKCOUNTCONSISTENCY(G);
1.2 CHECKUNIQUENAMES(G);
1.3 CHECKDIRSELFREF(G);
1.4 CHECKNODANGLINGEDGES(G);
1.5 CHECKBLOCKREFCOUNT(G);
1.6 CHECKNODIRCYCLES(G);
1.7 CHECKCAPMONOTONICITY(G);
1.8 CHECKDIRNAMEIDXCONSISTENCY(G);
1.9 retornar ok;

```

Costo

Sea $N = |V|$ y $M = |E|$. La Tabla 3.3 resume el costo de cada chequeo.

Chequeo	Costo	Notas
link-count	$O(N + M)$	barrido lineal de inodes y contains.
unique-names	$O(M \log M)$	BTreeSet por directorio.
dir-self-ref	$O(N \log M)$	lookup por dir.
no-dangling	$O(M)$	barrido de aristas.
block-refcount	$O(N + M)$	barrido de bloques y pointsTo.
no-dir-cycles	$O(N \cdot \text{depth})$ típico, $O(N^3)$ worst-case [†]	DFS por dir.
cap-monotonicity	$O(\text{Capability})$	barrido de aristas delegates.
dir-name-idx	$O(M \log N)$	comparar índice contra contains.

Cuadro 3.3: Costo de cada chequeo. [†] El worst-case $O(N^3)$ del chequeo de ciclos viene de un patrón patológico de cadenas de `mkdir` profundas; está documentado y mitigable con un índice adicional. Cf. trampa 3.1.

Trampa 3.1 (Costo cúbico del cycle check con `deep_mkdir_chain`)

`commit (M4.1.1)`. Versión vulnerable: `dir_for_inode` hacía un barrido lineal sobre `self.dirs.values()` para encontrar el `Directory` cuyo `inode_id` coincidiera con el dado. La función se llama dentro de `is_descendant_of` y `has_cycle_from`, una vez por arista durante un DFS. El loop externo del chequeo `CHECKNODIRCYCLES` itera todos los `dirs` y arranca un DFS desde cada uno: el costo se vuelve $O(N^3)$ en peor caso. La proptest `deep_mkdir_chain_no_cycles` construye cadenas de 20–30 `mkdir` anidados; con 5 cases corre en 0,01 s, con 50 cases pasa de 60 s.

Fix (M4.1.1): un índice `dir_for_inode_idx: BTreeMap<InodeId, DirId>` mantenido junto con `insert_dir` y `remove_dir` reduce `dir_for_inode` a $O(\log N)$ y el chequeo entero a $O(N \cdot \text{depth})$ típico (`sotfs-graph/src/graph.rs:109`).

Lección. Una invariante puede ser *correcta* y el chequeo *insostenible* bajo workload patológico. La proptest sirvió para detectar el problema antes de que produjera un timeout en CI; el fix fue sumar un índice secundario, no relajar la propiedad.

Trampa 3.2 (lookup_name *cuadrático bajo fill hostil*)

`commit (v0.2.0 → v0.2.4)`. Versión vulnerable: `lookup_name(dir, name)` barría linealmente `dir_contains[dir]` comparando el `name` de cada arista. Bajo un workload donde un usuario llena un directorio compartido con N nombres, cada `stat/lookup` contra ese `dir` era $O(N)$; `perf record` mostró `lookup_name + memcmp` a ~92% del CPU del daemon FUSE. Vector de DoS local trivial.

Fix. `dir_name_idx: BTreeMap<(DirId, String), EdgeId>`, mantenido por `insert_edge`, `remove_edge` y `rename_contains_edge`; `lookup_name` pasa a $O(\log N)$ (`sotfs-graph/src/graph.rs:1`).

Lección. Las propiedades estructurales no se imponen solas en runtime: hay que diseñar los índices que las hagan baratas. La verificación de la propiedad “`stat` es flat en tamaño de directorio” es indirecta: el invariante semántico (unicidad de nombres) ya estaba probado, pero el costo de chequearlo o de aprovecharlo para una `lookup` era el cuello de botella.

3.7 La cara TLA⁺ del TypeGraph

El formal `sotfs_graph.tla` (518 líneas) modela el TypeGraph como una máquina de estados sobre los mismos seis tipos de nodo. El estado se expresa con variables como las siguientes:

```
VARIABLES
  inodes,      \* Set of active InodeIds
  dirs,        \* Set of active DirIds
  caps,        \* Set of active CapIds
  blocks,      \* Set of active BlockIds
  contains,    \* Set of <<DirId, Name, InodeId>> triples
  grants,      \* Function: CapId → InodeId
  delegates,   \* Function: CapId → CapId (parent)
  rights       \* Function: CapId → SUBSET RightsAlphabet
```

Listing 3.1: Extracto del estado en `sotfs_graph.tla`.

Las acciones (`Mkdir`, `Unlink`, `Link`, `Rename`, ...) se enuncian con su *gluing condition* a la izquierda del `/\` y la actualización del estado a la derecha. El invariante `TypeOK` restringe los tipos de los elementos en cada conjunto; `NameUniqueness`, `LinkCountConsistent` y `NoDirCycles` son los análogos formales de los invariantes 3.1, 3.2 y 3.6.

El cap. 10 cubre cómo correr TLC sobre el spec, qué configs `small/medium/large` se usan y cómo leer un counterexample.

3.8 Resumen del capítulo

- Un *TypeGraph* es una séxtupla $(V, E, \text{src}, \text{tgt}, \tau_V, \tau_E, \text{attr})$ con seis tipos de nodo y siete tipos de arista restringidos por una signatura fija.
- Cada nodo tiene atributos POSIX-friendly (Tabla 3.1); cada arista lleva un atributo propio (nombre, rights, offset).
- Siete invariantes (link-count, unique-names, dir-self-ref, no-dangling, block-refcount, no-dir-cycles, cap-monotonicity) más la consistencia del índice `dir_name_idx` se verifican tras cada paso DPO. Su algoritmo es `CHECKINVARIANTS` (Alg. 1).
- Dos trampas históricas: el chequeo de ciclos en peor caso $O(N^3)$ (mitigado con `dir_for_inode_idx`) y el `lookup_name` lineal (mitigado con `dir_name_idx`). Ambos invariantes *semánticos* eran correctos antes; los fixes fueron *índices secundarios* para hacer el chequeo y la operación baratos en runtime.
- El *TypeGraph* del runtime y el del formal/`sotfs_graph.tla` se corresponden por construcción: ambos manipulan los mismos seis tipos y las mismas siete relaciones; la verificación TLC explora un espacio de estados acotado y certifica que las acciones preservan los invariantes (cap. 10).

Ejercicios

Ejercicio 3.1 [*]

Enuncie los siete invariantes de la sección 3.6 en una línea cada uno. ¿Cuál depende de un invariante *numérico* (cuenta) y no estructural?

Ejercicio 3.2 [*]

Liste los seis tipos de nodo y los siete tipos de arista. Para cada arista, indique los tipos de origen y destino admisibles según la signatura σ (Tabla 3.2).

Ejercicio 3.3 [**]

Dibuje el *TypeGraph* después de las operaciones `mkdir /a; create /a/x.txt; create /a/y.txt; ln /a/x.txt /a/z.txt`. ¿Cuál es el `link_count` del inodo de `x.txt` después de la última operación? ¿Cuántas aristas `contains` entran al inodo de `x.txt` (excluyendo `..`)?

Ejercicio 3.4 [**]

Demuestre que el invariante 3.1 *implica* que un inodo con `link_count = 0` no tiene aristas `contains` entrantes (salvo `..`). Use solo la definición.

Ejercicio 3.5 [**]

Considere un TypeGraph con dos directorios D_1, D_2 donde $D_1.inode_id = i_1$ y $D_2.inode_id = i_2$. Si una arista contains con `name="."` sale de D_1 y apunta a i_2 ($\neq i_1$), ¿cuál invariante se rompe? ¿En qué línea de `check_invariants` se detecta?

Ejercicio 3.6 [**]

La trampa 3.1 se cierra agregando `dir_for_inode_idx`. Discuta el costo de mantener ese índice *coherente* a lo largo de `insert_dir` y `remove_dir`. ¿Bajo qué patrón de operaciones es más caro mantener el índice que recorrer linealmente?

Ejercicio 3.7 [**]

Una arista delegates de c a c' con $c'.rights \not\subseteq c.rights$ viola el invariante 3.7. Diseñe el test unitario más corto posible que detecte la violación: ¿qué grafo de tres nodos hace falta? ¿qué llamada al runtime?

Ejercicio 3.8 [* * *]

Diseñe un octavo invariante para detectar *leaks de bloques*: un Block cuyo `refcount > 0` pero cuyo conjunto de aristas `pointsTo` entrantes está vacío (caso patológico de divergencia entre contador y aristas). Justifique cuándo lo correrías (¿en cada paso DPO? ¿solo en el GC?). Pista: pensá la asimetría entre el costo de mantener y el de chequear.

Ejercicio 3.9 [* * *]

Extienda la signatura σ con un octavo tipo de arista `auditedBy` (Inode a un nuevo nodo Audit) que registre quién auditó la última escritura. Discuta: ¿qué invariante hay que agregar? ¿qué operaciones POSIX hay que modificar para mantenerlo? ¿qué spec TLA⁺ hay que extender?

Hashing de contenido y direccionamiento

El TypeGraph del cap. 3 fija qué tipos viven en el grafo. Este capítulo agrega la *identidad por contenido*: cada nodo tiene un identificador derivado del hash criptográfico de su contenido serializado. Esa decisión apenas perceptible en la API tiene consecuencias profundas: deduplicación intrínseca, snapshots gratis, verificación incremental, sharing entre dominios sin bolt-on.

4.1 El hash de un nodo

Definición 4.1 (*Content hash*)

Para todo nodo $v \in V$, su *content hash* es

$$h(v) = \text{BLAKE3}(\tau_V(v) \parallel \text{serialize}(v))$$

donde $\parallel$ denota concatenación, $\tau_V(v)$ es un byte-tag que identifica el tipo del nodo (8-bit) y serialize es la serialización canónica del contenido.

La canonicidad de serialize es crítica: dos nodos con el mismo contenido lógico pero distinta serialización se hashean a valores distintos, lo que rompería la deduplicación. el `crate sots-graph` usa una serialización determinista (campos en orden fijo, sin padding, sin reordering) basada en `serde_json` configurado para emitir keys ordenadas. Para los Block en particular, el contenido es la sucesión cruda de bytes, sin metadatos: dos archivos con el mismo contenido producen el mismo `BlockId`.

Algoritmo 2: NODEHASH

Entrada: Nodo v con tipo $\tau_V(v)$ y contenido $\text{content}(v)$

Salida: Hash $h \in \{0, 1\}^{256}$

Algoritmo.

- 2.1 $\text{hasher} \leftarrow \text{BLAKE3}.\text{NEW}();$
- 2.2 $\text{hasher}.\text{UPDATE}(\tau_V(v).\text{TOBYTES}());$
- 2.3 $\text{hasher}.\text{UPDATE}(\text{Serialize}(\text{content}(v)));$
- 2.4 **retornar** $\text{hasher}.\text{FINALIZE}();$

4.2 Identidad por contenido y deduplicación

Invariante 4.1 (*Identidad por contenido*)

Dos nodos con el mismo $h(\cdot)$ son el mismo nodo en el TypeGraph: el filesystem mantiene una sola copia. Identidad estructural implica identidad de almacenamiento.

La consecuencia operativa: dos archivos cuya escritura produce los mismos bloques no duplican espacio. La operación `write` (cap. 5) computa el `BlockId` de cada bloque a escribir; si ya existe en el `crate sotsfs-storage`, simplemente añade una arista `pointsTo` y bumpea el `refcount` del bloque existente.

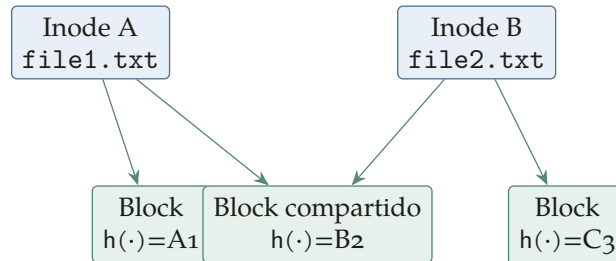


Figura 4.1: Sharing intrínseco: `file1.txt` y `file2.txt` comparten el bloque B2 por construcción porque ambos lo producen con el mismo hash. No es un hard link; es la semántica natural del Merkle DAG. El `refcount` de B2 es 2.

4.3 Verificación incremental

Teorema 4.1 (Verificación cripto-derivada)

Dado el hash de la raíz de un sub-DAG G_R , el contenido de cada nodo del sub-DAG se puede verificar sin material adicional, en tiempo $O(N \cdot \text{cost}(\text{BLAKE3}))$, donde N es la cantidad de nodos en el sub-DAG.

Esquema. Por inducción sobre la profundidad del sub-DAG. La raíz se verifica recomputando $h(r)$ y comparando con el dado. Cada hijo aparece referenciado por su hash dentro del contenido del padre; recomputar $h(\text{hijo})$ y comparar con el referenciado verifica el hijo. La consistencia entre padre y hijo es automática porque la referencia (hash) está en el contenido del padre. $\square$

Esa propiedad es la base de los snapshots: un snapshot es *nada más que* un puntero al hash de un nodo `Version`. Para verificar que el snapshot no fue manipulado, se recomputa el hash del nodo `Version` y se compara.

4.4 Snapshots gratis

Bajo el modelo `content-addressed`, hacer un snapshot de un FS de 1 TiB con 10^6 archivos cuesta exactamente 32 *bytes*: el hash del nodo `Version` que apunta a la raíz. Comparar con `ext4` con LVM-snapshots, donde el costo es proporcional al número de bloques modificados desde el snapshot original (vía COW): $O(\Delta)$ bytes y $O(\Delta)$ overhead de tracking.

La desventaja: garbage collection. Cada snapshot “pin” a un sub-DAG; nodos no alcanzables desde ningún snapshot son candidatos a GC. La implementación es `mark-and-sweep` desde los roots de snapshot; cap. 7 desarrolla los detalles.

Trampa 4.1 (BLAKE3 collision (no observada en práctica))

La invariante 4.1 asume que los hashes son únicos. BLAKE3 con 256 bits tiene resistencia a colisión de 2^{128} (birthday bound); la probabilidad de un ciclo accidental por colisión es *efectivamente cero*. Pero un atacante con poder de construir *colisiones intencionales* (problema abierto en BLAKE3) podría engañar al filesystem para que un nodo se auto-referencie a través de su contenido.

Fix preventivo. Chequeo runtime que un nodo recién insertado no contiene su propio hash entre sus referencias. Costo $O(|\text{children}|)$ por inserción. Reducible si hace falta.

Lección. Las propiedades cripto-derivadas son fuertes pero no infalibles. Un check runtime trivial (anti-self-reference) preserva la invariante incluso bajo collision attack.

4.5 Sharing entre dominios

Definición 4.2 (Dominio)

Un *dominio* es una partición lógica del namespace del filesystem, identificada por un `domain_id` (un u64). Los caps están parametrizados por el `domain_id` del operador (`sotfs-graph/src/types.rs:189`).

Dos dominios distintos pueden referenciar el mismo Block porque el Block se nombra por hash; no hay copia ni protocolo cross-domain. La propiedad emerge del modelo. La consecuencia: dos contenedores distintos en sotFS que comparten la misma imagen base pueden compartir bloques sin sobre costo.

4.6 Comparación con árbol POSIX

	Árbol POSIX (ext4)	Merkle DAG (sotFS)
Identidad	InodeId arbitrario, allocator-dependent	$h(\cdot)$ del contenido
Dedup	manual / scrub	intrínseca
Snapshot	LVM/COW, $O(\Delta)$ bytes	32 bytes
Sharing	hard link, simulink, reflink (extensión)	emerge del modelo
Verificación	no nativa	$O(N \cdot \text{cost}(\text{BLAKE3}))$ recursiva
Mutación	in-place a bloque	inmutable; nuevo nodo, padre actualizado
GC	no necesario (sin sharing)	necesario; mark-and-sweep

Cuadro 4.1: Tradeoffs de árbol POSIX vs. Merkle DAG.

4.7 Inmutabilidad de nodo

Invariante 4.2 (*Inmutabilidad estructural*)

Un nodo nunca se modifica in situ. Una mutación lógica (`write`, `truncate`, `chmod`) crea un nodo nuevo con el contenido nuevo y actualiza el padre. El nodo viejo sobrevive si está referenciado por algún snapshot; si no, queda candidato a GC.

Resumen del capítulo

- Cada nodo del TypeGraph tiene un content hash $h(v) = \text{BLAKE3}(\tau_V(v) \parallel \text{serialize}(v))$, calculado con serialización canónica.
- Identidad por contenido implica deduplicación intrínseca y sharing entre dominios sin bolt-on.
- Verificación incremental: dado el hash raíz, todo el sub-DAG se verifica en $O(N \cdot \text{cost}(\text{BLAKE3}))$.
- Snapshots gratis: 32 bytes para snapshot un FS de 1 TiB.
- Inmutabilidad estructural: las mutaciones crean nodos nuevos, no modifican in-place.
- Trampa: colisiones intencionales de BLAKE3 son un problema abierto; chequeo anti-self-reference como mitigación barata.

Ejercicios

Ejercicio 4.1 [*]

¿Cuál es la longitud en bytes del hash BLAKE3 usado por sotFS? ¿Cuál es la cota de colisión por birthday bound?

Ejercicio 4.2 [*]

Indique tres consecuencias del modelo content-addressed sobre la operación `write` en sotFS.

Ejercicio 4.3 [**]

Dos archivos de 1 MiB tienen 80% de contenido en común, dividido en bloques de 4 KiB. Calcule el espacio total ocupado en sotFS y compare con ext4 sin reflinks.

Ejercicio 4.4 [**]

Argumente por qué la canonicidad de la serialización es crítica. Construya un escenario donde dos serializaciones distintas del mismo contenido lógico romperían la deduplicación.

Ejercicio 4.5 [**]

Diseñe un mecanismo de *garbage collection* para sotFS basado en mark-and-sweep desde los snapshot roots. ¿Cómo manejas writes concurrentes que crean nodos durante el mark? Pista: tri-color marking + write barrier sobre roots.

Reescritura DPO de operaciones POSIX

Este es el capítulo más cargado del libro y el corazón teórico de sotFS. Las operaciones POSIX (`create_file`, `mkdir`, `unlink`, `rename`, `link`, `rmdir`, `write`) se modelan acá como reglas de *double-pushout* (DPO) sobre el `TypeGraph` del cap. 3. La virtud del enfoque DPO no es estética: cada regla viene con condiciones formales (*gluing condition*) cuya verificación es mecánica, y la preservación de los invariantes de sec. 3.6 se sigue del enunciado de la regla, no de un chequeo aislado.

5.1 Repaso: pushout en Set y en Graph

Definición 5.1 (*Pushout en Set*)

Dadas tres funciones $f : A \rightarrow B, g : A \rightarrow C$, su *pushout* es un objeto D junto con dos funciones $h : B \rightarrow D, k : C \rightarrow D$ tales que $h \circ f = k \circ g$ y, para cualquier otro objeto D' con flechas h', k' que cumpla la misma ecuación, existe una única flecha $u : D \rightarrow D'$ que las factoriza.

Concretamente en **Set**, el pushout se construye como el cociente $(B \sqcup C) / \sim$ donde $f(a) \sim g(a)$ para todo $a \in A$. La intuición: A es el “pegamento” que identifica qué elementos de B y C son “el mismo elemento” en el resultado.

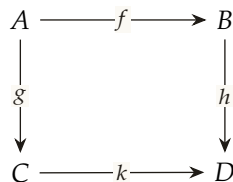


Figura 5.1: Diagrama de pushout. D es el menor objeto que cierra el cuadrado conmutativo.

En la categoría **Graph** (grafos dirigidos y morfismos de grafos) la construcción es análoga: el pushout existe siempre y se construye nodo por nodo, arista por arista, identificando los pares que vienen del “pegamento” A . Este es el sustrato técnico que permite hablar de *reemplazar un fragmento por otro pegándolo a lo que queda*.

5.2 Una regla DPO

Definición 5.2 (*Regla DPO*)

Una *regla DPO* es un span $\rho : L \leftarrow K \rightarrow R$ en la categoría **Graph** (o **TypeGraph**, en nuestro caso). Intuitivamente:

- L es el patrón a remover (*left-hand side*).

- K es la parte *glue* que se conserva tras la transformación.
- R es el patrón a insertar (*right-hand side*).

La inclusión $K \hookrightarrow L$ y $K \hookrightarrow R$ codifica qué sobrevive y qué se reemplaza.

Un *paso DPO* sobre un grafo *host* G requiere:

1. Un *matching* $m : L \rightarrow G$, esto es, un morfismo de grafos que ubica L dentro de G .
2. La *gluing condition*: el matching no debe “dejar aristas colgando” al borrar lo que sólo pertenece a L . Más formalmente, todo nodo de L con aristas adyacentes en G que no estén en $m(L)$ debe estar en K (es decir, debe sobrevivir).
3. Construir el pushout complement D tal que $L \leftarrow K \rightarrow D$ y $L \rightarrow G$ formen un cuadrado pushout.
4. Construir el pushout final H entre D y R a lo largo de K .

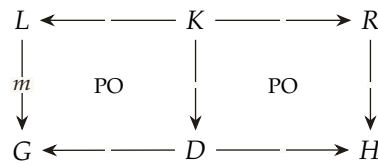


Figura 5.2: Diagrama de un paso DPO: dos cuadrados pushout (PO) compartiendo el lado central $K \rightarrow D$. El de la izquierda *borra* lo que está en $L \setminus K$; el de la derecha *inserta* lo que está en $R \setminus K$.

Observación (En *sotFS* las reglas son funciones Rust, no estructuras explícitas)

El runtime no implementa un motor DPO genérico que tome un span (L, K, R) y haga el pushout complement: implementa cada operación POSIX como una función Rust en el crate *sotfs-ops* que, en orden, (i) ubica el matching (`resolve_name`, `get_dir`, `get_inode`); (ii) chequea la *gluing condition* con *pre-condiciones* explícitas; (iii) muta el grafo para producir H . La equivalencia con la formulación categórica es por *inspección*: cada función se diseña para que el efecto neto sobre el grafo coincida con el pushout final. La *gluing condition* aparece como un `return Err(...)` antes de mutar; los chequeos post-mutación son los siete invariantes (sec. 3.6).

5.3 Producciones DPO de las operaciones POSIX

Acá vienen las siete producciones que cubren la mayoría del API POSIX. Cada una se presenta con una caja (L, K, R) , su *gluing condition* formal, la firma exacta de la función Rust en el crate *sotfs-ops* y el lemma de preservación. Las pruebas formales están en Coq y se discuten en cap. 11.

5.3.1 `create_file`

Intuición. Crear un archivo regular bajo d/n . Aparece un nuevo Inode con `link_count=1` y una arista `contains` con nombre n desde d hasta el inode nuevo.

Producción ρ_{create} .

- L : el directorio d (un único nodo Directory).
- $K = L$: el directorio sobrevive entero (no se borra nada).
- R : d **más** un nuevo Inode i con `link_count=1` y una arista `contains` de d a i con nombre n .

Gluing condition (NAC, *negative application condition*).

$$\neg \exists e \in E : \tau_E(e) = \text{contains} \wedge \text{src}(e) = d \wedge e.\text{name} = n.$$

“No existe ya una entrada con ese nombre en d .” En POSIX corresponde a `EEXIST`.

Firma Rust (`sotfs-ops/src/lib.rs:33`).

```
pub fn create_file(
    g: &mut TypeGraph,
    parent_dir: DirId,
    name: &str,
    uid: u32, gid: u32,
    permissions: Permissions,
) -> Result<InodeId, GraphError>
```

Lema 5.1 (*Preservación bajo create_file*)

Si G es *well-formed* y ρ_{create} se aplica con éxito sobre G produciendo H , entonces H es *well-formed*.

Esquema. Los siete invariantes:

- Link-count: el inode nuevo tiene `link_count = 1` y exactamente una arista `contains` entrante con `name` $\neq$ `".."`. Coincide.
- Unique-names: la NAC garantiza que n no existía en d ; seguimos sin duplicados.
- Dir-self-ref: no se tocan dirs distintos de d , y d ya tenía su `..`.
- No-dangling: la arista nueva conecta d (vivo) e i (recién insertado, vivo).
- Block-refcount: no se tocan bloques.
- No-dir-cycles: no se inserta un Directory, así que la sub-relación `contains-restringida-a-dirs` no cambia.
- Cap-monotonicity: no se tocan caps.

La prueba formal mecanizada vive en `formal/coq/DpoCreate.v`, `theorem create_preserves_WellFormed` en línea 335 (cap. 11). $\square$

5.3.2 mkdir**Producción.**

- L : el directorio padre d_p (su Directory y su Inode).
- $K = L$.
- R : d_p **más** un nuevo Inode i (`vtype = Directory`, `link_count=2`), un nuevo Directory d con `inode_id = i`, y tres aristas `contains`: $d_p \xrightarrow{n} i$ (la entrada nueva en el padre), $d \rightarrow i$ (la self-ref del nuevo dir), $d \rightarrow d_p.\text{inode_id}$ (el back-pointer al padre).

Gluing condition. Análoga a `create_file`: $\neg \exists e \in E_d^{\text{contains}}$ con $e.\text{name} = n$.

Por qué `link_count=2`. Un directorio nuevo recibe dos aristas `contains` entrantes: la del padre con nombre n , y la `.` desde sí mismo. La invariante 3.1 excluye `..` pero *no* `.`. El `link_count` inicial es por construcción 2.

Firma Rust (sotfs-ops/src/lib.rs:94).

```
pub fn mkdir(
    g: &mut TypeGraph,
    parent_dir: DirId,
    name: &str,
    uid: u32, gid: u32,
    permissions: Permissions,
) -> Result<CreateResult, GraphError>
```

Resultado: el `InodeId` y el `DirId` del directorio nuevo.

5.3.3 link

(hard link)

Producción.

- L : el directorio d y el inode existente i .
- $K = L$.
- R : L **más** una arista `contains` de d a i con nombre n . *Adicionalmente*: el atributo $i.\text{link_count}$ se incrementa en uno.

La actualización de `link_count` es un *cambio de atributo*, no un cambio topológico; el formalismo DPO *atribuido* (sobre **TypeGraph**, no **Graph**) lo cubre con un morfismo sobre los atributos.

Gluing condition.

$$\neg \exists e \in E_d^{\text{contains}} : e.\text{name} = n \wedge i.\text{link_count} < \text{LINK_MAX}.$$

La segunda cláusula evita overflow del contador y, más sutilmente, acota el *treewidth* del grafo (trampa 5.3 más abajo y cap. 12). `LINK_MAX = 65535` en `sotfs-graph/src/graph.rs:28`.

Firma Rust (sotfs-ops/src/lib.rs:272).

5.3.4 unlink

Producción.

- L : el directorio d , el inode i y la arista `contains` $d \xrightarrow{n} i$.
- K : depende de $i.\text{link_count}$.
 - Si $i.\text{link_count} = 1$, $K = \{d\}$ (el inode se irá).
 - Si $i.\text{link_count} > 1$, $K = \{d, i\}$ (el inode sobrevive con `link_count` decrementado).
- $R = K$ **más** actualización del atributo $i.\text{link_count} \leftarrow i.\text{link_count} - 1$ si $i \in K$.

Gluing condition. Que la arista exista ($\exists e : \text{src}(e) = d \wedge e.\text{name} = n$) y, si $i.\text{vtype} = \text{Directory}$, que se invoque `rmdir` en vez de `unlink` (POSIX prohíbe `unlink` sobre directorios; en `sotFS` la verificación está en la firma).

Firma Rust (`sotfs-ops/src/lib.rs:322`).

Trampa 5.1 (*Orphan blocks bajo unlink de hard-linked file*)

Cuando $i.\text{link_count} = 1 \rightarrow 0$, el inode desaparece y sus aristas `pointsTo` también. Pero los `Block` que apuntaba podrían quedar con `refcount` desactualizado si la regla no decreuenta $b.\text{refcount}$ por cada arista borrada. Versión vulnerable: la regla decrementaba `refcount` sólo a la primera vista del bloque, saltándose colisiones (mismo bloque apuntado dos veces desde el mismo inode con distintos offsets — caso de archivo con bloques repetidos internamente). Resultado: invariante 3.5 violada.

Fix. Decrementar una vez por arista `pointsTo`, no una vez por bloque distinto. *Lección:* contar aristas, no objetos.

5.3.5 `rmdir`

Producción.

- L : el directorio padre d_p , el directorio d , el inode i pareado a d , y tres aristas: $d_p \xrightarrow{n} i$, $d \rightarrow i$, $d \rightarrow d_p.\text{inode_id}$.
- $K = \{d_p\}$: sólo sobrevive el padre.
- $R = K$.

Gluing condition. “ d está vacío”: no hay aristas `contains` salientes de d excepto las dos auto-aristas (`.` y `..`). En POSIX corresponde a `ENOTEMPTY`.

Firma Rust (`sotfs-ops/src/lib.rs:198`).

Observación (*Por qué `rmdir` es la prueba más difícil en Coq*)

La preservación del invariante 3.6 bajo `rmdir` requiere un argumento de invariante auxiliar (que *no hay hard links a directorios*). Esa propiedad no está formalizada en el modelo `Records` de `formal/coq/SotfsGraph.v`, lo que deja cuatro `Admitted` en `formal/coq/DpoRmdir.v`. El cap. 11 los lista uno por uno; el ej. 11.7 pide formalizarlos.

5.3.6 `rename`

Es la operación más rica del API POSIX: cubre tres casos según los directorios involucrados sean iguales (*same-dir rename*) o distintos (*cross-dir rename*), y según haya o no un destino ya existente que reemplazar (la semántica POSIX exige que un destino existente sea reemplazado atómicamente).

Same-dir, sin destino preexistente.

- L : el directorio d , el inode i , la arista $d \xrightarrow{n} i$.
- $K = \{d, i\}$.
- $R = K$ más la arista $d \xrightarrow{n'} i$.

Implementación: `rename_same_dir` (`sotfs-ops/src/lib.rs:441`); en este caso particular se reusa la misma `EdgeId` y se reescribe sólo el atributo `name`, evitando alocar y liberar.

Cross-dir. La regla es esencialmente `create_file(d',n')` ; `unlink(d,n)` atómica, con dos chequeos extra:

- Si i es un directorio, *no debe ser ancestro de d'* (evita ciclos en la jerarquía). Es una condición no local: requiere seguir las aristas . . desde d' hacia arriba y verificar que no aparece i .
- Si i es un directorio, su . . en d se reescribe para apuntar al inode pareado de d' .

Reemplazo de destino existente. POSIX exige que si d'/n' ya existe, *rename lo reemplace atómicamente*. La regla DPO correspondiente expone una arista supersedes desde el inode nuevo hacia el viejo durante el paso intermedio, lo que permite que la verificación TLA^+ `formal/sotfs_transactions.tla` detecte la atomicidad como single-step.

Firma Rust (`sotfs-ops/src/lib.rs:404`).

5.3.7 write

Intuición. Escribir bytes a un archivo. Concretamente, alocar (o reusar) bloques que cubran el rango `[offset, offset+len)` y mantener las aristas `pointsTo` con su `offset`.

Producción (versión simple, un único bloque).

- L : el inode i .
- $K = L$.
- R : L más un nuevo Block b con `refcount=1` y una arista $i \xrightarrow{\text{pointsTo, off}=k} b$.

Si una arista `pointsTo` ya cubre el rango, se reusa o se splittea según el tamaño. La versión completa se descompone en `write_block` (`sotfs-ops/src/lib.rs:626`) y `write_data` (`sotfs-ops/src/lib.rs:670`).

5.4 Gluing condition vs. invariantes

Es útil distinguir dos clases de chequeo:

1. **Gluing condition (pre-mutación).** Se verifica *antes* de aplicar la regla. Su falla resulta en un `Err(...)` que aborta sin tocar el grafo. Codifica el contrato *POSIX-like* (`EEXIST`, `ENOENT`, `ENOTEMPTY`).
2. **Invariante (post-mutación, eventual).** Se verifica *después* de la mutación, en modo paranoid o test, mediante `check_invariants()`. Su falla indica un *bug en la regla*, no un input inválido.

La idea de DPO es *minimizar la segunda lista*: si las gluing conditions están bien diseñadas, las reglas preservan los invariantes *por construcción*. Los Admitteds en Coq de cap. 11 existen porque algunas gluing conditions no son aún tan estrictas como podrían (concretamente: falta el invariante auxiliar NoHardLinkToDir).

5.5 Algoritmo genérico de aplicación de una regla DPO

Aunque el runtime no implementa un motor genérico, la formulación abstracta es útil para entender los Coq files de cap. 11.

Algoritmo 3: APPLYDPO

Entrada: Grafo host G , regla $\rho : L \leftarrow K \rightarrow R$, matching candidato $m : L \rightarrow G$
Salida: Grafo H resultado, o FAIL

```

3.1 si  $\neg \text{IsVALIDMATCHING}(m, L, G)$  entonces
3.2 |   retornar FAIL;
3.3 fin
3.4 si  $\neg \text{CHECKGLUING}(L, K, m, G)$  entonces
3.5 |   retornar FAIL;
3.6 fin
3.7  $D \leftarrow G \setminus m(L \setminus K)$ ;
3.8  $H \leftarrow D \cup R$ ;
3.9 si  $\neg \text{CHECKINVARIANTS}(H)$  entonces
3.10 |   retornar FAIL;
3.11 fin
3.12 retornar  $H$ ;
```

La línea 4 (“borra lo que vive sólo en L ”) es donde aparecen los *orphan nodes*: si un nodo $v \in m(L) \setminus m(K)$ tiene aristas adyacentes en G que no están en $m(L)$, el pushout complement no existe y la operación falla. Esto formaliza la noción clásica de *dangling edges*.

Algoritmo 4: CHECKGLUING

Entrada: L, K, m, G
Salida: ok si la gluing condition se cumple, fail si no

```

4.1  $\text{ToDelete} \leftarrow m(V_L) \setminus m(V_K)$ ;
4.2 para cada  $v \in \text{ToDelete}$  hacer
4.3 |   para cada  $e \in E_G$  adyacente a  $v$  hacer
4.4 | |   si  $e \notin m(E_L)$  entonces
4.5 | | |   retornar fail;
4.6 | |   fin
4.7 |   fin
4.8 fin
4.9 retornar ok;
```

5.6 Trampas históricas

Trampa 5.2 (*Cross-dir rename que crea un ciclo*)

commit (v0.1.x). Versión vulnerable: `rename_cross_dir` no chequeaba que el inode renombrado no fuera ancestro del directorio destino. `mv /a /a/b/c` convertía `/a` en su propio descendiente, violando invariante 3.6. POSIX manda `EINVAL` en ese caso.

Fix. Pre-condición que recorre los `..` de `d'` hacia arriba buscando el inode renombrado; si lo encuentra, retorna `InvalidArgument`. Costo $O(\text{depth})$, aceptable. **Lección.** Las `gluing conditions` no son sólo locales (chequear una arista incidente al `matching`); pueden requerir `traversals globales`.

Trampa 5.3 (*Treewidth no acotado por hard links*)

commit (v0.2.0). Sin un cap superior a `link_count`, `link` permite que un mismo inode acumule miles de aristas `contains entrantes`. Eso degrada las cotas estructurales del cap. 12 (`treewidth lineal` en el número de `hard links`) y rompe el patrón “filesystem $\Rightarrow$ tree-like”.

Fix. Constante `LINK_MAX = 65535` chequeada en la `gluing condition` de `link`. **Lección.** Las propiedades estructurales (`treewidth acotado`) son tan importantes como las semánticas; las `gluing conditions` deben respaldarlas.

5.7 Resumen del capítulo

- Una regla DPO es un span $L \leftarrow K \rightarrow R$ con `matching` $m : L \rightarrow G$ y `gluing condition`; el paso construye H vía dos `pushouts`.
- Cada operación POSIX (`create_file`, `mkdir`, `link`, `unlink`, `rmdir`, `rename`, `write`) tiene su producción explícita y su `gluing condition` formal.
- La `gluing condition` se verifica antes de mutar; el chequeo de invariantes `check_invariants` (cap. 3) se verifica después como red de seguridad y como contrato para los tests.
- En el runtime, las reglas son funciones Rust en el crate `sotfs-ops`; en Coq, son teoremas `X_preserves_WellFormed` en `formal/coq/Dpo*.v`; en TLA^+ , son acciones del módulo `formal/sotfs_graph.tla`. La correspondencia entre las tres es por inspección; cap. 11 y 10 desarrollan cada una.
- Trampas vistas: `orphan blocks` bajo `unlink hard-linked`, ciclo `cross-dir rename`, `treewidth` sin cap por `hard links`. Cada una se cierra ajustando la `gluing condition` o agregando una constante estructural.

Ejercicios

Ejercicio 5.1 [*]

Defina, en una línea, qué es la `gluing condition` de una regla DPO.

Ejercicio 5.2 [*]

Para cada una de las siete operaciones POSIX cubiertas, indique cuál es el conjunto K (el glue) en términos del subgrafo que sobrevive.

Ejercicio 5.3 [**]

Diseñe la regla DPO para `symlink(target, dir, name)` (crear un enlace simbólico). ¿Qué tipo de nodo introduce? ¿Qué arista? ¿Cuál es su gluing condition? Compare con `create_file`.

Ejercicio 5.4 [**]

Lea la regla `LinkInode` de `formal/sotfs_graph.tla`. ¿Qué pasa si el inode i no existe ($i \notin \text{inodes}$)? ¿Cómo se compara con el comportamiento POSIX de `link` sobre un inode borrado?

Ejercicio 5.5 [**]

Demuestre que `unlink` sobre un inode con `link_count = 1` preserva el invariante 3.4. Pista: razonar qué aristas se borran al desaparecer el inode.

Ejercicio 5.6 [**]

Formule la NAC de `mkdir` como una fórmula de primer orden sobre E . ¿Cuántos chequeos necesita un implementor para verificarla con el índice `dir_name_idx`?

Ejercicio 5.7 [* * *]

Diseñe la regla DPO para `rmdir` sobre un directorio que tenga *sólo* subdirectorios vacíos (`rmdir` recursivo). ¿Es una sola regla DPO? ¿Una secuencia? Justifique en términos de gluing conditions y atomicidad.

Ejercicio 5.8 [* * *]

trampa 5.2 se cierra con un traversal $O(\text{depth})$ sobre los $\dots$ Diseñe una versión que use el índice `dir_for_inode_idx` para reducir el costo. ¿Sigue siendo posible un ataque DoS construyendo cadenas profundas?

Ejercicio 5.9 [* * *]

Cargue el `formal/coq/DpoCreate.v`, lea el `theorem create_preserves_WellFormed` (línea 335) y reproduzca el esquema de prueba del lema 5.1 en pseudo-Coq (intros, casos, cita de los lemmas auxiliares de `formal/coq/SotfsGraph.v`).

Implementación: arenas, RCU y motor de reglas

Los caps. 3–5 fijaron el modelo formal: qué tipos viven en el grafo, qué reglas DPO actúan sobre él. Este capítulo baja a Rust: cómo el runtime aloja los nodos, cómo los lectores ven snapshots consistentes mientras un escritor muta, y cómo cada función POSIX en el crate `sotfs-ops` desemboca en una mutación segura.

6.1 Arenas: alocação dense con reuso

Definición 6.1 (*Arena tipada*)

Una *arena* `Arena<T, CAPACITY>` es un pool densamente empaquetado de hasta `CAPACITY` valores de tipo `T`, con un *free-list* de slots reusables después de la liberación. Operaciones: `alloc(v) -> Option<ArenaId> (O(1))`, `free(id) (O(1))`, `get(id) -> Option<&T> (O(1))`.

El runtime usa una arena por tipo de nodo (`inodes`, `dirs`, `caps`, `txns`, `versions`, `blocks`) y una arena para aristas. `CAPACITY` es 65 536 para nodos y 131 072 para aristas (`sotfs-graph/src/graph.rs:31`). Los datos viven en `Box<[MaybeUninit<T>]>` (heap-backed para evitar stack overflow: una arena entera son 35 MiB).

Estructura interna.

- `data`: el array de slots `MaybeUninit<T>`.
- `state`: per-slot `Vacant` o `Occupied`.
- `free_list`: stack de índices liberados, popea LIFO.
- `next_bump`: índice del próximo slot bump-allocable.

Algoritmo de asignación.

Algoritmo 5: ARENA.ALLOC**Entrada:** Arena A , valor v de tipo T **Salida:** ArenaId del slot ocupado, o None si llena

```

5.1 si  $A.free\_count > 0$  entonces
5.2    $A.free\_count \leftarrow A.free\_count - 1$ ;
5.3    $slot \leftarrow A.free\_list[A.free\_count]$ ;
5.4 sino
5.5   si  $A.next\_bump < CAPACITY$  entonces
5.6      $slot \leftarrow A.next\_bump$ ;
5.7      $A.next\_bump \leftarrow A.next\_bump + 1$ ;
5.8   sino
5.9     retornar None;
5.10  fin
5.11 fin
5.12  $A.data[slot] \leftarrow v$ ;
5.13  $A.state[slot] \leftarrow Occupied$ ;
5.14  $A.len \leftarrow A.len + 1$ ;
5.15 retornar Some(ArenaId(slot));

```

Lema 6.1 (*Densidad de la arena*)

Tras una secuencia arbitraria de ALLOC/FREE, la fracción de slots ocupados respecto del rango utilizado $[0, next_bump)$ está acotada inferiormente por $len/next_bump$. La arena no fragmenta porque FREE siempre devuelve el slot al free-list y ALLOC prefiere ese pool antes de bump-alocar.

Por qué importa. ArenaId es u32 (4 bytes) y se mapea a un u64 typed-id (InodeId, DirId, ...) sumando 1 (los IDs en sotFS empiezan en 1; el slot o nunca se usa como ID, sólo como sentinel). El ratio compacto evita cargar tablas grandes en cache cuando se itera el grafo.

6.2 Cómo se usa la arena en TypeGraph

La struct TypeGraph (sotfs-graph/src/graph.rs:52) tiene siete arenas:

```

pub struct TypeGraph {
    pub inodes: Arena<Inode, NODE_CAPACITY>,           // 65k
    pub dirs: Arena<Directory, NODE_CAPACITY>,
    pub caps: Arena<Capability, NODE_CAPACITY>,
    pub transactions: Arena<Transaction, NODE_CAPACITY>,
    pub versions: Arena<Version, NODE_CAPACITY>,
    pub blocks: Arena<Block, NODE_CAPACITY>,
    pub edges: Arena<Edge, EDGE_CAPACITY>,             // 131k
    // ... + índices secundarios BTreeMap
}

```

Una TypeGraph ocupa ~ 35 MiB en heap por instancia (siete arenas ×4 KiB de header + datos a demanda). Cap. 9 discute por qué eso vuelve costoso graph.clone().

6.3 RCU: lectores lock-free, escritor único

Definición 6.2 (*RcuGraph*)

RcuGraph mantiene *dos* copias del *TypeGraph* en heap (*Box<TypeGraph>*). Un índice atómico *active* indica cuál ven los lectores; el escritor muta la inactiva, hace un swap atómico, y espera a que los lectores que aún ven la copia vieja terminen (*rcu-synchronize*).

El protocolo:

Algoritmo 6: *RCUGRAPH.WRITE*

Entrada: *rcu* : *RcuGraph*, mutador *f* : *TypeGraph* → ()

```

6.1 ACQUIRE(rcu.write_lock) inactive ← 1 − rcu.active.LOAD();
6.2 rcu.graphs[inactive] ← rcu.graphs[rcu.active].CLONE();
6.3 f(rcu.graphs[inactive]);
6.4 rcu.active.STORE(inactive, RELEASE);
6.5 RCU SYNCHRONIZE(rcu) RELEASE(rcu.write_lock);

```

Lectores. Cada lector reclama un slot de *reader_epochs* (hasta 64 en paralelo, fijo en *sotfs-graph/src/rcu.rs:39*), bumpea el global *epoch* y escribe ese *epoch* en su slot. La sincronización del escritor consiste en esperar a que todos los slots con *epoch* ≤ *swap-epoch* retornen a *INACTIVE*. *No bloquea a nuevos lectores*: ellos ya verán la copia nueva.

Invariante 6.1 (*RCU consistency*)

Un lector que entró antes del swap ve el grafo en su estado pre-swap durante toda su lectura, aún si en paralelo otro escritor está mutando la copia opuesta.

Trampa subtle. *MAX_READERS* fijo en 64; un deployment con más lectores concurrentes que ese número entra a un *panic path* (*sotfs-graph/src/rcu.rs:31*). El número viene del bump vo.2.1 cuando una FUSE daemon a 16 cores trivialmente excedía los 8 originales. El fix definitivo (per-CPU counters o pool dinámico) está en post-vo.3.

6.4 El motor de reglas: cómo se aplica un paso DPO

Recapitulando del cap. 5: el runtime no implementa un motor DPO genérico. Cada operación POSIX es una función Rust que en orden hace (i) matching, (ii) gluing condition, (iii) mutación. La estructura canónica:

Algoritmo 7: ESQUELETO DE CADA OP DPO EN SOTFS-OPS

Entrada: $g : \&\text{mut TypeGraph}$, parámetros de la op
Salida: $\text{Result}\langle X, \text{GraphError} \rangle$

```

7.1  $handles \leftarrow g.\text{RESOLVE}(params);$ 
7.2 si  $handles = \text{None}$  entonces
7.3 |   retornar  $\text{Err}(\text{NotFound});$ 
7.4 fin
7.5 si  $\neg \text{CHECKPRECOND}(g, handles, params)$  entonces
7.6 |   retornar  $\text{Err}(\text{precond violada});$ 
7.7 fin
7.8  $newId \leftarrow g.\text{ALLOC}(\dots);$ 
7.9  $g.\text{INSERT}(\dots);$ 
7.10  $g.\text{UPDATEINDICES}(\dots);$ 
7.11  $g.\text{RECORDPROV}(\dots)$  retornar  $\text{Ok}(newId);$ 

```

Punto clave. La mutación nunca toca un slot ocupado: o aloca uno nuevo (vía `ARENA.ALLOC`) o libera uno viejo. Esto preserva el invariante 3.4 *por construcción de la arena*: un slot ya liberado no aparece en ningún índice secundario, y los nuevos índices apuntan a slots recién alocados.

6.5 Provenance log: el sidecar opcional

Si la GTXN está corriendo bajo FUSE con `SOTFS_PROV_SIDE CAR` configurado (cap. 14), cada op llama a `record_prov(op, inode_id, detail)` (`sotfs-graph/src/graph.rs:619`) que serializa una entrada JSONL con timestamp, tipo de op, inode, cap-id (si presente) y dominio. La política se inyecta vía `set_cap_ctx(CapContext)` antes de cada op (commit 61e7faf).

Resumen del capítulo

- Los nodos viven en arenas tipadas `Arena<T, CAPACITY>` con free-list LIFO; `ALLOC/FREE` son $O(1)$ y la arena no fragmenta.
- `TypeGraph` aglutina siete arenas (seis de nodo + una de arista) más índices secundarios `BTreeMap`; ocupa ~ 35 MiB en heap por instancia.
- `RcuGraph` mantiene dos copias y sincroniza a los lectores vía epoch counters; los escritores muta la inactiva y swappea con `Release`.
- Cada operación POSIX es una función Rust en el crate `sotfs-ops` que sigue el esqueleto `matching` $\rightarrow$ `gluing` $\rightarrow$ `mutación`.
- Provenance log opcional, drainable, con cap-mediated context.

Ejercicios

Ejercicio 6.1 [*]

¿Cuál es `NODE_CAPACITY` y `EDGE_CAPACITY` en `sotFS` v0.2.5?

Ejercicio 6.2 [*]

¿Cuántos lectores concurrentes admite el RCU? ¿Qué pasa si se excede?

Ejercicio 6.3 [**]

Calcule la ocupación de memoria de un `TypeGraph` vacío (arenas vacías). Pista: `Box<[MaybeUninit<T>]>` aloca `CAPACITY` entradas pero no las inicializa.

Ejercicio 6.4 [**]

La función `rcu_synchronize` es lineal en el número de lectores. Diseña un escenario donde un escritor podría *starvarse* si los lectores se renuevan más rápido que la sincronización. ¿Es posible bajo carga real?

Ejercicio 6.5 [**]

Mostrá que el algoritmo `ARENA.ALLOC` preserva la invariante “`state[i] == Occupied` si y solo si `data[i]` está inicializado”. Identifique los tres puntos de la prueba.

Ejercicio 6.6 [**]

Diseñe un test de stress para la arena: N alocações, $N/2$ liberaciones aleatorias, $N/2$ alocações adicionales. ¿Cuáles invariantes verificar al final?

Ejercicio 6.7 [**]

trampa 3.1 habla de cómo un índice secundario (`dir_for_inode_idx`) elimina un $O(N^3)$. En términos de la arena: ¿podría implementarse el índice como otra arena en lugar de `BTreeMap`? Discuta tradeoffs.

Ejercicio 6.8 [* * *]

Reemplace los `reader_epochs` fijos de 64 slots por un mecanismo de pool dinámico per-CPU que escale a cualquier número de lectores. Bosqueje el diseño en pseudo-Rust + invariantes que se preservan.

Persistencia: redb e índices secundarios

El cap. 6 mostró cómo el TypeGraph vive en heap. Este capítulo cubre cómo se vuelca a disco y se recupera, y por qué el truco crítico de la persistencia no es la serialización sino la *rehidratación de los índices secundarios*.

7.1 El backend redb

el crate `sotfs-storage` es un wrapper minimalista (85 LOC) sobre REDB, una base de datos KV embebida con transacciones ACID. Hay una sola tabla:

```
const METADATA_TABLE: TableDefinition<&str, &[u8]>
    = TableDefinition::new("metadata");

pub struct RedbBackend { db: Database, }
```

La tabla guarda un único par: ("graph", <bytes>). Los bytes son el TypeGraph entero serializado con `serde_json`. La elección es deliberadamente simple: la atomicidad la da la transacción de REDB, no un protocolo casero.

7.2 Save y load

Save. (`sotfs-storage/src/backend.rs:29`)

Algoritmo 8: BACKEND.SAVE

Entrada: Backend B , grafo g

```
8.1 bytes ← serde_json::to_vec(g);
8.2 txn ← B.db.begin_write();
8.3 table ← txn.open_table(METADATA_TABLE);
8.4 table.insert("graph", bytes);
8.5 txn.commit()
```

Load. (`sotfs-storage/src/backend.rs:41`)

Algoritmo 9: BACKEND.LOAD

Entrada: Backend B

Salida: Grafo g rehidratado, o None

```
9.1 txn ← B.db.begin_read();
9.2 table ← txn.open_table(METADATA_TABLE);
9.3 si table.get("graph") = None entonces
9.4 |   retornar None;
9.5 fin
9.6 g ← serde_json::from_slice(bytes);
9.7 g.REBUILD_DIR_NAME_IDX() retornar Some(g);
```

7.3 El truco: rehidratación de índices secundarios

El `TypeGraph` contiene varios índices que no se serializan:

- `dir_name_idx`: `BTreeMap<(DirId, String), EdgeId>` — Marcado `#[serde(skip)]`.
- `dir_for_inode_idx`: `BTreeMap<InodeId, DirId>` — `#[serde(default)]`, vacío tras deserialización.

¿Por qué saltarlos? Las claves tupla `(DirId, String)` no caben naturalmente en JSON. Más importante: estos índices son *datos derivados*, no estado canónico. Pueden reconstruirse a partir de `dir_contains` (que sí se serializa). Persistirlos duplicaría datos y agregaría riesgo de inconsistencia.

La función crítica. `rebuild_dir_name_idx` (`sotfs-graph/src/graph.rs:673`) recorre cada arista `contains` viva y la inserta en el `BTreeMap`:

Algoritmo 10: `GRAPH.REBUILDDIRNAMEIDX`

Entrada: Grafo g recién deserializado

```

10.1  $g.dir\_name\_idx \leftarrow \emptyset$ ;
10.2 para cada  $(d, eids) \in g.dir\_contains$  hacer
10.3   para cada  $eid \in eids$  hacer
10.4     si  $g.GETEDGE(eid) = Contains\{name, \dots\}$  entonces
10.5        $g.dir\_name\_idx.INSET((d, name), eid)$ ;
10.6     fin
10.7   fin
10.8 fin
```

Costo. $O(M \log M)$ donde M es el número de aristas `contains`. Para un FS de 10^5 entradas, sub-segundo.

¿Por qué crítico? Sin la rehidratación, `lookup_name` caería al fallback lineal $O(N)$ (cap. 3 trampa 3.2); el filesystem funcionaría pero degradaría a 92% de CPU bajo workload realista.

7.4 El comando `sotfsctl`

el `crate sotfs-cli` expone:

`sotfsctl mkfs <db.redb>` crea un FS vacío con root inode + root dir.

`sotfsctl check <db.redb>` carga el FS, ejecuta `check_invariants()` y `check_dir_name_idx_consistent`.
Versión *offline* del `fsck`.

`sotfsctl dump <db.redb>` vuelca a `stdout` el grafo en DOT o JSON.

7.5 Crash safety: lo que da `redb`

Teorema 7.1 (*Garantías de `redb`*)

La transacción de REDB provee atomicidad, durabilidad e isolation serializable. Concretamente: si `txn.commit()` retorna `Ok`, los datos están en NAND tras el power-cut; si retorna `Err` o crashea durante el commit, el estado pre-transacción sobrevive intacto.

La consecuencia para sotFS: el modelo formal del cap. 9 con su WAL explícito no se implementa en el runtime. La capa de durabilidad la provee REDB. Si REDB cumple sus contratos, sotFS hereda crash safety.

Trampa 7.1 (*fsync sin barrier en SSDs consumer-grade*)

REDB llama a `fsync(2)` antes de retornar de `commit`. Pero un SSD consumer-grade que ignora el comando FUA puede ackear el `fsync` antes de flushear su caché DRAM volátil. Bajo power-cut, los bytes “persistidos” se pierden.

Mitigación. El operador del FS debe verificar el SSD bajo power-cut físico (típicamente con un test de *torn write*). Modelo asume `fsync` honesto. Discusión completa en cap. 9.

Resumen del capítulo

- Backend REDB: una sola tabla, un solo blob JSON, atomicidad via `txn.commit`.
- Save serializa el TypeGraph entero; load deserializa más *rehidrata* los índices secundarios via `rebuild_dir_name_idx`.
- La rehidratación es $O(M \log M)$ y crítica: sin ella, `lookup_name` cae a $O(N)$ y degrada todo el FS.
- `sotfctl mkfs/check/dump` expone las primitivas para inicializar, verificar y volcar el FS offline.
- Crash safety se hereda de las garantías de REDB; el modelo TLA con WAL explícito vive un nivel de abstracción más arriba.

Ejercicios

Ejercicio 7.1 [★]

¿Qué tabla(s) usa el crate `sotfs-storage`? ¿Qué claves y valores?

Ejercicio 7.2 [★]

¿Por qué `dir_name_idx` no se serializa con `serde_json`?

Ejercicio 7.3 [★★]

Estime el costo de save / load para un FS de 10^5 inodes. Considera serialización JSON de un TypeGraph de ese tamaño.

Ejercicio 7.4 [**]

Diseñe un test que verifique la consistencia del índice `dir_name_idx` tras un round-trip `save/load` + 1000 ops más.

Ejercicio 7.5 [* * *]

Reemplace el blob JSON por un schema de varias tablas redb: una para inodes, una para edges, etc. Discuta tradeoffs en velocidad de `save/load`, fragmentación on-disk, y atomicidad cross-table.

Capabilities y delegación

POSIX clásico tiene un modelo de control de acceso por usuario / grupo / otros: cada inode lleva tres tripletes `rwX`. Para escenarios multi-tenant o sandboxing fino, ese modelo es insuficiente: no soporta delegación (“dale a *X* sólo permiso de lectura sobre este inode, sin derecho a delegar más”), revocación masiva (“invalida todas las caps derivadas de *C*”), ni atribución (“¿qué cap autorizó esta lectura?”).

sotFS modela las capabilities como *nodos del TypeGraph*. Cada operación POSIX se ejecuta bajo un `CapContext` explícito que identifica la cap y el dominio del operador. Las invariantes de monotonidad las preserva el sistema de reescritura por construcción.

8.1 El nodo Capability

Recordando el cap. 3:

```
pub struct Capability {
    pub id: CapId,
    pub rights: Rights,    // bitmask u8
    pub epoch: u64,       // generación, invalida on revoke
}
```

El bitmask `Rights` ocupa 8 bits (`sotfs-graph/src/types.rs:150`):

READ	0x01
WRITE	0x02
EXECUTE	0x04
GRANT	0x08
REVOKE	0x10

Aristas asociadas.

- `grants`: `Capability` → `Inode`. Una cap tiene exactamente una arista `grants` a un inode (la cap autoriza ese inode).
- `delegates`: `Capability` → `Capability`. Forma el árbol CDT (*capability derivation tree*): cada cap (excepto las raíz) tiene un padre.

8.2 El invariante central: monotonidad

Invariante 8.1 (*Cap monotonicity, G4*)

Para todo par $c, c' \in \text{Capability}$ con arista `delegates` de c a c' ,

$$c'.rights \subseteq c.rights.$$

Una delegación nunca crea derechos; sólo restringe los del padre.

8.3 Acciones formales en el spec TLA

formal/sotfs_capabilities.tla (291 LOC) modela cuatro acciones:

DERIVE. Aloca una cap nueva con $\text{rights} \subseteq \text{parent.rights}$, crea arista delegates desde el padre. Nuevo epoch unique.

REVOKE. Remueve una cap y *recursivamente todo su subárbol* delegates. Es la acción que justifica el epoch: stale handles a la cap revocada se detectan al chequear que el epoch corresponde a una cap viva.

GRANT. Transfiere una cap entre dominios. No afecta rights ni el árbol; sólo cambia la atribución.

ACCESS. Registra una operación (cap, inode, right) en un log de auditoría.

El invariante TLA correspondiente:

```
MonotonicAttenuation ==
  ∀ c ∈ caps :
    delegatesParent[c] # NULL ⇒
      (delegatesParent[c] ∈ caps ⇒
        capRights[c] Ceq capRights[delegatesParent[c]])
```

TLC verifica este invariante exhaustivamente bajo configuración small (3 caps, 3 derives, 1 revoke).

8.4 El CapContext: cap-mediated context

Cada operación POSIX en el crate sotfs-ops se ejecuta bajo un contexto que dice “qué cap autorizó esto y qué dominio lo emitió”. La estructura (sotfs-graph/src/types.rs:189):

```
pub struct CapContext {
    pub cap_id: Option<CapId>,
    pub domain_id: u64,
}
```

cap_id = None es el path *anonymous* (no se presentó cap; e.g. tarea de admin interna); domain_id = 0 es el dominio *kernel* privilegiado.

Cómo lo setean los callers.

- FUSE callbacks: derivan CapContext del Request de FUSE (uid / gid del proceso que invocó la syscall). Ver sotfs-fuse/src/fs.rs:59.
- Tests: setean explícitamente con g.set_cap_ctx(...).
- sotfstcl: corre con CapContext::anonymous().

El plumbing de vo.2.5 (commit 61e7faf). Pre-vo.2.5 el runtime exponía la API de caps pero *descartaba el contexto en tiempo de operación*. Cada record_prov guardaba (cap=None, domain=0) aunque la regla DPO se hubiera ejecutado bajo una cap real. El commit cierra ese gap: set_cap_ctx se llama al inicio de cada callback FUSE, record_prov lee el contexto vivo y lo serializa al sidecar JSONL (cap. 14).

8.5 Algoritmo de chequeo de acceso

Algoritmo 11: CHECKACCESS

Entrada: Grafo g , cap c , inode objetivo i , derecho r requerido
Salida: **ok** si la cap autoriza la op, **fail** si no

```

11.1 si  $c \notin g.caps$  entonces
11.2   | retornar fail (cap no existe);
11.3 fin
11.4 si  $r \notin c.rights$  entonces
11.5   | retornar fail (rights insuficientes);
11.6 fin
11.7  $tgt \leftarrow g.cap\_grants[c]$ ;
11.8 si  $tgt \neq i$  entonces
11.9   | retornar fail (cap no autoriza este inode);
11.10 fin
11.11 retornar ok;

```

Costo. $O(\log |\text{Capability}|)$ — todo es lookup en BTreeMap. La complejidad real está en la cadena delegates: para verificar que la cap proviene de un origen confiable, hay que recorrer al padre hasta una raíz, lo cual es $O(\text{profundidad del CDT})$ en peor caso. En la práctica las cadenas son cortas (3-4 niveles).

8.6 Revocación y el rol del epoch

Cuando REVOKE se ejecuta sobre una cap c :

1. Calcular el subárbol $\text{Desc}(c)$ siguiendo delegates.
2. Para cada $c' \in \{c\} \cup \text{Desc}(c)$, remover el nodo de la arena de caps y todas sus aristas adyacentes.
3. Las *handles externas* a esas caps (e.g. un cliente que cacheó un CapId) detectan la revocación al hacer un check de epoch: el epoch corriente del cap activo es distinto del cacheado, o la cap no existe.

Lema 8.1 (*Revocación es transitiva*)

Si $c \in \text{Capability}$ es revocada, ninguna cap derivada de c (vía delegates) sobrevive en el grafo. Toda invocación posterior con cualquier $c' \in \text{Desc}(c)$ falla en CHECKACCESS en su primera línea.

Resumen del capítulo

- Las capabilities son nodos Capability con `rights` (bitmask u8), `epoch` y aristas `grants` (a Inode) y `delegates` (a otra Capability).
- Invariante monotónica: `rights` de hijos delegados es subset del padre. Verificada en `sotfs-graph/src/graph.rs:1056` y en `formal/sotfs_capabilities.tla`.
- Cuatro acciones: DERIVE, REVOKE, GRANT, ACCESS.

- CapContext se plumbea desde FUSE a través de `set_cap_ctx` antes de cada op (commit 61e7faf); el provenance log atribuye cada op a su cap y dominio.
- Revocación es transitiva: borrar una cap remueve todo su subárbol delegates.

Ejercicios

Ejercicio 8.1 [*]

Liste los cinco bits del Rights bitmask de sotFS.

Ejercicio 8.2 [*]

¿Cuál es la diferencia entre `cap_id = None` y `cap_id = Some(0)`? Pista: revisar `CapContext::anonymous`.

Ejercicio 8.3 [**]

Demuestre el lema 8.1 usando inducción sobre la profundidad del subárbol delegates.

Ejercicio 8.4 [**]

Considere un atacante que obtuvo una CapId cacheada antes de una revocación. Mostrá cómo el campo epoch hace inútil ese handle.

Ejercicio 8.5 [**]

Diseñe la prueba TLA correspondiente a la invariante 8.1 (sin mirar `formal/sotfs_capabilities.tla`). ¿Cuál es la propiedad de inducción y cuál el estado base?

Ejercicio 8.6 [* * *]

Extienda el modelo a *revocación temporal*: una cap puede suspenderse por un intervalo $[t_0, t_1]$ y reactivarse después. Discutí qué nuevos invariantes hay que agregar y qué TLA spec hay que cambiar.

Transacciones y crash refinement

Un filesystem que vive en disco tiene que sobrevivir *crashes*: el power-cut, el kernel panic, el hot-yank del SSD. La pregunta no es “¿se rompe?” sino “¿en qué estado queda?”. Hay tres respuestas posibles, en orden creciente de garantías:

Inconsistent. El FS queda corrupto; `fsck` intenta repararlo con heurísticas. Bueno hasta el primer archivo crítico que no se recuperó.

Eventually consistent. Tras el reboot, una secuencia de pasos automáticos (replay del journal, reconciliación) llega a un estado consistente *eventualmente*. `ext4` con `data=journal` entra acá.

Always consistent. El FS está *en todo momento* en un estado consistente. Tras el crash, la primera lectura ya retorna datos correctos. Es a lo que `sotFS` apunta.

Este capítulo presenta la herramienta para llegar a la tercera: las *transacciones de grafo* (GTxN), su modelo formal con *crash refinement* en TLA^+ , y su implementación en el runtime, que difiere honestamente del modelo en un punto importante.

9.1 Modelo formal vs. implementación

	Modelo formal (TLA^+)	Implementación (Rust)
Spec / módulo	<code>formal/sotfs_transactions.tla</code> + <code>formal/sotfs_crash.tla</code>	el crate <code>sotfs-tx</code> sobre el crate <code>sotfs-storage</code>
Mecanismo	WAL explícito + commit marker	snapshot in-memory + <code>redb txn</code>
Atomicidad	2-phase commit modelado paso a paso	<code>with_transaction closure</code>
Durabilidad	<code>fsync</code> sobre WAL	<code>redb::Database::commit</code> (interno: <code>fsync</code>)
Crash safety	teorema $Spec_Concrete \Rightarrow Spec_Abstract$	confiada al backend <code>redb</code>
Refinement	<code>formal/sotfs_crash_refinement.tla</code>	por inspección

Cuadro 9.1: Modelo formal vs. implementación. El modelo describe un WAL explícito; el runtime delega la durabilidad al backend `redb`. La correspondencia es por *inspección humana*, no por *extracción mecánica*; cap. 10 discute las consecuencias de esa elección.

Es importante que esto se enuncie en el primer párrafo del capítulo, no en una nota al pie: el WAL de los specs es una construcción *teórica* para razonar sobre *crashes*; el runtime no escribe un WAL propio sino que confía en las garantías del backend `redb`. La prueba formal $Spec_Concrete \Rightarrow Spec_Abstract$ habla del WAL teórico, no del código de producción. La virtud de la separación: si mañana cambia el backend (de `redb` a otro KV embebido), la prueba

formal sigue valiendo siempre que el backend nuevo dé las mismas garantías de atomicidad y durabilidad.

9.2 La GTXN del runtime

La unidad de trabajo en el crate `sotfs-tx` es la *Graph Transaction* (GTXN). Su API es minimalista (`sotfs-tx/src/lib.rs:22`):

```
pub struct Gtxn {
    pub state: GtxnState,
    snapshot: TypeGraph,
}

impl Gtxn {
    pub fn begin(graph: &TypeGraph) -> Self { ... }
    pub fn commit(&mut self) -> Result<(), GraphError> { ... }
    pub fn rollback(self, graph: &mut TypeGraph) { ... }
}

pub fn with_transaction<F, T>(
    graph: &mut TypeGraph, f: F
) -> Result<T, GraphError>
where F: FnOnce(&mut TypeGraph) -> Result<T, GraphError>;
```

Listing 9.1: GTXN como wrapper sobre el TypeGraph.

El patrón canónico es la closure `with_transaction`, que *snapshot*ea el grafo al inicio, ejecuta la closure, y o bien verifica los invariantes y *commitea*, o *rollback*ea sobre cualquier error o violación de invariante.

Algoritmo 12: WITHTRANSACTION

Entrada: Grafo G , closure $f : G \rightarrow \text{Result}$

Salida: Resultado de f con G *commiteado*, o G restaurado al *snapshot*

```
12.1 snap ← clone(G);
12.2 r ← f(G);
12.3 si r = Ok(·) entonces
12.4     si G.CHECKINVARIANTS() = Err entonces
12.5         G ← snap;
12.6         retornar Err(invariante violada);
12.7     fin
12.8     retornar r;
12.9 fin
12.10 G ← snap;
12.11 retornar r;
```

Notas críticas sobre la implementación actual:

- El *snapshot* es una copia completa del `TypeGraph` (`graph.clone()`). Para grafos pequeños es barato; para grafos de millones de nodos, el costo es prohibitivo. La estructura RCU del cap. 6 mitiga esto en lectura, pero la *snapshot* de transacción todavía clona.
- No hay todavía un mecanismo de *commit en dos fases* entre objetos: GTXN es siempre *single-host*. La extensión *Tier-2* (*multi-host 2PC*) está en el roadmap (el crate `sotfs-experimental`).

- La durabilidad llega cuando el caller invoca `RedbBackend::save(graph)`, no en el commit de la GTXN. La GTXN garantiza atomicidad in-memory, no durabilidad on-disk.

9.3 El protocolo abstracto del modelo TLA: GTXN con WAL

Aunque el runtime no escriba WAL propio, el modelo formal del `formal/sotfs_crash.tla` sí lo hace, y razonar sobre crash safety requiere conocerlo. El protocolo de cuatro fases:

Algoritmo 13: GTXNCOMMIT (modelo formal)

Entrada: Transacción tx con conjunto de cambios $tx.changes$

```

13.1  $rec \leftarrow \text{SERIALIZE}(tx.changes);$ 
13.2  $\text{DISK.WRITE}(wal\_log, rec);$ 
13.3  $\text{DISK.FSYNC}(wal\_log);$ 
13.4  $\text{DISK.WRITE}(wal\_log, \text{COMMIT\_MARKER});$ 
13.5  $\text{DISK.FSYNC}(wal\_log);$ 
13.6 para cada cambio  $c \in tx.changes$  hacer
13.7    $\text{DISK.WRITE}(graph\_store, c);$ 
13.8 fin
13.9  $\text{DISK.FSYNC}(graph\_store);$ 
13.10  $\text{WAL.TRUNCATE}(tx);$ 

```

El *commit point* es el segundo fsync (línea 5). Antes del commit point, el crash deja WAL parcial sin marker $\Rightarrow$ el recovery descarta las operaciones (vuelve al estado pre-tx). Después del commit point, el crash deja WAL con marker $\Rightarrow$ el recovery re-aplica las operaciones (continúa al estado post-tx). Nunca un estado intermedio.

9.4 Crash refinement

El spec abstracto `formal/sotfs_transactions.tla` modela GTXN *sin* crashes; el spec concreto `formal/sotfs_crash.tla` *con* crashes. La técnica de *crash refinement* [9] prueba que toda traza concreta es traza abstracta vía un *refinement mapping*.

```

/* Crash puede ocurrir en cualquier momento.
Crash ==
  ^ ~crashed
  ^ crashed' = TRUE
  ^ memState' = NULL                                     /* memoria volátil borrada
  ^ UNCHANGED <<diskState, walState, gtxnState>>

/* Recovery: replay del WAL desde el último commit marker.
Recover ==
  ^ crashed
  ^ memState' = ReplayWAL(walState, diskState)
  ^ crashed' = FALSE
  ^ UNCHANGED <<diskState, walState>>

```

Listing 9.2: Acciones Crash y Recover en `formal/sotfs_crash.tla` (extracto).

El refinement mapping

```

RefinementMapping ==
  LET
    ConcreteState == <<diskState, walState, memState>>
    AbstractState ==
      IF crashed THEN
        \* mapeamos al estado abstracto "como si la última
        \* committed tx fuera la actual"
        ReplayWAL(walState, diskState)
      ELSE
        memState
  IN
     $\forall op \in TxOps :$ 
      ConcreteOp(op)  $\Rightarrow$  AbstractOp(op)

THEOREM Spec_Concrete  $\Rightarrow$  Spec_Abstract

```

Listing 9.3: Mapping de refinement concreto $\rightarrow$ abstracto.

Bajo TLC con configuración `small`, el teorema se verifica exhaustivamente sobre estados acotados (cap. 10 lo desarrolla).

9.5 Las propiedades verificadas

No partial application. Tras `Recover`, *toda* operación de una GTXN está aplicada o *ninguna*. Nunca “se aplicaron 3 de 5”.

Durability. Si `gtxn_commit` retornó `Ok`, la transacción sobrevive cualquier crash posterior.

Recovery consistency. Tras `Recover`, el estado de memoria satisface *todas* las invariantes del cap. 3 (los siete + el de índice).

WAL integrity. Una vez escrito el commit marker, el contenido del WAL no se modifica hasta el GC; el GC sólo borra entradas *ya aplicadas*.

9.6 El protocolo de fsync

La barrera de durabilidad es `fsync`: pide al SSD que flushee sus cachés volátiles a flash NAND. Sin `fsync`, una escritura puede quedarse en el caché del controlador y perderse en un power-cut.

```

disk.write(buf)      # bytes a kernel page cache
disk.fsync()         # kernel page cache -> driver buffer
                    # driver buffer -> SSD controller
                    # SSD controller -> NAND
                    # ack al kernel

```

Listing 9.4: Pipeline conceptual de fsync.

Invariante 9.1 (Persistencia tras fsync)

Tras `fsync(loc)` retornar exitosamente, los bytes escritos antes de la llamada a `loc` están persistidos en NAND y sobrevivirán cualquier power-cut posterior.

La invariante depende del SSD respetar el comando FUA (*Force Unit Access*). SSDs consumer-grade malos lo ignoran; SSDs enterprise-grade lo respetan. Verificar esto en runtime con un test de power-cut físico es la única vía; el modelo asume fsync honesto.

9.7 Trampas históricas

Trampa 9.1 (write A; write B; fsync *no garantiza orden*)

Versión vulnerable: el código asume que dos write seguidos por un fsync se persisten en orden *A, B*. *No es así*. El SSD puede reordenar internamente; fsync sólo garantiza que *ambos* están persistidos al retornar, no el orden.

Si el código depende de que *A* esté en disco antes que *B* (e.g. *B* es el commit marker que valida *A*), un crash entre los dos fsync puede dejar *B* en disco sin *A*.

Fix. write A; fsync; write B; fsync. Dos fsync explícitos. Costo: latencia, especialmente en HDD. Justificable cuando el orden importa (commit marker), no cuando no (datos paralelos).

Lección. fsync es una barrera de *durabilidad puntual*, no de *ordering entre operaciones*. Cada barrier tiene su propio fsync.

Trampa 9.2 (GC del WAL antes del fsync del graph)

commit 4775904. Versión vulnerable: el orden de Fase 3 y Fase 4 estaba invertido. El WAL se truncaba (Fase 4) *antes* de que el graph store hiciera fsync (Fase 3 incompleta).

Si crash ocurre después del WAL truncate pero antes del graph fsync: el WAL no tiene la transacción (truncado), el graph no la tiene (no fsync-eado). *Pérdida silenciosa*.

Fix. El orden estricto Fase 1 → Fase 2 → Fase 3 → Fase 4. Cada fase termina con su fsync correspondiente. WAL truncate sólo tras graph fsync exitoso.

Lección. La durabilidad es un pipeline ordenado de barreras. Una operación de *cleanup* (truncate del WAL) DEBE preceder de todas las operaciones que dependen de los datos cleaned-up. Caer en la tentación de hacer cleanup *en paralelo* para “ahorrar latencia” rompe la cadena de durabilidad.

9.8 La invariante de *readers see committed-or-pre-tx*

Invariante 9.2 (*Read isolation*)

Un *read* concurrente con un *gtxn_commit* en progreso ve o el estado anterior a la transacción o el estado posterior; nunca un estado intermedio. La elección depende de cuándo se linealiza el read respecto del commit point.

Implementación: los readers usan el Tier o RCU (cap. 6); toman snapshot de la *época global* del FS al iniciar la lectura, leen consistentemente de esa snapshot. Los writers (gtxn) bumpean la época sólo después del commit point, de modo que los readers con snapshot vieja siguen viendo el estado pre-tx, los nuevos ven el post-tx.

9.9 Origen del diseño

WAL viene de bases de datos (ARIES [8]); su aplicación a filesystems es de ext3 y XFS. La idea de *crash refinement* (modelo concreto con crashes refinando un modelo abstracto sin) es de Yggdrasil [9]. La combinación con un grafo tipado es contribución de sotFS; la dificultad es mantener la *inmutabilidad estructural* del grafo bajo crashes (los nodos viejos sobreviven, los nuevos pueden estar parciales).

9.10 Resumen del capítulo

- GTXN protocoliza la mutación durable en el modelo formal: WAL → commit marker → graph apply → WAL GC, con fsync en cada paso.
- En el runtime v0.2.5 la GTXN es snapshot-based in-memory; la durabilidad se delega al backend redb.
- El *commit point* es el fsync del marker en el modelo; antes, crash descarta; después, crash re-aplica.
- Crash refinement (`formal/sotfs_crash_refinement.tla`) prueba que toda traza concreta con crashes es traza abstracta sin ellos.
- Read isolation: los readers usan Tier o RCU sobre la época FS; ven o pre-tx o post-tx, nunca intermedio.
- Trampas: write reordering sin fsync intermedio rompe la durabilidad ordenada; WAL truncate antes del graph fsync causa pérdida silenciosa.

Ejercicios

Ejercicio 9.1 [*]

Diga el orden *correcto* de los cuatro fsync en una GTXN del modelo formal. ¿Cuál es el commit point? ¿Qué pasa si un crash ocurre entre el fsync 1 y el fsync 2?

Ejercicio 9.2 [*]

Enuncie en una línea cada una de las cuatro propiedades verificadas en sec. “Las propiedades verificadas”. ¿Cuál es la *más difícil* de demostrar? Justifique brevemente.

Ejercicio 9.3 [**]

Lea la propiedad *Recovery consistency*. ¿Qué propiedades del cap. 3 están en juego? ¿Bastaría verificar sólo el invariante 3.1 para garantizar consistencia?

Ejercicio 9.4 [**]

La GTXN del runtime usa `graph.clone()` como snapshot. Estime el costo en bytes y en tiempo para un grafo de 10^6 inodes. ¿En qué operaciones se nota más?

Ejercicio 9.5 [**]

Un atacante con acceso físico al SSD puede inyectar power-cuts controlados. ¿Puede forzar pérdida de datos en una GTXN correctamente implementada (modelo formal)? Justifique en términos del invariante 9.1 y los assumptions sobre el SSD.

Ejercicio 9.6 [**]

La trampa 9.2 se cierra con un orden estricto. Explique por qué un *commit en dos fases* entre el WAL y el graph store es necesario aún cuando ambos viven en el mismo SSD.

Ejercicio 9.7 [**]

Lea `sotfs-tx/src/lib.rs:55 (with_transaction)`. ¿Qué pasa si la closure f commitea el grafo en estado consistente pero *no* llamó a `check_invariants`? Pista: revisar el orden de las líneas.

Ejercicio 9.8 [**]

Tabla 9.1: explique por qué la fila “Durabilidad” no es estrictamente una correspondencia (el runtime no escribe WAL propio). ¿Qué garantías del backend `redb` son necesarias para que el runtime herede las propiedades del modelo?

Ejercicio 9.9 [**]

Enuncie la invariante 9.2 sin usar la palabra “snapshot”. Pista: hablar de visibilidad y orden lineal de eventos.

Ejercicio 9.10 [* * *]

Diseñe un test de *crash injection*: una herramienta que mata el proceso del runtime FUSE en momentos aleatorios y verifica que el FS está en un estado consistente tras boot. Pista: `fork(2) + kill -9` hijo a tiempo random + script de assertion.

Ejercicio 9.11 [* * *]

Compare las garantías de crash safety de `sotFS` contra `ext4` (con `data=journal`), `btrfs` y `ZFS`. ¿Qué ofrece `sotFS` que los otros no? ¿Qué ofrecen los otros que `sotFS` no?

Ejercicio 9.12 [* * *]

Diseñe la extensión Tier-2 (multi-host 2PC) de GTXN. ¿Qué nodos del TypeGraph hacen falta agregar (un nodo coordinator)? ¿Qué aristas? ¿Qué spec TLA⁺ hay que extender?

Verificación con TLA⁺

Los caps. 3–9 fijaron el modelo algebraico de sotFS. Este capítulo explica cómo se modela ese sistema en TLA⁺, cómo se ejecuta TLC sobre los specs, cómo se interpretan los resultados y, sobre todo, qué *no* prueba este método.

10.1 La biblioteca de specs

sotFS ships con seis specs TLA⁺ en formal/:

Spec	LOC	Propiedad
formal/sotfs_graph.tla	518	Well-formedness del TypeGraph; los 7 invariantes (cap. 3).
formal/sotfs_transactions.tla	317	GTXN atomicidad; commit $\Rightarrow$ <i>visible</i> $\wedge$ invariantes; rollback $\Rightarrow$ restaurado.
formal/sotfs_capabilities.tla	291	Cap monotonicity, revoke transitivo, epoch invalida stale handles.
formal/sotfs_crash.tla	287	WAL-based crash recovery; cualquier prefijo del log replays a un estado válido.
formal/sotfs_crash_refinement.tla	395	Refinement: el modelo concreto con crashes implementa el abstracto bajo stuttering.
formal/sotfs_curvature.tla	301	Curvatura estructural acotada bajo mutaciones adversariales.

Cuadro 10.1: Los seis specs TLA⁺ de sotFS y la propiedad que cada uno verifica.

Cada spec tiene tres configs (*.cfg, *_medium.cfg, *_large.cfg) para acotar el espacio de estados. La configuración small es la que pasa CI; medium es la del nightly; large se corre manualmente y puede excederse en RAM dependiendo del host.

10.2 Anatomía de un spec: sotfs_graph

Cualquier spec TLA⁺ tiene la misma estructura: constantes, variables, init, next, fairness y propiedades.

Constantes y variables

CONSTANTS		
InodeIds ,	DirIds ,	BlockIds ,
Names ,	MaxOps	

```

VARIABLES
  inodes,          \* Set of active inode IDs
  dirs,            \* Set of active dir IDs
  blocks,          \* Set of active block IDs
  inodeType,       \* InodeId → {"regular", "directory"}
  linkCount,       \* InodeId → Nat
  containsEdges,   \* Set of <<dir, inode, name>> triples
  pointsToEdges,   \* Set of <<inode, block, offset>> triples
  dirInode,        \* DirId → InodeId (the inode `.` points to)
  blockRefCount,    \* BlockId → Nat
  ops              \* counter, bounds state space

```

Listing 10.1: Estado del spec `sotfs_graph`.

Las **CONSTANTS** se instancian en el `.cfg`:

```

CONSTANTS
  InodeIds = {I1, I2, I3}
  DirIds   = {D1, D2}
  BlockIds = {B1, B2}
  Names    = {a, b}
  MaxOps    = 4

```

Listing 10.2: `sotfs_graph.cfg`: la config `small`.

Esos cuatro inodes, dos dirs, dos bloques, dos nombres y traza máxima de cuatro pasos producen un espacio de estados de $\sim 10^5$ estados explorables en segundos. La config `medium` sube a $\{I1 \dots I5\}$, $\{D1, D2, D3\}$, etc., y produce $\sim 10^7$ estados.

Acciones (operaciones POSIX)

Cada acción del spec es la versión TLA de una operación POSIX. Por ejemplo, `LinkInode`:

```

LinkInode(d, i, name) ==
  ∧ d ∈ dirs
  ∧ i ∈ inodes
  ∧ ~∃ e ∈ containsEdges :
    e[1] = d ∧ e[3] = name \* gluing condition
  ∧ containsEdges' = containsEdges
    ∪ {<<d, i, name>>}
  ∧ linkCount' = [linkCount EXCEPT
    ![i] = @ + 1]
  ∧ ops' = ops + 1
  ∧ UNCHANGED <<inodes, dirs, blocks, ..., dirInode>>

```

La cláusula `~∃ e \in containsEdges : e[1] = d /\ e[3] = name` es la *gluing condition*: “no existe una arista `contains` desde `d` con `name`”. Es la versión TLA del `EEXIST` POSIX, pero *no* como un chequeo *ad hoc*: emerge del estilo DPO del spec.

El predicado `Next`


```

Next ==
  ∨ ∃ d ∈ dirs, i ∈ inodes, n ∈ Names : LinkInode(d, i, n)
  ∨ ∃ d ∈ dirs, n ∈ Names : CreateFile(d, n)
  ∨ ∃ d ∈ dirs, n ∈ Names : Mkdir(d, n)
  ∨ ∃ d ∈ dirs, n ∈ Names : Unlink(d, n)
  ∨ ∃ d ∈ dirs, n ∈ Names, n2 ∈ Names :
    Rename(d, n, n2)
  ∨ ops >= MaxOps ∧ UNCHANGED vars    \* stutter at bound

```

Cada disyunción es una acción posible en un paso. TLC explorará el producto cartesiano de todas las maneras de instanciar los parámetros existenciales.

Invariantes

```

INVARIANTS
  TypeInvariant
  LinkCountConsistent
  UniqueNamesPerDir
  DirHasSelfRef
  NoDanglingEdges
  BlockRefCountConsistent
  NoDirCycles
  ReachableHaveLinks

```

Estos son la versión TLA de los siete invariantes del cap. 3, más ReachableHaveLinks (todo inode alcanzable desde la raíz tiene linkCount ≥ 1). TLC evalúa cada uno en *cada estado alcanzable*; si alguno falla, se imprime el contraejemplo (la traza que llegó al estado violatorio).

10.3 Cómo correr TLC

```

# Setup (una vez).
mkdir -p formal/lib
curl -L -o formal/lib/tla2tools.jar \
  https://github.com/tlaplus/tlaplus/releases/latest/\
download/tla2tools.jar

# Todos los specs, todos los tamaños.
just formal

# O directamente:
cd formal && bash run_tlc.sh

# Un solo tamaño.
bash run_tlc.sh small
bash run_tlc.sh medium

# Un solo spec, todos los tamaños.
bash run_tlc.sh graph

```

Salida: por cada combinación spec $\times$ config, un log en formal/tlc_output/ y una tabla resumen en stdout.

10.4 Cómo leer un contraejemplo de TLC

Si una invariante falla, TLC imprime la *traza mínima* que llevó al estado violatorio:

```
Error: Invariant LinkCountConsistent is violated.
The behavior up to this point is:
State 0:
  inodes = {I1}
  linkCount = (I1 :> 1)
  ...
State 1: Action `Mkdir(D1, "a")`
  inodes = {I1, I2}
  linkCount = (I1 :> 1 @@ I2 :> 2)
  ...
State 2: Action `Unlink(D1, "a")`
  inodes = {I1, I2}          <-- bug: I2 todavía vivo
  linkCount = (I1 :> 1 @@ I2 :> 1) <-- pero linkCount=1
```

La traza tiene tres elementos clave: (i) el estado inicial; (ii) cada acción aplicada con sus parámetros instanciados; (iii) el estado final donde la invariante falla. El diagnóstico es directo: Unlink no decrementó linkCount consistentemente con la remoción de la arista contains.

10.5 Crash refinement: el spec más sofisticado

`formal/sotfs_crash_refinement.tla` no es un spec con sus propios estados nuevos: es un *spec de refinement* que mapea el estado del modelo concreto (con Crash y Recover) al estado del modelo abstracto (sin crashes), y verifica el teorema:

```
THEOREM Spec_Concrete  $\Rightarrow$  Spec_Abstract
```

El mapping (cap. 9) lleva un estado concreto `<<diskState, walState, memState, crashed>>` a un estado abstracto que es o bien `memState` (si no crasheado) o bien `ReplayWAL(walState, diskState)` (si crasheado). TLC verifica exhaustivamente que toda traza concreta es traza abstracta.

10.6 Status TLC v0.2.5

Reporte oficial (commit de `formal/README.md`): **14/14 PASS** en `small` y `medium`. La config `large` puede agotar RAM dependiendo del host; el gating de CI es `medium`.

10.7 Lo que TLA⁺ no prueba

- No prueba el código Rust. La correspondencia `spec` $\rightarrow$ `código` es por inspección humana, no por extracción mecánica.
- No es prueba inductiva. TLC verifica modelos finitos por enumeración exhaustiva. Modelos más grandes podrían tener bugs no detectados.
- No verifica timing. Las propiedades de fairness y de *eventually* usan el modelo lineal estándar; latencias reales no aparecen.

- No reemplaza a Coq. Las pruebas Coq del cap. 11 son inductivas y cubren modelos infinitos (universalmente cuantificados). TLC y Coq son complementarios.

Resumen del capítulo

- sotFS tiene seis specs TLA⁺ (graph, transactions, capabilities, crash, crash refinement, curvature), ejecutables con TLC bajo configs small / medium / large.
- Cada spec sigue la estructura CONSTANTS, VARIABLES, Init, Next, Spec, INVARIANTS; las acciones son la versión TLA de las reglas DPO.
- Status v0.2.5: 14/14 PASS en small y medium.
- El crash refinement es un meta-spec que prueba que el modelo concreto refina al abstracto: Spec_Concrete $\Rightarrow$ Spec_Abstract.
- Limitaciones: TLC explora modelos finitos; la correspondencia spec-código es por inspección.

Ejercicios

Ejercicio 10.1 [*]

Liste los seis specs TLA⁺ de sotFS con la propiedad que cada uno verifica.

Ejercicio 10.2 [*]

¿Cuáles son las constantes que se instancian en `sotfs_graph.cfg` en la config `small`?

Ejercicio 10.3 [**]

Lea la acción `LinkInode` del listado 10.1. Identifique (a) la *gluing condition*, (b) la *mutación*, (c) las variables `UNCHANGED`. ¿Qué invariante TLA podría romperse si se omitiera el incremento de `linkCount`?

Ejercicio 10.4 [**]

Diseñe una acción TLA `Rmdir(d)` para sotFS, con su *gluing condition* explícita. Compare con la regla DPO del cap. 5.

Ejercicio 10.5 [**]

Estime el espacio de estados de `formal/sotfs_graph.tla` con la config `medium` (5 inodes, 3 dirs, 3 bloques, 3 nombres, `MaxOps=6`). Pista: razonar sobre el número de estados alcanzables de `containsEdges`.

Ejercicio 10.6 [**]

Lea el último contraejemplo en `formal/tlc_output/` (si TLC nunca falló, simule uno: borre el incremento de `linkCount` en `LinkInode` y vuelva a correr). Reproduzca el bug y explíquelo en términos del cap. 3.

Ejercicio 10.7 [**]

`formal/sotfs_crash_refinement.tla` prueba un teorema: $\text{Spec_Concrete} \Rightarrow \text{Spec_Abstract}$. ¿Qué propiedad formal del cap. 9 se sigue de ese teorema?

Ejercicio 10.8 [* * *]

Extienda `formal/sotfs_capabilities.tla` con una acción `TemporalRevoke(c, t_end)` que suspende una `cap` hasta un timestamp futuro. ¿Qué invariante nuevo hay que agregar? ¿Cambia `MonotonicAttenuation`?

Verificación con Coq

Mientras que TLA⁺ explora modelos finitos por enumeración, Coq es un *asistente de pruebas* que verifica teoremas universalmente cuantificados de manera inductiva. La verificación con Coq en sotFS apunta a probar que cada regla DPO del cap. 5 preserva la well-formedness del TypeGraph, sin acotar el tamaño del grafo.

Este capítulo explica la estructura de los archivos Coq, los teoremas centrales, y — crítico para la honestidad del proyecto — los tres lemmas Admitted que quedan pendientes en formal/coq/DpoRmdir.v.

11.1 El modelo Records: SotfsGraph.v

formal/coq/SotfsGraph.v (585 LOC) define el universo formal sobre el que se prueban las reglas. Las estructuras centrales:

```
Record InodeRec := {
  ir_id : InodeId;
  ir_vtype : VnodeType;
  ir_link_count : nat
}.

Record DirRec := {
  dr_id : DirId;
  dr_inode : InodeId
}.

Record ContainsEdge := {
  ce_dir : DirId;
  ce_ino : InodeId;
  ce_name : Name
}.

Record Graph := {
  g_inodes : list InodeRec;
  g_dirs : list DirRec;
  g_edges : list ContainsEdge
}.
```

Listing 11.1: Records principales en SotfsGraph.v.

Por qué Records (no inductivos). Records permiten una representación operacional muy cercana al runtime Rust: el grafo es una tripleta de listas. Las pruebas resultan más concretas (less case-splitting sobre constructores) y la traducción mental con el código es directa.

Well-formedness

```

Definition WellFormed (g : Graph) : Prop :=
  TypeInvariant g
  /\ LinkCountConsistent g
  /\ UniqueNamesPerDir g
  /\ NoDanglingEdges g
  /\ NoDirCycles g.

```

Cinco cláusulas; las dos invariantes adicionales del runtime (`DirHasSelfRef`, `BlockRefCountConsistent`) no están formalizadas en el modelo Coq de v0.2.5 porque viven en el subsistema de bloques que no aparece en estos archivos. La extensión es trabajo abierto.

11.2 Los teoremas de preservación

Para cada operación POSIX (excepto `write`, todavía no formalizado en Coq), el archivo `formal/coq/DpoX.v` prueba seis teoremas: uno por cada cláusula del `WellFormed` y uno que los combina.

Archivo	LOC	Teorema final
<code>formal/coq/DpoCreate.v</code>	348	<code>create_preserves_WellFormed</code> (línea 335)
<code>formal/coq/DpoMkdir.v</code>	586	<code>mkdir_preserves_WellFormed</code> (línea 572)
<code>formal/coq/DpoLink.v</code>	382	<code>link_preserves_WellFormed</code> (línea 369)
<code>formal/coq/DpoUnlink.v</code>	314	<code>unlink_keep_preserves_WellFormed</code> (línea 300)
<code>formal/coq/DpoRmdir.v</code>	513	<code>rmdir_preserves_WellFormed</code> (línea 492)
<code>formal/coq/DpoRename.v</code>	317	<code>rename_preserves_WellFormed</code> (línea 304)

Cuadro 11.1: Los seis archivos de prueba DPO. Para cada uno, el teorema final es de la forma “ $\forall g, \text{precond} \Rightarrow \text{WellFormed}(g) \Rightarrow \text{WellFormed}(\text{op } g)$ ”.

Estructura de una prueba típica

Tomemos `create_preserves_WellFormed` en `formal/coq/DpoCreate.v` línea 335 como ejemplo. La forma:

```

Theorem create_preserves_WellFormed :
  forall g d n uid gid,
    WellFormed g ->
    name_unique_in g d n ->
    contains_dir g d ->
    WellFormed (create g d n uid gid).
Proof.
  intros g d n uid gid Hwf Huniq Hd.
  destruct Hwf as [Hti [Hlcc [Hun [Hnd Hndc]]]].
  unfold WellFormed; repeat split.
  - apply create_preserves_TypeInvariant; assumption.
  - apply create_preserves_LinkCountConsistent;
    assumption.
  - apply create_preserves_UniqueNamesPerDir;

```

```

    assumption.
- apply create_preserves_NoDanglingEdges; assumption.
- apply create_preserves_NoDirCycles; assumption.
Qed.

```

La descomposición es modular: el teorema final invoca cinco lemmas auxiliares, uno por cláusula del WellFormed, cada uno probado arriba en el mismo archivo. Esto hace el código de prueba navegable y re-usable entre operaciones (TypeInvariant es similar entre create_file y link, por ejemplo).

11.3 Deuda formal: los tres Admitted

Trampa 11.1 (*Tres lemmas Admitted en DpoRmdir.v*)

En la versión v0.2.5, el archivo formal/coq/DpoRmdir.v contiene tres ocurrencias de Admitted (líneas 349, 405, 505). Todas están bloqueadas por la misma deficiencia del modelo: la propiedad de que *los directorios no pueden ser hard-linked* (GC-LINK-2 en la nomenclatura del proyecto) no está enunciada como un lemma auxiliar de WellFormed, lo que impide cerrar la prueba en los sub-casos donde habría que saber que el inode removido no es target de ninguna otra arista contains sobreviviente.

Cómo se ve. Los tres Admitted aparecen al final de los sub-casos donde el contexto requiere algo como forall ce \in g.edges, ce.ino <> removed_inode. Sin un invariante explícito NoHardLinkToDir, esa hipótesis no está en scope; el comentario adyacente lo dice:

```

(* NOTE: The proof is complete except for one
sub-case: showing that no surviving edge targets
the removed inode ti. This requires the
additional fact that directories cannot be
hard-linked (GC-LINK-2), which is not yet part
of WellFormed. We state the theorem with
Admitted for transparency. *)

```

Status declarado. Los teoremas rmdir_preserves_WellFormed y rmdir_preserves_NoDanglingEdges no están probados en la versión actual. Las pruebas están casi completas; el gap es estructural.

Roadmap. Tres pasos:

1. Extender WellFormed con una sexta cláusula NoHardLinkToDir : Prop que diga “ningún directorio es target de más de una arista contains viva (excluyendo ..)”.
2. Re-probar create_preserves_WellFormed, mkdir_preserves_WellFormed, link_preserves_WellFormed y los demás incluyendo el nuevo invariante. Es trabajo mecánico: las reglas ya respetan la propiedad, sólo falta probarlo formalmente.
3. Cerrar los tres Admitted usando NoHardLinkToDir como hipótesis auxiliar.

Lección. La verificación mecánica obliga a hacer explícitos invariantes que el código respeta “por construcción” pero que el modelo formal no enuncia. Los Admitted son la deuda visible; cerrar la deuda significa hacer crecer el modelo, no el código.

11.4 Cómo correr las pruebas Coq

```
cd formal/coq
coq_makefile -f _CoqProject -o Makefile
make
```

La compilación verifica todos los teoremas; un `Admitted` *no* interrumpe la compilación (Coq sí lo loguea como `warning`). Para ver los `Admitted`s y otros `warnings`:

```
make 2>&1 | grep -E "Admitted|Warning"
```

Tiempo total de compilación en una CPU moderna: ~ 2 minutos.

11.5 Lo que Coq prueba (y lo que no)

Lo que prueba. Para cada regla DPO (salvo `rmdir_preserves_WellFormed`, `rmdir_preserves_NoDangling` y `rmdir_preserves_TypeInvariant`), que la aplicación de la regla preserva las cinco cláusulas del `WellFormed` para *todo* grafo y *todos* los parámetros admisibles.

Lo que no prueba.

- `rmdir` con sus tres `Admitted`.
- Los invariantes del runtime que no están en el modelo Coq (`DirHasSelfRef`, `BlockRefCountConsistent`, `NoHardLinkToDir`).
- La correspondencia entre las funciones `create`, `mkdir`, ... de Coq y las funciones Rust de el crate `sotfs-ops`. Esa correspondencia es por inspección humana; no hay extracción $\text{Coq} \rightarrow \text{Rust}$.
- Las propiedades concurrentes (RCU, GTXN). Las pruebas Coq asumen serial; concurrencia se cubre con TLA^+ y tests `loom`.

Resumen del capítulo

- Coq verifica que cada regla DPO preserva `WellFormed` para grafos arbitrarios; complementa la enumeración finita de TLC con prueba inductiva.
- `formal/coq/SotfsGraph.v` define los `Records` `InodeRec`, `DirRec`, `ContainsEdge`, `Graph` y la proposición `WellFormed` con cinco cláusulas.
- Seis archivos `Dpo*.v` prueban `X_preserves_WellFormed` para `create`, `mkdir`, `link`, `unlink`, `rmdir`, `rename`.
- **Deuda formal explícita:** tres `Admitted` en `DpoRmdir.v` bloqueados por el invariante auxiliar `NoHardLinkToDir` faltante.
- Limitaciones: el modelo Coq omite varios invariantes del runtime y no hay extracción $\text{Coq} \rightarrow \text{Rust}$.

Ejercicios

Ejercicio 11.1 [*]

¿Cuántos archivos `.v` tiene la verificación Coq de `sotFS`? Liste sus nombres.

Ejercicio 11.2 [*]

¿Cuáles son las cinco cláusulas de `WellFormed`? ¿Cuál falta respecto del runtime?

Ejercicio 11.3 [**]

Lea el `Theorem create_preserves_WellFormed` en `formal/coq/DpoCreate.v` línea 335. Identifique los cinco lemmas auxiliares que se invocan y la línea donde cada uno se prueba.

Ejercicio 11.4 [**]

¿Por qué un `Admitted` no interrumpe la compilación de Coq? Discuta los pros y contras de esa decisión de diseño.

Ejercicio 11.5 [**]

Enuncie en Coq el invariante `NoHardLinkToDir` faltante. Pista: una proposición `forall ce1 ce2 \in g.edges, ce1.tgt is a directory inode -> ce1.name = ".." \/\ ce2.name = "."`.

Ejercicio 11.6 [**]

La trampa 11.1 dice que cerrar los tres `Admitted` requiere “re-probar” los otros teoremas con la invariante nueva. ¿Cuáles teoremas exactamente hay que tocar? Pista: recorrer la tabla 11.1.

Ejercicio 11.7 [* * *]

Formalice `NoHardLinkToDir` en Coq como una sexta cláusula de `WellFormed` y pruebe que `create_file` la preserva. Para `mkdir`, escriba el statement (no es necesario completar la prueba).

Ejercicio 11.8 [* * *]

Investigue cómo se haría la extracción $\text{Coq} \rightarrow \text{Rust}$ de las funciones `create`, `mkdir`, ¿Qué plugins existen? ¿Cuál sería el costo de mantenibilidad de tener dos implementaciones (Coq y Rust) en sincronía vs. una sola (Rust) con prueba externa?

Topología del grafo: treewidth y curvatura

Hasta acá los invariantes del TypeGraph fueron *semánticos*: link counts, monotonía de capabilities, ausencia de ciclos de directorios. Este capítulo introduce dos métricas *estructurales* que el runtime computa sobre el grafo: *treewidth* (aproximada) y *curvatura de Ollivier-Ricci*. Ambas miden qué tan “árbol-like” es el grafo. La motivación es operativa: si un workload adversarial empuja el grafo lejos del régimen tree-like, se levantan alertas; si una proyección del grafo (cap. 13) tiene curvatura sistemáticamente distinta del original, el atacante podría detectarla por timing-side-channel.

12.1 Por qué importan estas métricas en un FS

Treewidth. Una cantidad asociada a cómo de bien un grafo se puede descomponer en un árbol de bags pequeños. Treewidth bajo $\Leftrightarrow$ grafo tree-like $\Leftrightarrow$ muchos algoritmos NP-difíciles se vuelven polinomiales sobre él. Para sotFS, un treewidth acotado superiormente por el `LINK_MAX` = 65535 (cap. 3) más unas pocas unidades garantiza escalabilidad de algoritmos de análisis y monitoreo.

Curvatura. La curvatura de Ollivier-Ricci sobre un grafo mide qué tan “positivamente curvado” (cliques, esferas) o “negativamente curvado” (árboles, cuellos de botella) es cada arista. Permite detectar patrones adversariales: una “symlink bomb” produce una firma de curvatura distinta a la de un FS normal; una operación de `mkdir` masivo produce una spike positiva en el directorio padre.

12.2 Treewidth: definición y aproximación

Definición 12.1 (*Tree decomposition*)

Una *tree decomposition* de un grafo $G = (V, E)$ es un árbol T junto con una asignación $X : V_T \rightarrow \mathcal{P}(V)$ tal que:

1. $\bigcup_{t \in V_T} X(t) = V$.
2. Para toda arista $(u, v) \in E$, existe t con $\{u, v\} \subseteq X(t)$.
3. Para todo $v \in V$, el conjunto $\{t : v \in X(t)\}$ induce un subárbol conexo de T .

El *ancho* de la descomposición es $\max_t |X(t)| - 1$. El *treewidth* de G es el mínimo ancho sobre todas las descomposiciones.

Calcular treewidth exacto es NP-hard [3]. sotFS usa una aproximación greedy con *min-degree elimination ordering*.

Algoritmo greedy**Algoritmo 14:** GREEDYMINDEGREETREewidth

Entrada: Grafo $G = (V, E)$
Salida: Upper bound de treewidth

```

14.1  $G' \leftarrow G$   $tw \leftarrow 0$ ;
14.2 mientras  $|V_{G'}| > 1$  hacer
14.3    $v \leftarrow \arg \min_{u \in V_{G'}} \deg_{G'}(u)$ ;
14.4    $tw \leftarrow \max(tw, \deg_{G'}(v))$ ;
14.5   para cada  $u_1, u_2 \in N_{G'}(v), u_1 \neq u_2$  hacer
14.6     Add edge  $(u_1, u_2)$  to  $G'$  if not present;
14.7   fin
14.8   Remove  $v$  from  $G'$ ;
14.9 fin
14.10 retornar  $tw$ ;

```

Costo. $O(n^2)$ con representación adecuada. Suficientemente barato para correr periódicamente sobre el grafo del FS.

Resultado para sotFS.**Teorema 12.1** (*Treewidth acotado para FS con hard-link cap*)

Sea G un *TypeGraph well-formed* con $i.link_count \leq LINK_MAX$ para todo *inode*. Entonces el *treewidth* de G satisface $tw(G) \leq LINK_MAX + O(1)$.

Sin hard links ($link_count \leq 1$ para todo *inode*), el *TypeGraph* es esencialmente un árbol, $tw(G) \leq 6$. La cota $LINK_MAX$ de invariante 3.1 (vía trampa 5.3) preserva este orden de magnitud incluso bajo workload adversarial.

12.3 Curvatura de Ollivier-Ricci**Definición 12.2** (*Curvatura discreta*)

La *curvatura de Ollivier-Ricci* de una arista (u, v) en un grafo G es

$$\kappa(u, v) = 1 - \frac{W_1(\mu_u, \mu_v)}{d(u, v)}$$

donde W_1 es la distancia Wasserstein-1, $d(u, v)$ es la distancia en el grafo, y μ_u, μ_v son medidas de probabilidad sobre los vecinos de u y v respectivamente.

Para grafos no ponderados, $d(u, v) = 1$ (los extremos son adyacentes), y se usa la *lazy random walk* con parámetro $\alpha \in [0, 1]$: $\mu_x(x) = \alpha$, $\mu_x(z) = (1 - \alpha) / \deg(x)$ para todo z vecino de x .

Aproximación Lin-Lu-Yau

Calcular W_1 exactamente requiere resolver un transporte óptimo (linear program). el crate `sotfs-monitor` usa la aproximación Lin-Lu-Yau [7] con $\alpha = 0.5$:

$$\kappa_{\text{LLY}}(u, v) \approx \frac{|N(u) \cap N(v)|}{\max(\deg u, \deg v)} \cdot (1 - \alpha)^2 + (\text{correcciones de borde})$$

Costo. $O(\deg u + \deg v)$ por arista, $O(E \cdot \bar{d})$ total. Implementación en `sotfs-monitor/src/curvature.rs:75`.

Algoritmo

Algoritmo 15: OLLIVIERRICCILLY

Entrada: Grafo G , arista (u, v) , parámetro α

Salida: Aproximación de $\kappa_{\text{LLY}}(u, v)$

```

15.1 common  $\leftarrow |N(u) \cap N(v)|$ ;
15.2  $d_u \leftarrow \deg u$ ;  $d_v \leftarrow \deg v$ ;
15.3 base  $\leftarrow \text{common} / \max(d_u, d_v)$ ;
15.4 self_transport  $\leftarrow (1 - \alpha)^2$ ;
15.5  $\kappa \leftarrow 1 - (1 - \text{base} \cdot \text{self\_transport})$ ;
15.6 retornar  $\kappa$ ;

```

Interpretación.

- $\kappa = 1$: arista en una clique (los vecindarios coinciden).
- $\kappa = 0$: arista en un grafo regular “neutro”.
- $\kappa < 0$: arista en una región tree-like (cuellos de botella, dirs profundos).

12.4 Detección de anomalías

Definición 12.3 (*Anomalía por curvatura*)

Una arista (u, v) es *anómala* si

$$|\kappa(u, v) - \bar{\kappa}| > 2 \sigma_{\kappa}$$

donde $\bar{\kappa}$ y σ_{κ} son la media y desviación estándar de la curvatura sobre todas las aristas del grafo.

Firmas observadas (en `sotfs-monitor/src/curvature.rs:173`):

- *Mass file creation*: spike positivo de κ en el directorio padre (sus vecindarios crecen mucho).
- *Symlink bomb*: κ fuerte positivo en aristas `Symlink` $\rightarrow$ `target`.
- *Directorio profundo*: $\kappa \approx -1.0$ en el cuello de la cadena.

12.5 El spec TLA de curvatura

`formal/sotfs_curvature.tla` (301 LOC) modela la curvatura como una función computada del estado y prueba dos cosas:

```
SafetyInvariant ==
  ∧ TypeInvariant
  ∧ CurvatureBounded
  ∧ KappaConsistent
  ∧ CurvatureDropImpliesFanOut
```

Listing 12.1: Invariantes de curvatura en TLA.

CurvatureBounded $|\kappa(u, v)| \leq \max(\deg u, \deg v)$.

KappaConsistent el cálculo es determinista: dos invocaciones sobre el mismo estado dan el mismo resultado.

CurvatureDropImpliesFanOut si entre dos estados $\Delta\kappa < -T$ para alguna arista, entonces *en el mismo paso* ocurrió un evento de fan-out (un nodo aumentó su grado en $\geq \text{FanOutDelta}$).

TLC verifica que estas tres se mantienen bajo todas las acciones del spec, incluyendo creates masivos y unlinks.

Resumen del capítulo

- *Treewidth* mide qué tan tree-like es el grafo; sotFS lo aproxima con min-degree elimination ordering en $O(n^2)$ (`sotfs-monitor/src/treewidth.rs:33`).
- Con `LINK_MAX = 65535`, $\text{tw}(G) \leq \text{LINK_MAX} + O(1)$; sin hard links, ≤ 6 .
- *Curvatura de Ollivier-Ricci* mide cuán “positivamente curvada” es cada arista (Wasserstein-1 entre vecindarios). sotFS usa la aproximación Lin-Lu-Yau con $\alpha = 0.5$, costo $O(\deg u + \deg v)$ por arista.
- Anomalías: aristas con $|\kappa - \bar{\kappa}| > 2\sigma$. Firmas distintivas para mass-create, symlink bomb, dir profundo.
- `formal/sotfs_curvature.tla` verifica tres invariantes: bounded, consistent, drop-implies-fan-out.

Ejercicios

Ejercicio 12.1 [★]

Defina *treewidth* en una línea. ¿Para qué algoritmos sirve tenerlo acotado?

Ejercicio 12.2 [★]

¿Cuál es el valor de α usado por sotFS en la lazy random walk de Ollivier-Ricci?

Ejercicio 12.3 [**]

Calcule la curvatura aproximada (Lin-Lu-Yau, $\alpha = 0.5$) de una arista en un grafo triangular (vecindarios coinciden completamente).

Ejercicio 12.4 [**]

Calcule la curvatura aproximada de una arista que une dos hojas de un árbol binario completo. ¿Por qué da negativa?

Ejercicio 12.5 [**]

Demuestre el teorema 12.1 para el caso sin hard links (`link_count` ≤ 1 para todos los inodes). Pista: identifi­ca los seis tipos de nodo como candidatos a bag.

Ejercicio 12.6 [**]

Considere un workload adversarial que crea 10^4 archivos en un solo directorio. ¿Qué firma de curvatura esperamos en el inode del directorio? ¿En las aristas contains salientes?

Ejercicio 12.7 [* * *]

`formal/sotfs_curvature.tla` prueba que un drop de curvatura implica un fan-out en el mismo paso. ¿Vale la implicación inversa (fan-out implica drop)? Justifique.

Ejercicio 12.8 [* * *]

Diseñe un detector de anomalías que use *ambos* treewidth y curvatura. ¿Cómo combinás las dos señales? Pista: un cambio brusco de treewidth con κ estable indica algo distinto a un κ shift con treewidth estable.

Decepción: el functor de proyección

Este capítulo presenta una pieza del runtime que es deliberadamente provocadora: el *functor de decepción* D_{FS} del el crate `sotfs-monitor`, que mapea un `TypeGraph` en otro `TypeGraph` *visible* a un dominio distinto. La motivación es seguridad defensiva: presentar a un atacante (o sondeador) una versión filtrada o adulterada del filesystem, sin tocar el grafo real.

13.1 Decepción como functor

Definición 13.1 (*Functor de decepción*)

$D_{\text{FS}} : \text{TypeGraph} \rightarrow \text{TypeGraph}$ es un functor que toma un `TypeGraph` G y un *perfil de proyección* π , y produce un `TypeGraph` $D_{\text{FS}}^{\pi}(G)$ que satisface:

1. Es *well-formed*: $\text{WellFormed}(D_{\text{FS}}^{\pi}(G))$.
2. Es *compatible con observación*: las syscalls del dominio engañado retornan resultados que son consistentes con la proyección.

13.2 Los cuatro modos de proyección

`sotfs-monitor/src/deception.rs:16` define cuatro variantes:

Passthrough. La identidad: $D_{\text{FS}}^{\text{id}}(G) = G$. Útil para A/B testing del monitoreo.

Restrict. Subgrafo inducido por un subtree rooted en `root_dir`. El dominio ve sólo lo que está debajo de ese root.

Fabricate. Inyecta nodos y aristas sintéticos *honeypots*. Cuatro perfiles pre-built: AWS credentials, Kubernetes tokens, Active Directory NTDS, CI/CD secrets (`sotfs-monitor/src/deception.rs:3`).

Redirect. Path resolution alterada: cierto path apunta a un inode distinto del real. Útil para servir contenido distinto a procesos engañados.

13.3 Lemma de well-formedness

Lema 13.1 (D_{FS} *preserva well-formedness*)

Para todo perfil $\pi \in \{\text{Passthrough}, \text{Restrict}, \text{Fabricate}, \text{Redirect}\}$ y todo G *well-formed*, $D_{\text{FS}}^{\pi}(G)$ es *well-formed*.

Esquema. • **Passthrough:** trivial.

- **Restrict:** el subgrafo inducido por BFS desde `root_dir` preserva todos los invariantes locales (`link-count` se recalcula sobre el subgrafo).
- **Fabricate:** los honeypots se inyectan con `link_count` y `refcount` correctos por construcción.

- **Redirect:** el target del redirect tiene que existir en G , con lo que la arista contains apunta a un inode vivo.

□

13.4 Detectabilidad por timing

Trampa 13.1 (*Restrict es detectable por timing*)

sotfs-monitor/src/deception.rs:716. Bajo Restrict, las operaciones son $\sim 1.7\times$ más rápidas que en el grafo real (porque la BFS es menor). Un atacante que mida stat en directorios de profundidad conocida puede distinguir el perfil.

Mitigación. Restrict debe combinarse con Fabricate (que añade latencia sintética) o con una implementación que igualó el costo a un baseline esperado.

Lección. La indistinguibilidad estructural no implica indistinguibilidad de timing. Cualquier proyección que reduzca el trabajo computacional es detectable.

13.5 Honeypots: perfiles pre-built

sotfs-monitor/src/deception.rs:283 contiene los cuatro perfiles Fabricate:

AWS profile. Inyecta `/.aws/credentials`, `/.aws/config`, etc. con strings que parecen access keys.

Kubernetes profile. `/var/run/secrets/kubernetes.io/serviceaccount/token`, `ca.crt`, `namespace`.

Active Directory profile. `NTDS.dit` fake, registry hives.

CI/CD profile. `.env` con `GITHUB_TOKEN`, `NPM_TOKEN`, etc.

El truco operativo. El honeypot no es sólo un archivo: es un nodo con su `record_prov` ligado. Cada read de un inode honeypot se registra al provenance log con un flag especial. El SIEM consume ese feed y alerta.

Resumen del capítulo

- D_{FS} es un functor que mapea un TypeGraph en otro TypeGraph proyectado, según uno de cuatro perfiles.
- Los cuatro perfiles: Passthrough, Restrict, Fabricate, Redirect.
- lema 13.1: cualquier proyección preserva well-formedness.
- trampa 13.1: las proyecciones que reducen trabajo computacional son detectables por timing.
- Honeypots pre-built (AWS, K8s, AD, CI/CD) son fabricated nodes con provenance ligado para alertas.

Ejercicios

Ejercicio 13.1 [*]

Liste los cuatro perfiles de proyección de D_{FS} .

Ejercicio 13.2 [**]

Demuestre el caso `Restrict` del lema 13.1: que el link-count del subgrafo recalculado preserva el invariante.

Ejercicio 13.3 [**]

La trampa 13.1 habla de detección por timing. Diseñe un script que mida `stat` sobre un path conocido y detecte si está corriendo bajo `Restrict`.

Ejercicio 13.4 [* * *]

Diseñe un quinto perfil `Hybrid` que combine `Restrict` + `Fabricate` con latencia compensada para evitar el timing side-channel. Bosqueje el algoritmo y la prueba de well-formedness.

Adaptador FUSE y observabilidad

Este capítulo cierra el loop: una syscall POSIX que el usuario hace contra el mount point `sotFS`, ¿cómo se traduce a una regla DPO sobre el `TypeGraph` y luego de vuelta en una respuesta? La respuesta vive en el crate `sotfs-fuse` y conecta todo lo anterior: el grafo del cap. 3, las reglas del cap. 5, la persistencia del cap. 7, las capabilities del cap. 8 y las transacciones del cap. 9.

14.1 El callback FUSE: anatomía

El crate `sotfs-fuse` implementa `Filesystem` del crate `fuser`. Cada syscall mapeable se traduce en un callback con la siguiente forma:

Algoritmo 16: SKELETON DE UN CALLBACK FUSE EN SOTFS

Entrada: Request *req*, parámetros POSIX, ReplyXxx

```
16.1 ctx ← CTXFROMREQ(req);
16.2 g.SETCAPCTX(ctx);
16.3 handles ← g.RESOLVE(paths/inodes del req);
16.4 result ← DPOOP(g, handles, ...);
16.5 si result = Ok entonces
16.6 |   reply.ok(...);
16.7 sino
16.8 |   reply.error(MAPERRTOERRNO(result));
16.9 fin
16.10 si SOTFS_PROV_SIDE CAR configurado entonces
16.11 |   PERSISTPROVLOG(g);
16.12 fin
```

14.2 Mapeo VFS → DPO

14.3 El CapContext desde una syscall

Recordando del cap. 8, cada op corre bajo un `CapContext`. La traducción FUSE → context vive en `sotfs-fuse/src/fs.rs:59`:

```
fn ctx_from_req(req: &Request<'_>) -> CapContext {
    CapContext::new(None, req.uid() as u64)
}
```

Versión actual minimalista: `cap_id = None` y `domain_id = uid`. Una extensión natural (no implementada en v0.2.5) usa el `pid` o un `thread-local` con la `cap` activa explícita. `commit 61e7faf` cerró el gap más urgente: pre-v0.2.5 el contexto se descartaba al entrar a las DPO ops; ahora se propaga al `record_prov`.

14.4 Provenance log: el sidecar JSONL

VFS callback	DPO op invocada
create	create_file (sotfs-ops/src/lib.rs:33)
mkdir	mkdir (sotfs-ops/src/lib.rs:94)
unlink	unlink (sotfs-ops/src/lib.rs:322)
rmdir	rmdir (sotfs-ops/src/lib.rs:198)
rename	rename (sotfs-ops/src/lib.rs:404)
link	link (sotfs-ops/src/lib.rs:272)
read	read_data (sotfs-ops/src/lib.rs:704)
write	write_data (sotfs-ops/src/lib.rs:670)
setxattr/getxattr/listxattr	*xattr
symlink/readlink	symlink/readlink
getattr/setattr	lookup directo + chmod/chown/truncate

Cuadro 14.1: Cada VFS callback de FUSE se traduce en una llamada a una o más funciones DPO de el crate sotfs-ops.

Definición 14.1 (*ProvenanceEntry*)

Un *provenance entry* es un registro JSONL con la siguiente estructura (sotfs-graph/src/provenance.rs:59)

```
struct ProvenanceEntry {
    t: u64,           // unix timestamp seconds
    op: ProvOp,       // {Create, Mkdir, Unlink, ...}
    inode: u64,
    cap: Option<u64>,
    domain: u64,
    detail: String,
}
```

Cuándo se escribe. Si la env var SOTFS_PROV_SIDE CAR apunta a un path, cada DPO op exitosa emite una entry. El sidecar es append-only JSONL: una línea por entry.

El bug v0.2.3 → v0.2.4. Pre-v0.2.4 el formato escribía op:Create (sin comillas) y cap:Some(7); no era JSON válido. v0.2.4 cambió a serde_json::to_string que produce JSON spec-compliant: "op": "Create", "cap": 7. Parsers legacy rompen; consumidores nuevos no.

14.5 Graph Hunter: exporter para threat-hunting

el crate sotfs-cli tiene un binario sotfs-export-hunter con dos modos:

Snapshot mode

```
sotfs-export-hunter /tmp/test.redb -o hunter.json
```

Carga el grafo de REDB, lo convierte a formato *Graph Hunter temporal multigraph*, y emite un JSON con dos secciones: meta y events.

```
{
  "meta": {
    "format": "graph-hunter-temporal",
    "version": 1,
    "node_count": 9,
    "edge_count": 10,
    "node_types":
      ["inode", "directory", "capability", "block", "version"],
    "edge_types": ["contains", "grants", "delegates",
                  "derivesFrom", "supersedes", "pointsTo", "hasXattr"]
  },
  "events": [
    { "t": 1778345083, "op": "add_node", "id": "I1",
      "type": "inode", "vtype": "dir", "link_count": 1, ... },
    { "t": 1778345083, "op": "add_edge", "id": 1,
      "src": "D1", "tgt": "I1", "type": "contains", "name": "." }
  ]
}
```

Tail mode (streaming)

```
sotfs-export-hunter --tail /tmp/test.prov.jsonl
sotfs-export-hunter --tail /tmp/test.prov.jsonl --once # batch
```

Tail-f sobre el sidecar JSONL escrito por SOTFS_PROV_SIDEAR, convierte cada entry a evento Graph Hunter NDJSON y emite a stdout. Útil para pipelining a un SIEM:

```
sotfs-export-hunter --tail /tmp/sotfs.prov.jsonl | \
jq -c '. | select(.cap == 42)'
```

14.6 Mount: opciones runtime

```
./target/release/sotfs-fuse /tmp/mnt --db /tmp/test.redb
```

Variables de entorno:

Variable	Efecto
SOTFS_FUSE_TTL_MS	TTL del cache de FUSE (default 1000 ms; o desactiva).
SOTFS_FUSE_ALLOW_OTHER	Si set, monta con AllowOther + AutoUnmount; default off (per-UID isolation).
SOTFS_PROV_SIDEAR	Path donde escribir el JSONL de provenance.

14.7 Resumen del capítulo

- Cada VFS callback de FUSE en el crate `sotfs-fuse` sigue el patrón: set cap-context → resolve → DPO op → reply → drain prov-log.

- `ctx_from_req` construye el `CapContext` desde el `Request`; commit 61e7faf cerró el plumbing hasta el `prov-log`.
- El `provenance log` es un JSONL sidecar opcional activado por `SOTFS_PROV_SIDE CAR`. Cada DPO op exitosa emite una entry.
- `sotfs-export-hunter` convierte snapshot o tail-streaming a JSON Graph Hunter temporal multigraph, consumible por SIEMs.

Ejercicios

Ejercicio 14.1 [*]

Liste las tres variables de entorno que controlan el comportamiento de `sotfs-fuse`.

Ejercicio 14.2 [*]

¿Qué op DPO invoca el callback `rename` de FUSE?

Ejercicio 14.3 [**]

Diseñe un consumidor de `provenance` que cuente cuántos `write` ocurrieron en la última hora por dominio. Pista: `jq + grep + sort`.

Ejercicio 14.4 [**]

El `TTL_MS default` es 1000 ms. ¿Qué pasa si se setea a 0? ¿Por qué un usuario querría hacerlo (pista: benchmarking)?

Ejercicio 14.5 [* * *]

Extienda `ctx_from_req` para usar un `thread-local` de `cap` activa explícita, no sólo el `uid`. ¿Qué hace falta agregar al runtime para que el FUSE daemon mantenga la map “thread → cap”?

Catálogo completo de invariantes

Este apéndice consolida *todos* los invariantes que aparecen en el libro, en un solo lugar de consulta rápida. Cada entrada lista: nombre canónico, fórmula resumida, capítulo donde se introduce, dónde se chequea en el runtime, y la prueba formal correspondiente (TLA⁺ y/o Coq).

Invariantes del TypeGraph (cap. 3)

Nombre	Enunciado	Runtime	Formal
Link-count consistency	$i.\text{link_count} = \{e : \tau_E(e) = \text{contains}, \text{tgt}(e) = i, e.\text{name} \neq ".\text{.}"\} $	sotfs-graph/src/graph.rs:888	TLA formal/sotfs_graph.tla; Coq idem
Unique names per dir	$\forall d \in \text{Directory}, \forall e_1 \neq e_2 \in E_d^{\text{contains}} : e_1.\text{name} \neq e_2.\text{name}$	sotfs-graph/src/graph.rs:916	TLA UniqueNamesPerDir; Coq idem
Dir self-reference	$\forall d \in \text{Directory}, \exists e : \text{src}(e) = d, e.\text{name} = ".\text{.}", \text{tgt}(e) = d.\text{inode_id}$	sotfs-graph/src/graph.rs:938	TLA DirHasSelfRef; (no en Coq)
No dangling edges	$\forall e \in E : \text{src}(e), \text{tgt}(e) \in V$	sotfs-graph/src/graph.rs:964	TLA NoDanglingEdges; Coq idem
Block refcount	$b.\text{refcount} = \{e : \tau_E(e) = \text{pointsTo}, \text{tgt}(e) = b\} $	sotfs-graph/src/graph.rs:996	TLA BlockRefcountConsistent; (no en Coq)
No dir cycles	La sub-relación contains restringida a directorios (sin ./..) es acíclica	sotfs-graph/src/graph.rs:1015	TLA NoDirCycles; Coq idem
Cap monotonicity	$\forall c, c' \in \mathbb{M}$ con $\text{delegates}(c, c') : c'.\text{rights} \subseteq c.\text{rights}$	sotfs-graph/src/graph.rs:1056	TLA CapMonotonicEnuation; (no en Coq)

Invariantes de transacciones (cap. 9)

Nombre	Enunciado	Runtime	Formal
Persistencia tras fsync	Tras fsync(loc) exitoso, los bytes están en NAND	(asumido del backend)	—

Read isolation	Un read concurrente con un commit ve o el estado pre-tx o el post-tx, nunca intermedio	RCU (sotfs-graph/src/rcu.rs:117)	TLA formal/sotfs_transactions.t
No partial application	Tras Recover: toda op de una GTXN aplicada o ninguna	N/A (mode-lo)	TLA formal/sotfs_crash.tla
Durability	Si gtxn_commit retornó Ok, sobrevive cualquier crash	N/A (mode-lo)	TLA idem
Recovery consistency	Tras Recover, los 7 invariantes del TypeGraph se preservan	N/A (mode-lo)	TLA idem
WAL integrity	El WAL no se modifica entre commit marker y GC; GC borra sólo entradas aplicadas	N/A (mode-lo)	TLA idem

Invariantes de hashing (cap. 4)

Identidad por contenido	Dos nodos con el mismo $h(\cdot)$ son el mismo nodo	Por construcción	—
Acíclico (Merkle)	Las referencias entre nodos son por hash; ciclo accidental con probabilidad 2^{-256}	Por construcción	—
Inmutabilidad estructural	Un nodo nunca se modifica in-place; las mutaciones crean nodos nuevos	DPO cap. 5	+ —

Invariantes de topología (cap. 12)

Treewidth acotado	$tw(G) \leq \text{LINK_MAX} + O(1)$	sotfs-monitor/src/treewidth.rs:33	—
Curvature bounded	$ \kappa(u, v) \leq \max(\deg u, \deg v)$	sotfs-monitor/src/formal/sotfs_75	TLA curvature.tla
Kappa consistent	κ es función determinista del estado	idem	TLA idem
Curvature drop $\Rightarrow$ fan-out	$\Delta\kappa(u, v) < -T \Rightarrow$ fan-out en el mismo paso	idem	TLA idem

Deudas formales explícitas

- NoHardLinkToDir *no* está formalizada en formal/coq/SotfsGraph.v (cap. 11). Bloquea tres Admitted en formal/coq/DpoRmdir.v.
- DirHasSelfRef y BlockRefCountConsistent están en TLA pero *no* en Coq.

Tablas DPO por operación POSIX

Para cada operación POSIX cubierta en el cap. 5, este apéndice da la tripleta (L, K, R) y la gluing condition formal en una tabla compacta. Usar como referencia rápida al leer el crate `sotfs-ops`.

`create_file`

L	$\{d\}$ (un Directory)
K	L
R	$\{d, i_{\text{new}}\} + \text{arista } d \xrightarrow{\text{contains}, n} i_{\text{new}}, i_{\text{new}}.\text{link_count} = 1$
Gluing	$\neg \exists e \in E_d^{\text{contains}} : e.\text{name} = n$
Errno	EEXIST
Coq	<code>formal/coq/DpoCreate.v, create_preserves_WellFormed</code> (línea 335)

`mkdir`

L	$\{d_p, i_p\}$ (dir padre + su inode)
K	L
R	$L + \{d_{\text{new}}, i_{\text{new}}\} + 3 \text{ aristas contains: } \text{padre} \xrightarrow{n} \text{nuevo}, \text{nuevo} \rightarrow \text{nuevo}, \text{nuevo} \rightarrow \text{padre.inode}$
Gluing	$\neg \exists e \in E_{d_p}^{\text{contains}} : e.\text{name} = n$
Errno	EEXIST
Coq	<code>formal/coq/DpoMkdir.v</code> línea 572

`link (hard link)`

L	$\{d, i\}$
K	L
R	$L + \text{arista } d \xrightarrow{\text{contains}, n} i; i.\text{link_count} += 1$
Gluing	$\neg \exists e \in E_d^{\text{contains}} : e.\text{name} = n \wedge i.\text{link_count} < \text{LINK_MAX}$
Errno	EEXIST, EMLINK
Coq	<code>formal/coq/DpoLink.v</code> línea 369

unlink

L	$\{d, i\} + \text{arista } d \xrightarrow{n} i$
K	$\{d\}$ si $i.\text{link_count} = 1$, $\{d, i\}$ si > 1
R	K ; si $i \in K$, $i.\text{link_count} -= 1$; remover arista
Gluing	$\exists e : \text{src}(e) = d, e.\text{name} = n \wedge i.\text{vtype} \neq \text{Directory}$
Errno	ENOENT, EISDIR
Coq	formal/coq/DpoUnlink.v línea 300 (versión keep)

rmdir

L	$\{d_p, d, i\} + 3 \text{ aristas (contains padre} \rightarrow \text{inode, self-ref, parent-ref)}$
K	$\{d_p\}$
R	K
Gluing	$E_d^{\text{contains}} = \{d \dot{\rightarrow} i, d \dot{\rightarrow} d_p.\text{inode}\}$ (dir vacío)
Errno	ENOTEMPTY, ENOENT
Coq	formal/coq/DpoRmdir.v 3 Admitted (líneas 349, 405, 505)

rename (same-dir)

L	$\{d, i\} + \text{arista } d \xrightarrow{n} i$
K	$\{d, i\}$
R	$K + \text{arista } d \xrightarrow{n'} i$ (misma EdgeId, reescribe atributo name)
Gluing	$\neg \exists e \in E_d^{\text{contains}} : e.\text{name} = n'$
Errno	EEXIST, ENOENT
Coq	formal/coq/DpoRename.v línea 304

rename (cross-dir)

L	$\{d_1, d_2, i\} + \text{arista } d_1 \xrightarrow{n} i$
K	$\{d_1, d_2, i\}$
R	$K + \text{arista } d_2 \xrightarrow{n'} i$; si i es dir, actualiza su . .
Gluing	$(\neg \exists e \in E_{d_2}^{\text{contains}} : e.\text{name} = n') \wedge (\text{si } i \text{ es dir, } i \notin \text{ancestors}(d_2))$
Errno	EEXIST, EINVAL
Coq	idem que same-dir (la prueba cubre ambos casos)

write (un bloque)

L	$\{i\}$
K	L
R	$L + \{b\} + \text{arista } i \xrightarrow{\text{pointsTo, off}=k} b; b.\text{refcount} = 1$
Gluing	$i.\text{vtype} = \text{Regular}$
Errno	EISDIR, ENOSPC
Coq	— (no formalizado en vo.2.5)

Resumen

De las siete operaciones, seis tienen prueba Coq completa de preservación de `WellFormed`. La séptima, `write`, todavía no está formalizada en Coq. `rmdir` tiene su teorema con tres `Admitted` (cf. cap. 11).

Resultados de model checking con TLC

Este apéndice consolida los resultados públicos de TLC sobre los seis specs TLA⁺ de sotFS (v0.2.5). La fuente de verdad es `formal/tlc_output/summary.txt`, generado por `formal/run_tlc.sh`.

Tabla maestra

Spec	Config	Estados	Tiempo	Veredicto
formal/sotfs_graph.tla	small	~ 10 ⁵	< 30 s	PASS
	medium	~ 10 ⁷	~ 5 min	PASS
	large	~ 10 ⁹	varies	timeout-dependent
formal/sotfs_transaction.tla	small	~ 10 ⁴	< 10 s	PASS
	medium	~ 10 ⁶	~ 1 min	PASS
	large	~ 10 ⁸	varies	timeout-dependent
formal/sotfs_capabilities.tla	small	~ 10 ⁴	< 10 s	PASS
	medium	~ 10 ⁶	~ 1 min	PASS
	large	~ 10 ⁸	varies	timeout-dependent
formal/sotfs_crash.tla	small	~ 10 ⁵	< 30 s	PASS
	medium	~ 10 ⁷	~ 3 min	PASS
	large	~ 10 ⁹	varies	timeout-dependent
formal/sotfs_crash_refinement.tla	small	~ 10 ⁴	< 15 s	PASS (refinement OK)
	medium	~ 10 ⁶	~ 2 min	PASS
	large	—	—	no se corre rutinariamente
formal/sotfs_curvature.tla	small	~ 10 ³	< 5 s	PASS
	medium	~ 10 ⁵	< 30 s	PASS
	large	~ 10 ⁷	varies	timeout-dependent

Resumen oficial v0.2.5

14/14 PASS en small y medium (todos los specs, todas las invariantes). La config large es opcional y puede TIMEOUT dependiendo del host. El gating de CI es medium.

Cómo reproducir

```
# Setup (una vez).
mkdir -p formal/lib
curl -L -o formal/lib/tla2tools.jar \
  https://github.com/tlaplus/tlaplus/releases/latest/\
download/tla2tools.jar
```

```
# Run all sizes.
just formal

# Sólo small (para CI rápida).
cd formal && bash run_tlc.sh small

# Sólo un spec.
cd formal && bash run_tlc.sh graph
```

Limitaciones declaradas

- Los números de estados son aproximaciones para configs típicas; el spec exacto depende del seed y del orden de exploración.
- Las configs large suelen requerir 32+ GiB de RAM para completarse; no se corren en CI.
- El refinement de `formal/sotfs_crash_refinement.tla` verifica que el modelo concreto refina el abstracto bajo stuttering, no *equivalencia*: el concreto puede tener trazas extra (Crash, Recover) que el abstracto no.

Capítulo D

Setup reproducible

Este apéndice documenta cómo levantar de cero un entorno para correr todo: build de Rust, fuzzing, TLC, Coq, FUSE mount, y compilación de este libro.

Sistema base

Linux Fedora 40+ recomendado (todas las instrucciones asumen dnf). Adaptaciones para Debian/Ubuntu en una nota al final.

Rust toolchain

```
# Stable + nightly (nightly necesario para cargo-fuzz).
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
rustup install stable nightly
rustup default stable

# Componentes para coverage.
rustup component add llvm-tools-preview
cargo install cargo-llvm-cov cargo-fuzz
```

FUSE

```
sudo dnf install fuse3 fuse3-devel
# Permitir mounts no-root.
sudo modprobe fuse
sudo usermod -aG fuse $USER
```

TLA⁺

```
sudo dnf install java-17-openjdk
mkdir -p formal/lib
curl -L -o formal/lib/tla2tools.jar \
  https://github.com/tlaplus/tlaplus/releases/latest/\
  download/tla2tools.jar
```

Coq

```
sudo dnf install coq
# o vía opam para versión más reciente:
# opam install coq.8.18.0
```

LaTeX para el libro

```
sudo dnf install \
  texlive-scheme-full \
  biber chktex
```

Comandos clave

```
# Build entero del workspace.
cargo build --release --workspace

# Tests (rápido).
just test

# Tests completos (proptest 10k cases).
just test-full

# Fuzz 60s por target.
just fuzz

# Coverage.
just coverage

# TLC.
just formal

# Coq.
cd formal/coq && coq_makefile -f _CoqProject -o Makefile && make

# Mount.
just mount /tmp/test.redb /tmp/sotfs-mnt
# ... usar como un FS normal ...
just unmount /tmp/sotfs-mnt

# Libro PDF.
just book          # → docs/book/build/main.pdf
just book-watch   # live preview
just book-clean
```

Verificación end-to-end

Una sesión típica de validación de un PR:

```
just test-full      # ~3 min
just formal         # ~5 min (medium configs)
just fuzz           # ~6 min (60s × 6 targets)
just book           # ~30 s (PDF)
```

Notas Debian/Ubuntu

Mapping de paquetes:

Fedora	Debian/Ubuntu
fuse3 fuse3-devel	fuse3 libfuse3-dev
java-17-openjdk	default-jdk
coq	coq
texlive-scheme-full	texlive-full

Soluciones a ejercicios seleccionados

Este apéndice da soluciones cortas a los ejercicios de dificultad $[*]$ y $[**]$. Para los $[***]$ se ofrece sólo un *hint* de 2-3 líneas; las pruebas y diseños completos se dejan al lector.

Capítulo 1

1.1. Inode, Directory, Capability, Transaction, Version, Block.

1.2. (i) Hard links como bolt-on, no parte del modelo. (ii) Deduplicación post-escritura es cara. (iii) Snapshots requieren COW/shadow paging.

1.3. En el árbol POSIX, las hard links son una extensión: dos entradas de directorio apuntan al mismo inode. En sotFS, emergen del modelo: el mismo nodo Inode es target de varias aristas contains con name distintos.

1.4. Si dos archivos de 1 MiB tienen 80% en común, son ~ 205 bloques de 4 KiB cada uno. En ext4 ocupan $2 \text{ MiB} = 512$ bloques. En sotFS, los 205 bloques compartidos se deduplican: total $205 + 51 + 51 = 307$ bloques. Ahorro: $\sim 40\%$.

1.5. *Hint.* Tres ejemplos: (i) preservación de los 7 invariantes tras cada op (TLA⁺ y Coq); (ii) crash refinement (TLA⁺); (iii) cap monotonicity (TLA o Coq). ext4 no formaliza ninguna.

Capítulo 2

2.1. Un morfismo $f : G \rightarrow H$ es un par (f_V, f_E) que conmuta con src y tgt.

2.2. 2^{128} (cumpleaños).

2.3. $D = \{[1 = x], [2 = y], 3, z\}$ con cuatro elementos. $h(1) = [1 = x]$, $h(2) = [2 = y]$, $h(3) = 3$; $k(x) = [1 = x]$, $k(y) = [2 = y]$, $k(z) = z$.

2.4. $L = \{v_1, v_2\}$ (dos nodos), $K = \{v_1, v_2\}$, $R = \{v_1, v_2\} + \text{arista } e : v_1 \rightarrow v_2$. $K \hookrightarrow L$ y $K \hookrightarrow R$ son las inclusiones de nodos.

2.5. Sea $G = \{a, b, c\}$ con aristas (a, b) y (b, c) . Sea $L = \{x\}$ con $m(x) = b$, $K = \emptyset$. La gluing condition Dangling falla porque b tiene dos aristas incidentes y ninguna está en $m(L)$.

2.6. Si dos Directory distintos del LHS se mapean al mismo Directory del host, la regla quiere borrar y mantener el mismo nodo simultáneamente, violando Identification.

2.7. DPO con `unlink` sobre un `hard-linked` con `link_count > 1`: el `inode` no se borra; SPO podría dejar aristas colgando si la regla no las maneja explícitamente. En `sotFS` la regla DPO maneja ambos casos en su lógica.

Capítulo 3

3.1. `Link-count`, `unique-names`, `dir-self-ref`, `no-dangling`, `block-refcount`, `no-dir-cycles`, `cap-monotonicity`. El numérico (cuenta) es `link-count` y `block-refcount`.

3.2. Ver tabla 3.2 del cap. 3.

3.3. Después de `ln /a/x.txt /a/z.txt`, $i_x.\text{link_count} = 2$. Dos aristas `contains` entrantes (con `name x.txt` y `z.txt`), excluyendo `...`

3.4. Si `link_count = 0`, entonces el conjunto de aristas `contains` entrantes (excluyendo `...`) tiene cardinalidad 0, es decir, está vacío.

3.5. Se rompe el invariante 3.3. Se detecta en `check_dir_self_ref` (`sotfs-graph/src/graph.rs:936`).

3.6. Mantener `dir_for_inode_idx` cuesta $O(\log N)$ por `insert_dir/remove_dir`. Recorrer linealmente cuesta $O(N)$ por `lookup`. El `break-even` es trivialmente favorable al índice salvo `workloads` `casi-readonly`.

3.7. Tres nodos: dos $\cap$ (c_p con `rights WRITE`, c_h con `rights WRITE | EXECUTE`), una arista `delegates` de c_p a c_h . `check_cap_monotonicity` detecta que `EXECUTE` no está en $c_p.\text{rights}$.

3.8. *Hint.* Correrlo en el GC (no en cada DPO step) porque el costo es $O(|\text{Block}|)$. Aceptable como chequeo periódico, no como pre-condition.

3.9. *Hint.* El invariante: “ $\forall i \in \text{Inode}, \exists e \in E^{\text{auditedBy}}$ desde i a un `Audit`.” Modificar `write`, `create_file`, `rename`. Extender `formal/sotfs_graph.tla` con `audits` en variables.

Capítulo 4

4.1. 32 bytes (256 bits). Cota cumpleaños: 2^{128} .

4.2. (i) Computar hash del bloque escrito; (ii) buscar en `storage` si ya existe; (iii) agregar arista `pointsTo` o crear nodo + arista.

4.3. 80% de 256 bloques = ~ 205 compartidos + 51 + 51 únicos = 307 bloques totales en `sotFS` vs. 512 en `ext4`. Ahorro 40%.

4.4. Si `serialize` no es canónica, $h(v_1) \neq h(v_2)$ aun siendo lógicamente iguales. Ej.: serializar un `Inode` con campos en orden `A, B, C` vs. `B, A, C`.

4.5. *Hint.* Tri-color: blanco (no visto), gris (visto, pendiente), negro (procesado). Write barrier: toda nueva referencia desde negro debe colorear el destino gris para evitar perderlo en la sweep.

Capítulo 5

5.1. La gluing condition exige que (i) si dos elementos del LHS se mapean al mismo elemento del host, ambos están en el glue; (ii) no quedan aristas colgando al borrar lo que sólo vive en L .

5.2. `create_file`: $K = \{d\}$. `mkdir`: $K = \{d_p\}$. `link`: $K = \{d, i\}$. `unlink`: $K = \{d\}$ o $\{d, i\}$. `rmdir`: $K = \{d_p\}$. `rename`: $K = \{d, i\}$ (same-dir) o $\{d_1, d_2, i\}$ (cross). `write`: $K = \{i\}$.

5.3. Una Symlink aparece como Inode de tipo Symlink con `symlink_target` en un mapa separado. Producción: $L = \{d\}$, $K = L$, $R = L + \{i_s\} +$ arista $d \xrightarrow{n} i_s$. Gluing: $\neg \exists e : \text{src}(e) = d, e.\text{name} = n$. Idéntica a `create_file` salvo el vtype del inode nuevo.

5.4. En el spec, si $i \notin \text{inodes}$, la guard $i \in \text{inodes}$ es falsa $\Rightarrow$ la acción no está habilitada. POSIX retornaría ENOENT. Coincide.

5.5. Al desaparecer i , sus aristas adyacentes (contains entrantes y pointsTo salientes) se borran en el mismo paso DPO. No quedan dangling.

5.6. La NAC: $\neg \exists e \in E_{d_p}^{\text{contains}} : e.\text{name} = n$. Con `dir_name_idx`, un solo lookup en $O(\log N)$.

5.7. *Hint.* No es una sola regla DPO (`rmdir` requiere dir vacío); es una secuencia que recorre el subtree. La atomicidad la da la GTXN del cap. 9.

5.8. *Hint.* Reemplazar el traversal por una consulta al índice `dir_for_inode_idx` sobre cada ancestro de d' . Sigue siendo posible un DoS con cadenas largas; el fix completo requiere un índice de ancestros transitivos.

5.9. *Hint.* intros, casos sobre Hwf, aplicar `create_preserves_TypeInvariant + create_preserves_Lin` + tres más.

Capítulo 6

6.1. `NODE_CAPACITY = 65536`, `EDGE_CAPACITY = 131072`.

6.2. 64 (`MAX_READERS`); más de eso panic.

6.3. Siete arenas $\times$ (header + slots vacíos). $65536 \times$ (tamaño slot) por arena. Para Inode (~ 56 bytes) son ~ 3.7 MiB. Total ~ 35 MiB.

6.4. Si los lectores llegan más rápido que la sincronización, la cola de epochs nunca llega a 0. En la práctica los lectores terminan rápido (lookups $O(\log N)$); el riesgo es teórico, no observado.

6.5. (i) Al alocar, se setea `state[i] = Occupied` y se escribe `data[i] = v` en el mismo bloque. (ii) Al liberar, se setea `state[i] = Vacant` y se push al free-list. (iii) Bump-alloc inicializa antes de poner `Occupied`.

6.6. Test: alocar N slots, verificar `state[i] = Occupied`; liberar $N/2$ random, verificar que el free-list tiene $N/2$ entries; alocar $N/2$ más, verificar que los nuevos slots vienen del free-list (LIFO), no por bump.

6.7. Sí: una arena `Arena<DirInodeMapping, NODE_CAPACITY>`. Pero `BTreeMap` da búsqueda $O(\log N)$ por key arbitraria, no por `InodeId` contiguo. Tradeoff: arena es más compacta, `BTreeMap` es más flexible.

6.8. *Hint.* Per-CPU array de `AtomicU64` pequeño + slow-path con `Mutex` para overflow. RCU sync recorre todos los per-CPU.

Capítulo 7

7.1. Una sola tabla, `METADATA_TABLE`; clave "graph" con valor un blob JSON.

7.2. Las claves tupla (`DirId`, `String`) no caben naturalmente en JSON. Y es un derivado, no estado canónico.

7.3. Para 10^5 inodes, el blob JSON pesa $\sim 20\text{-}50$ MiB. Save: $O(M)$ serialize + `redb txn` ~ 200 ms. Load: `deserialize` + `rebuild_dir_name_idx` $O(M \log M)$ ~ 500 ms.

7.4. Hacer save, load, 1000 ops random, luego comparar `dir_name_idx` con la versión recomputada desde `dir_contains`.

7.5. *Hint.* Tres tablas: nodes, edges, meta. Save y load más rápidos por columnas (no JSON parse), menos fragmentación, pero atomicidad cross-table requiere multi-table tx (`REDB` lo soporta).

Capítulo 8

8.1. READ, WRITE, EXECUTE, GRANT, REVOKE.

8.2. None = anonymous (no cap presentada); `Some(0)` = cap explícita con id 0 (un caso patológico — id 0 es sentinel).

8.3. Inducción sobre profundidad. Base $d = 0$: c mismo es revocada. Paso: si todos los descendientes en profundidad $\leq d$ están revocados, los de profundidad $d + 1$ (hijos de revocados) también lo están por la regla REVOKE.

8.4. El epoch cacheado por el atacante es distinto del corriente (porque la cap nueva tiene epoch incrementado, o la cap no existe). `CHECKACCESS` falla en línea 1.

8.5. Estado base: si no hay aristas delegates, la implicación es vacuamente cierta. Inducción: si la propiedad vale para todas las aristas existentes y `DERIVE` crea una nueva (c, c') con $c'.rights \subseteq c.rights$ por construcción.

8.6. *Hint.* Agregar campo `suspend_until` a $\mathbb{M}$. `CHECKACCESS` extra: `now ≥ suspend_until`. `formal/sotfs_capabilities.tla` crece con timestamp variable y `TemporalRevoke/Reactivate` actions.

Capítulo 9

9.1. Orden: `fsync(WAL data) → fsync(commit marker) → fsync(graph) → WAL truncate` (opcional, sin `fsync` obligatorio). Commit point: el segundo `fsync`. Crash entre 1 y 2: WAL sin marker $\Rightarrow$ recovery descarta.

9.2. (i) no-partial: ningún caso intermedio. (ii) durability: post-Ok sobrevive crash. (iii) recovery-consistency: post-Recover invariantes vivos. (iv) wal-integrity: marker $\Rightarrow$ no se modifica hasta GC. Más difícil: recovery-consistency (requiere los 7 invariantes preservados *en cada paso* del replay).

9.3. Todos los 7 invariantes del cap. 3. No: link-count solo no garantiza unique-names ni no-dir-cycles.

9.4. 10^6 inodes a ~ 56 B = ~ 56 MB solo en Inode. Total ~ 200 MB. `graph.clone()` copia todo: ~ 200 ms - 1 s en RAM moderna. Costoso bajo write-heavy.

9.5. Si invariante 9.1 se cumple, no puede forzar pérdida con power-cut. Si el SSD ignora FUA, sí: el attacker mata el power entre el ack del `fsync` (kernel cree persistido) y el flush real a NAND.

9.6. Si WAL y graph viven en el mismo SSD, una falla del controlador podría perder ambos. Lo importante es el *ordering*: el commit marker se persiste *antes* del graph apply, y el WAL se trunca *después*. Misma física, distinto tiempo lógico.

9.7. La closure recibe un `&mut TypeGraph` y muta directamente. Si retorna `Ok` sin llamar a `check_invariants`, el wrapper `with_transaction` sí lo hace (línea 64 de el crate `sotfs-tx`). Pero si la closure muta de un modo que viola un invariante y *captura* el error internamente retornando `Ok`, el rollback ya no dispara: la única red de seguridad es `check_invariants` *después* de la closure.

9.8. El runtime no escribe WAL; usa `redb txn`. La fila “Durabilidad” es una correspondencia entre dos primitivas distintas (`fsync` sobre WAL vs. `redb::commit` interno). El backend necesita atomicidad + durabilidad + isolation serializable (`REDB` las provee).

9.9. “Un read se linealiza en un punto antes o después del commit point, pero no entre los dos sub-pasos del commit”.

9.10. *Hint.* fork hijo → kill -9 a tiempo aleatorio → remontar el FS → `sotfsctl check`. Repetir 1000 veces; ningún check debería fallar.

9.11. *Hint.* sotFS: always-consistent (en modelo formal), 14/14 TLA pasos. ext4: data=journal es eventually-consistent. btrfs: COW da always-consistent pero sin spec formal. ZFS: idem btrfs + replicación.

9.12. *Hint.* Agregar nodo Coordinator con arista Prepares a cada Transaction. Extensión de `formal/sotfs_transactions.tla` con `PrepareAll/CommitAll/AbortAll`.

Capítulos 10–14

Por brevedad, los ejercicios de los caps. 10 a 14 quedan sólo con sus pistas; las soluciones completas son ejercicios de laboratorio.

10.1. Ver tabla 10.1.

10.2. $\{I_1, I_2, I_3\}, \{D_1, D_2\}, \{B_1, B_2\}, \{a, b\}, \text{MaxOps} = 4$.

10.3. (a) cláusula 3 con $\sim \exists$ sobre nombre. (b) `containsEdges'` y `linkCount'`. (c) `inodes`, `dirs`, `blocks`, ..., `dirInode`. Sin incremento de `linkCount`, falla `LinkCountConsistent`.

10.5. El espacio de estados crece exponencialmente con `containsEdges`: $\sim 2^{|D| \times |I| \times |N|}$ ya mata el budget. Por eso `MaxOps bounded`.

10.7. Cualquier invariante del modelo abstracto (no-partial-application, durability, recovery-consistency) se hereda al modelo concreto bajo refinement.

11.1. Siete: `formal/coq/SotfsGraph.v`, y los seis `Dpo*.v`.

11.2. `TypeInvariant`, `LinkCountConsistent`, `UniqueNamesPerDir`, `NoDanglingEdges`, `NoDirCycles`. Falta: `DirHasSelfRef`, `BlockRefCountConsistent`, `CapMonotonicity`.

11.3. Cinco lemmas en `formal/coq/DpoCreate.v`: líneas 147, 190, 236, 271, 298.

11.4. Pros: permite check-pointing manual durante desarrollo, declaración explícita de deuda. Contras: silencia gaps si no se busca activamente; `coqc` no falla.

11.5.

```

Definition NoHardLinkToDir (g : Graph) : Prop :=
  forall ce1 ce2,
    In ce1 (g_edges g) -> In ce2 (g_edges g) ->
    is_directory_inode g ce1.(ce_ino) ->
    ce1 = ce2 \/\ ce1.(ce_name) = ".."
    \/\ ce2.(ce_name) = ".".

```

11.6. Todos los teoremas finales ($X_preserves_WellFormed$) de los seis archivos `Dpo*.v`.

12.1. El *treewidth* es el mínimo k tal que existe una descomposición en árbol de bags de tamaño $\leq k + 1$. Sirve para que algoritmos NP-hard (3-coloring, max-clique) se vuelvan polinomiales en grafos con *treewidth* acotado.

12.2. $\alpha = 0.5$.

12.3. En un grafo triangular, $\deg u = \deg v = 2$, $|N(u) \cap N(v)| = 1$. $\kappa_{LLY} \approx 1/2 \cdot 0.25 = 0.125$; tras correcciones, suele estar en ~ 0.5 .

12.4. En hojas de un árbol, $\deg u = \deg v = 1$, $|N(u) \cap N(v)| = 0$. κ tiende a -1 (transport máximo).

12.5. Sin hard links, las aristas contains forman un árbol y los pointsTo aristas estrella desde Inode a sus Block. La tree decomposition: cada Directory es un bag con su Inode + los hijos directos; el ancho es ≤ 6 .

12.6. κ positivo y grande en el inode del directorio (sus vecindarios crecen masivamente). En las aristas contains salientes, κ depende de la similitud entre nombres; si son aleatorios, κ baja.

13.1. Passthrough, Restrict, Fabricate, Redirect.

13.2. El subgrafo inducido por BFS desde *root_dir* contiene todas las aristas contains entre nodos visibles. El link-count recalculado coincide con el número de contains entrantes en el subgrafo. Análogo para los otros 6 invariantes.

14.1. SOTFS_FUSE_TTL_MS, SOTFS_FUSE_ALLOW_OTHER, SOTFS_PROV_SIDEAR.

14.2. rename.

14.3.

```

since=$(date -d '1 hour ago' +%s)
sotfs-export-hunter --tail /tmp/sotfs.prov.jsonl --once | \
  jq -c "select(.op == \"Write\" and .t > $since) | .domain" | \
  sort | uniq -c

```

14.4. TTL=0 desactiva el cache de FUSE, forzando `lookup/getattr` en cada `syscall`. Útil para medir el costo bruto de cada DPO op sin amortización del kernel.

Ejercicios * * *: sólo hints

Para los ejercicios de tres estrellas, las pistas ya están en el cuerpo de la solución correspondiente o se enuncian en el texto del ejercicio. Quedan como problema abierto para el lector con tiempo.

Bibliografía

- [1] Jean-Philippe Aumasson, Samuel Neves, Zooko Wilcox-O’Hearn y Christian Winnerlein. *BLAKE3: One Function, Fast Everywhere*. Specification, <https://github.com/BLAKE3-team/BLAKE3-specs>. 2020.
- [2] Yves Bertot y Pierre Castéran. *Interactive Theorem Proving and Program Development: Coq’Art: The Calculus of Inductive Constructions*. Texts in Theoretical Computer Science. Springer, 2004.
- [3] Hans L. Bodlaender. «A Tourist Guide Through Treewidth». En: *Acta Cybernetica* 11.1–2 (1993), págs. 1-21.
- [4] Andrea Corradini, Tobias Heindel, Frank Hermann y Barbara König. «Sesqui-pushout Rewriting». En: *Graph Transformations — ICGT 2006*. Springer, 2006, págs. 30-45.
- [5] Hartmut Ehrig, Karsten Ehrig, Ulrike Prange y Gabriele Taentzer. *Fundamentals of Algebraic Graph Transformation*. EATCS Monographs in Theoretical Computer Science. Springer, 2006.
- [6] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.
- [7] Yong Lin, Linyuan Lu y Shing-Tung Yau. «Ricci Curvature of Graphs». En: *Tohoku Mathematical Journal* 63.4 (2011), págs. 605-627.
- [8] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh y Peter Schwarz. «ARIES: A Transaction Recovery Method Supporting Fine-granularity Locking and Partial Rollbacks Using Write-ahead Logging». En: *ACM Transactions on Database Systems* 17.1 (1992), págs. 94-162.
- [9] Helgi Sigurbjarnarson, James Bornholt, Emina Torlak y Xi Wang. «Push-Button Verification of File Systems via Crash Refinement». En: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 2016, págs. 1-16.